

Cybersecurity Rookie

Start from zero: how computers, networks, the web, Linux and Windows work, before touching security.

11 modules

60 lessons

45 simulated labs

11 module checkpoints

Best for: Complete beginners and career changers. No experience needed.

Needs: Nothing. This is the first path.

Contents

STAGE 1: HOW COMPUTERS AND NETWORKS WORK

ROOK-01	Welcome to Cybersecurity	p. 3
ROOK-02	How Computers Work	p. 17
ROOK-03	Networking Fundamentals	p. 34

STAGE 2: THE WEB AND YOUR FIRST OPERATING SYSTEM

ROOK-04	How the Web Works	p. 59
ROOK-05	Linux I: Meet the Terminal	p. 78
ROOK-06	Linux II: Files and Folders	p. 95
ROOK-07	Linux III: Users and Permissions	p. 112
ROOK-08	Linux IV: Pipes, Processes and Network	p. 129

STAGE 3: WINDOWS, SCRIPTING AND SECURITY BASICS

ROOK-09	Windows Fundamentals	p. 146
ROOK-10	Scripting Basics	p. 165
ROOK-11	Security Foundations	p. 184

Welcome to Cybersecurity

Beginner | 4 lessons | 3 simulated labs | checkpoint | about 3h

This module introduces the field you are entering. You will define cybersecurity in terms of protecting systems, data and people, learn how red, blue and purple teams divide the work, and see which job roles exist. You will study three famous breaches to understand how real failures happen, then learn the legal and ethical rules that separate a professional from a criminal. It matters because every later skill in this path, from networking to Linux, is used inside these boundaries.

BY THE END YOU CAN

- + Define cybersecurity and distinguish red, blue and purple team work
- + Match common job roles to their daily tasks
- + Apply basic legal and ethical rules to security testing

IN THIS MODULE

Lesson 1 What Cybersecurity Is and Why It Matters

Lesson 2 Red, Blue and Purple Teams, Job Roles

Lesson 3 Famous Breaches: Target, Equifax, SolarWinds

Lesson 4 Ethics, the Law and How to Study

Lab 1 Sort the Job Roles (Scenario)

Lab 2 Breach Timeline Triage (Log viewer)

Lab 3 Authorized or Not? (Chat sim)

Checkpoint 10 questions, 80% to pass

What Cybersecurity Is and Why It Matters

Everything else in this path is a means to one goal: keeping systems, data and people safe from misuse.

A working definition

Cybersecurity is the practice of protecting computer systems, networks, applications and the information they hold from unauthorized access, misuse, disruption or damage. The key word is practice: it is not a product you buy once but a continuous set of decisions about people, process and technology. Security professionals describe the goals with three words, confidentiality, integrity and availability. Data should be seen only by those allowed to see it, should not be changed without permission, and should be usable when needed. Later modules treat this CIA triad formally, but you will see it behind every example here.

What is being protected

Three kinds of assets are at stake. Systems include laptops, servers, phones, cloud accounts and the network gear that links them. Data includes customer records, passwords, source code, financial files and medical history. People are the third asset and are often forgotten: a leaked home address, a drained bank account or a hospital that cannot treat patients because its computers are locked all cause human harm. This is why security is not only a technical field. A well-patched server is still at risk if an employee can be tricked into sharing a password.

Why it matters now

Almost every organization depends on connected systems, so attackers have a very large number of targets and many motives. Criminals want money through fraud, ransomware and data theft. Some groups want secrets for governments, and others act for political reasons or simple disruption. Defenders must protect everything all the time, while an attacker needs only one working path. That imbalance explains why defenders use layers of controls, monitor for unusual behavior and plan for failure. Good security does not promise zero incidents. It reduces how often they happen, limits how far they spread and shortens the time to notice and recover.

Prevent, detect, respond

Security work falls into three broad activities. Prevention stops bad things from happening, for example by patching software, requiring strong authentication and limiting who can reach a service. Detection notices when prevention fails, using logs, alerts and monitoring tools. Response contains the damage, removes the attacker, restores systems and learns from the event. A mature program invests in all three. If you only prevent, you are blind when something slips through. If you only respond, you are always cleaning up. Keep this three-part pattern in mind, because job roles and tools in this field map onto it neatly.

TERMINOLOGY

Confidentiality : keep data private
Integrity : keep data accurate and unaltered
Availability : keep systems usable

Activities: Prevent -> Detect -> Respond

WORKED EXAMPLE

A small dental clinic stores patient records on one office server. One morning the front desk cannot open any file, and a message demands payment. Availability is lost because the files are unusable. If the attacker also copied the records first, confidentiality is lost too. The harm reaches people: patients miss appointments and their data may be sold. Prevention would have been patching and offline backups, detection would have been alerts on mass file changes, and response is isolating the server and restoring from clean backups. One incident touches every goal and every activity.

KEY TERMS

Cybersecurity: Protecting systems, networks and data from unauthorized access, misuse or damage.

Confidentiality: Ensuring information is accessible only to authorized parties.

Integrity: Ensuring information is accurate and not altered without permission.

Availability: Ensuring systems and data are usable when needed.

Ransomware: Malware that locks or encrypts data and demands payment to restore it.

REMEMBER

- Cybersecurity protects systems, data and people from misuse.
- Confidentiality, integrity and availability are the core goals.
- Prevent, detect and respond are the three activities of a security program.

QUICK CHECK

1. Which goal is violated when a ransomware attack makes files unreadable?
 - A) Confidentiality only
 - B) Availability
 - C) Non-repudiation
 - D) Authentication
2. Which activity notices that a control has failed?
 - A) Prevention
 - B) Detection
 - C) Procurement
 - D) Budgeting
3. Why is security described as a practice rather than a product?
 - A) Products never work
 - B) It is a continuous set of decisions about people, process and technology
 - C) It only applies to large firms
 - D) It is only about passwords
4. Which is an example of harm to people rather than only systems?
 - A) A server reboot
 - B) A leaked home address enabling stalking
 - C) A full disk
 - D) A slow network

Answers: 1-B 2-B 3-B 4-B

1. Encrypted files cannot be used, which is a loss of availability.
2. Detection uses logs and alerts to notice failed prevention.
3. Security needs ongoing effort across people, process and technology.
4. Exposure of personal data can directly endanger individuals.

Red, Blue and Purple Teams, Job Roles

Knowing who does what helps you choose a career direction and understand how organizations defend themselves.

Red team: thinking like an attacker

A red team simulates a real adversary to test whether an organization can withstand an attack. Members use the same tactics as criminals, such as phishing, exploiting weak configurations and moving across a network, but they do so with written permission and defined rules. Their goal is to show where defenses fail, not to cause harm. A related role is the penetration tester, who usually works on a narrower, time-boxed engagement against a specific application or network and produces a report of findings. Red teaming often runs longer and aims to test detection and response as well as technical controls.

Blue team: defending and detecting

The blue team defends. It builds and operates the controls that prevent attacks and watches for signs of compromise. Typical work includes configuring firewalls, collecting logs in a SIEM (security information and event management) platform, tuning alerts and hunting for hidden threats. The Security Operations Center, or SOC, is the group that monitors alerts around the clock. Tier 1 analysts triage alerts, Tier 2 analysts investigate in depth, and incident responders take over when an alert becomes a confirmed attack. Blue team members value patience, curiosity and careful note taking.

Purple team: working together

Red and blue teams can fall into a contest where each hides what it knows. A purple team is not usually a separate group but a way of working in which attackers and defenders share techniques openly. For example, the red team runs a specific technique, the blue team checks whether an alert fired, and together they improve the detection rule on the spot. Frameworks such as MITRE ATT&CK, a public catalog of attacker techniques, give both sides a common vocabulary. The result is faster improvement than a surprise test that ends in a report nobody acts on.

Common job roles

Beyond the colors, many roles exist. A SOC analyst triages alerts. A penetration tester performs authorized attacks and writes findings. An incident responder contains and investigates breaches. Governance, risk and compliance (GRC) staff write policies, measure risk and prepare audits. Security engineers design and build controls, and cloud security specialists protect cloud accounts. A digital forensics analyst recovers evidence. Many people start in IT support or system administration, because understanding how normal systems behave is the base for spotting abnormal ones. Pick a role by the daily tasks you enjoy, not only by the title.

ROLE TO TASK

```
SOC analyst -> triage alerts in the SIEM
Pentester -> authorized attack, written report
Incident resp. -> contain and investigate a breach
GRC -> policy, risk register, audit
Purple teaming -> attack and detect, tuned together
```

WORKED EXAMPLE

A company runs a phishing test. The red team sends a harmless fake email and records that 12 staff clicked. The blue team checks its email gateway and sees only two alerts fired. In a purple session, both teams review which message features were missed, and the blue analyst writes a new detection rule while the red operator replays the test to confirm it fires. A GRC specialist then adds mandatory phishing training to the policy. Each role contributed a different piece of one improvement loop.

KEY TERMS

Red team: Authorized group that emulates attackers to test an organization's defenses.

Blue team: Group that defends systems and detects and responds to attacks.

Purple team: A collaborative approach where red and blue share techniques to improve detection.

SOC: Security Operations Center, the team that monitors and triages security alerts.

GRC: Governance, risk and compliance, the policy and oversight side of security.

REMEMBER

- Red attacks with permission, blue defends and detects, purple shares knowledge.
- SOC analysts, pentesters, responders and GRC staff do distinct daily work.
- Many careers begin in IT support.

QUICK CHECK

1. Which role writes policies and prepares for audits?
 - A) SOC analyst
 - B) GRC specialist
 - C) Pentester
 - D) Network cabler
2. What distinguishes a purple team approach?
 - A) Attackers hide their methods
 - B) Red and blue share techniques openly to improve detection
 - C) It replaces the SOC
 - D) It avoids testing
3. Who first triages alerts in a SOC?
 - A) Tier 1 analyst
 - B) The CEO
 - C) Auditor
 - D) Developer
4. What is MITRE ATT&CK used for?
 - A) Encrypting disks
 - B) A shared catalog of attacker techniques
 - C) Billing
 - D) Cable standards

Answers: 1-B 2-B 3-A 4-B

1. GRC covers governance, risk and compliance work.
2. Purple teaming is collaboration to tune defenses.
3. Tier 1 analysts handle initial triage.
4. It is a public knowledge base of adversary tactics and techniques.

Famous Breaches: Target, Equifax, SolarWinds

Real incidents show how small weaknesses chain into large losses, and which basic controls would have limited the damage.

Target, 2013: a vendor as the way in

Attackers first stole credentials from a heating and air conditioning contractor that had network access to Target, a large US retailer. Using that vendor account they reached Target's network and eventually the point-of-sale systems, where malware captured payment card data from tens of millions of cards. Root causes included a supplier with more access than it needed, a network that was not well segmented between corporate and payment systems, and security alerts that were generated but not acted on. Controls that would have limited it are vendor access restrictions, network segmentation and a process to respond to alerts promptly.

Equifax, 2017: an unpatched flaw

Equifax, a credit reporting company, ran a public web application built on the Apache Struts framework. A serious vulnerability in Struts had been announced and a fix was available, but the application was not patched in time. Attackers used the flaw to enter and stayed for weeks, taking personal data of roughly 147 million people. Contributing problems included incomplete asset inventory, a scanner that missed the system and an expired certificate that reduced visibility into outbound traffic. The lesson is that patch management, asset tracking and working monitoring are basic controls with enormous impact.

SolarWinds, 2020: supply chain compromise

In this incident attackers broke into the build process of SolarWinds, a maker of network monitoring software, and inserted a backdoor into a legitimate update of its Orion product. Customers who installed the signed update unknowingly installed the backdoor, which gave the attackers a foothold in thousands of networks, including government agencies. This is a supply chain attack: the trusted channel itself was abused. Defenses include protecting build systems, verifying software integrity, limiting what monitoring tools can reach, and watching for unusual outbound connections from trusted servers. Detection took many months, showing the importance of monitoring.

Patterns across the three

Notice the shared themes. None began with a movie-style genius hack. Each exploited a trust relationship or an ordinary weakness: a vendor login, a missed patch, a trusted update. Each had a detection gap, meaning time passed between the attacker's first access and anyone noticing. Each also affected people who never made a mistake, such as shoppers and credit file holders. When you read breach reports, practice asking four questions. How did the attacker get in? How did they move? Why was it not detected sooner? Which basic control would have broken the chain?

BREACH TIMELINE PATTERN

Day 0 initial access (vendor login / unpatched web app / bad update)
 Day 1+ lateral movement inside the network
 Weeks data collected, no alert acted on
 Later outside party reports the breach
 Gap between Day 0 and detection = dwell time

WORKED EXAMPLE

Imagine a fictional retailer, ExampleMart. A refrigeration vendor account is phished and used to log in to a portal. From there the attacker reaches the card payment network because both share one flat network. Alerts about unusual file transfers sit unread for three weeks. Applying the four questions: entry was vendor credentials, movement was possible because of no segmentation, detection failed because alerts were ignored, and the best breaking controls were vendor multi-factor authentication, segmentation of payment systems and an on-call process for high-priority alerts.

KEY TERMS

Lateral movement: An attacker's movement from one compromised system to others inside a network.

Supply chain attack: Compromise of a trusted supplier or update channel to reach its customers.

Patch management: The process of tracking and applying software fixes in a timely way.

Network segmentation: Dividing a network into zones so a breach in one does not reach all.

Dwell time: The period between an attacker's initial access and its detection.

REMEMBER

- Real breaches chain ordinary weaknesses such as vendor access, missed patches and trusted updates.
- Detection gaps let attackers stay for weeks or months.
- Segmentation, patching and monitoring are basic controls with large impact.

QUICK CHECK

1. What was the initial access path in the Target breach?
 - A) A zero-day in the cash register
 - B) Credentials from an HVAC vendor
 - C) A stolen laptop
 - D) A DNS outage
2. Which basic control failure is most associated with Equifax?
 - A) Weak Wi-Fi
 - B) Unpatched Apache Struts flaw
 - C) No backups
 - D) Open printer
3. What makes SolarWinds a supply chain attack?
 - A) It used phishing only
 - B) A legitimate software update was trojanized
 - C) It targeted one laptop
 - D) It used USB drives
4. What does dwell time measure?
 - A) Time between initial access and detection
 - B) Download speed
 - C) Patch size
 - D) Login duration

Answers: 1-B 2-B 3-B 4-A

1. Attackers used a third-party contractor's credentials.
2. A known vulnerability was left unpatched.
3. The trusted update channel carried the backdoor.
4. Dwell time is how long an attacker stays unnoticed.

Ethics, the Law and How to Study

Security knowledge can harm if misused, so you must understand authorization, scope and the law before you practice.

Authorization is everything

The line between security testing and a crime is permission. Accessing a computer or network you do not own, without authorization from the owner, can be illegal even if your intent is curious or helpful. Professionals work under a written agreement that states who authorizes the test, which systems are included, what methods are allowed and what time window applies. This is called the scope. Anything outside scope is off limits. If a tester notices an interesting system that is not listed, the correct action is to stop and ask, not to look. Verbal approval is not enough; get it in writing from someone with authority.

Laws you should know exist

Many countries criminalize unauthorized access. In the United States the Computer Fraud and Abuse Act, or CFAA, is the main federal law, and the United Kingdom has the Computer Misuse Act. Other laws cover interception of communications, theft of data and privacy, such as GDPR in the European Union. Details differ by country and change over time, and this book is not legal advice. The practical rule is simple: only test systems you own or have explicit written permission to test, and use practice environments designed for learning. Intent to learn does not replace authorization in the eyes of the law.

Responsible disclosure

Sometimes you find a flaw by accident in a system you do not own. Responsible, or coordinated, disclosure is the accepted approach: report the issue privately to the owner, give reasonable time to fix it, and avoid publishing details that help attackers before a fix exists. Many organizations publish a security.txt file or run a bug bounty program that states how to report and what testing is allowed. Always read the program rules, because even a bounty has scope. Do not access more data than needed to prove the problem, and never keep or share personal data you encounter.

Building a study routine

Skills in this field come from repeated practice, not passive reading. Plan short, regular sessions, for example 45 minutes five days a week, rather than one long weekend. For each topic, read, then do a hands-on exercise, then explain it in your own words. Keep notes with commands and mistakes. Build a home lab using virtual machines so you can break things safely. Review earlier modules with spaced repetition, revisiting them after a day, a week and a month. Join communities, but never ask for or share help attacking systems you lack permission to test.

SCOPE STATEMENT (EXAMPLE)

```
Client: Example Corp (fictional)
Authorized: Written approval signed by CIO
In scope: 203.0.113.0/28 and www.example.com
Out of scope: all other hosts, social engineering
Window: Mon 09:00 - Fri 17:00 UTC
```

WORKED EXAMPLE

A freelancer is hired to test www.example.com. While scanning, she notices that 198.51.100.7, owned by another company, responds with an admin login page. It is not in the scope document. She does not touch it. She notes the observation, tells her client contact, and asks whether the owner should be informed. The client confirms the host is a different organization's, so the contact reports it through the owner's published channel. By stopping at the scope line and documenting, she stays legal and protects her client and her own reputation.

KEY TERMS

Scope: The written list of systems, methods and times a tester is allowed to use.

Authorization: Explicit permission from the system owner to perform a test.

CFAA: The US Computer Fraud and Abuse Act, a federal law against unauthorized computer access.

Responsible disclosure: Privately reporting a vulnerability to its owner and allowing time to fix it.

Bug bounty: A program that rewards researchers for reporting vulnerabilities within stated rules.

REMEMBER

- Only test systems you own or have written permission to test.
- Scope defines what is allowed; stop at its edge.
- Report found flaws privately and study with regular hands-on practice.

QUICK CHECK

1. What most clearly separates a pentest from a crime?
 - A) Tool choice
 - B) Written authorization and scope
 - C) Time of day
 - D) Operating system
2. You find an out-of-scope host during a test. What should you do?
 - A) Test it quickly
 - B) Stop and ask the client
 - C) Publish it online
 - D) Ignore the contract
3. What is responsible disclosure?
 - A) Selling the flaw
 - B) Privately reporting the flaw and allowing time to fix
 - C) Posting exploit code immediately
 - D) Keeping it secret forever
4. Which study habit is best supported for skill building?
 - A) One long monthly session
 - B) Short regular sessions with hands-on practice
 - C) Only watching videos
 - D) Memorizing without practice

Answers: 1-B 2-B 3-B 4-B

1. Permission and agreed scope make testing lawful.
2. Out-of-scope systems must not be touched without approval.
3. It means private reporting and a reasonable fix window.
4. Regular hands-on practice with review builds durable skills.

Sort the Job Roles

SCENARIO

You are shadowing a security team at the fictional firm Example Corp for a day. At the end of the shift, six short task cards describe what staff members actually did. The simulator lists four possible roles: SOC analyst, pentester, GRC, and incident responder. Your job is to read each card carefully, think about whether the work is monitoring, authorized attacking, policy and risk paperwork, or breach cleanup, and then assign each card to the single role that best fits.

OBJECTIVE

Assign each of six tasks to the correct security role.

STEPS IN THE SIMULATOR

- 1 Do:** Open card 1: 'Reviewed 40 SIEM alerts and closed 36 as false positives.'

Simulator shows:

Role choices shown: SOC analyst, pentester, GRC, incident responder.

- 2 Do:** Assign card 1 to SOC analyst.

Simulator shows:

Correct. Alert triage is SOC work.

- 3 Do:** Open card 2: 'Wrote a report after testing the web portal with client approval.'

Simulator shows:

Card 2 text shown with the four role buttons.

- 4 Do:** Assign card 2 to pentester.

Simulator shows:

Correct. Authorized testing with a findings report.

- 5 Do:** Assign card 3 'Updated the risk register and prepared for the ISO audit' to GRC.

Simulator shows:

Correct. Policy, risk and audit are GRC.

- 6 Do:** Assign card 4 'Isolated an infected laptop and collected memory' to incident responder.

Simulator shows:

Correct. Containment and evidence collection.

- 7 Do:** Assign cards 5 and 6: 'Tuned a detection rule' to SOC analyst and 'Ran a phishing simulation under a signed scope' to pentester.

Simulator shows:

Both correct. Score: 6 of 6.

ANSWER

SOC analyst: alert triage and rule tuning. Pentester: authorized testing and phishing simulation under scope. GRC: risk register and audit preparation. Incident responder: isolation and evidence collection. Each role maps to prevention, detection, response or oversight.

Hint shown in the app: Ask whether the task is about watching, attacking with permission, paperwork, or cleaning up after an attack.

Breach Timeline Triage

SCENARIO

You receive an exported timeline log from a fictional breach at a retailer called ExampleMart. The events are listed in time order but carry no labels, and the incident team needs a quick summary before a morning briefing. Using the log viewer, you must mark the initial access event, the lateral movement events, and the point where detection failed, then work out how many days passed between first entry and discovery by an outside party.

OBJECTIVE

Label initial access, lateral movement and the detection gap in the timeline.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the timeline log.

Simulator shows:

Shows 7 entries from 03 Mar to 28 Mar.

- 2 **Do:** Read entry 1: '03 Mar 02:14 vendor-hvac login from 203.0.113.50 to portal.example.com'.

Simulator shows:

Note: first sign of outside access using a vendor account.

- 3 **Do:** Mark entry 1 as Initial Access.

Simulator shows:

Marked. The vendor credential was the entry.

- 4 **Do:** Read entries 2 and 3: '03 Mar 02:40 portal host connects to 10.0.5.20 over SMB' and '04 Mar 11:05 10.0.5.20 connects to pos-db 10.0.9.4'.

Simulator shows:

Both show internal hops.

- 5 **Do:** Mark entries 2 and 3 as Lateral Movement.

Simulator shows:

Marked. The attacker moved from portal to payment database.

- 6 **Do:** Read entry 5: '12 Mar alert: large outbound transfer to 198.51.100.9, status: unreviewed'.

Simulator shows:

Alert exists but nobody acted.

- 7 **Do:** Mark entry 5 as Detection Gap and entry 7 '28 Mar bank reports fraud' as External Detection.

Simulator shows:

Marked. Dwell time: 25 days.

ANSWER

Initial access: vendor login on 03 Mar. Lateral movement: portal to file server to payment database on 03-04 Mar. Detection gap: the unreviewed outbound transfer alert on 12 Mar. Detection came from the bank on 28 Mar, 25 days after entry.

Hint shown in the app: Initial access is the first event from outside; the gap is an alert that was ignored.

Authorized or Not?

SCENARIO

You act as a junior tester in a chat with a fictional client contact named Dana at Example Corp. The signed scope document covers only the address block 203.0.113.0/28 and excludes social engineering. During the engagement Dana sends four requests, some reasonable and some not. For each one you must decide whether it is already in scope, whether it needs written authorization before you act, or whether you must refuse it because the client cannot grant permission.

OBJECTIVE

Classify each client request as in scope, needs written authorization, or refuse.

STEPS IN THE SIMULATOR

- 1 **Do:** Read Dana's request 1: 'Please scan 203.0.113.5.'

Simulator shows:

Chat shows the scope document: 203.0.113.0/28 authorized.

- 2 **Do:** Choose 'In scope' for request 1.

Simulator shows:

Correct. 203.0.113.5 is inside the /28.

- 3 **Do:** Read request 2: 'Also test our payroll server at 203.0.113.200.'

Simulator shows:

Not in the signed scope.

- 4 **Do:** Choose 'Needs written authorization' for request 2.

Simulator shows:

Correct. Ask for a scope amendment before testing.

- 5 **Do:** Read request 3: 'Test our partner's site at 198.51.100.20, they will not mind.'

Simulator shows:

Dana does not own that host.

- 6 **Do:** Choose 'Refuse' for request 3.

Simulator shows:

Correct. Only the owner can authorize testing.

- 7 **Do:** Read request 4: 'Send phishing emails to staff.' Choose 'Needs written authorization'.

Simulator shows:

Correct. Social engineering was listed out of scope and needs an amendment.

ANSWER

Request 1 is in scope. Requests 2 and 4 need written scope amendments before any action. Request 3 must be refused because the client cannot authorize testing of a third party's system.

Hint shown in the app: Check the host against the signed scope, then check who actually owns it.

Welcome to Cybersecurity: 10 questions, 80% to pass

1. Which set best describes the CIA triad?
 - A) Control, Inspect, Alert
 - B) Confidentiality, Integrity, Availability
 - C) Cloud, Internet, Access
 - D) Code, Identity, Audit
2. A team sends harmless phishing emails with written approval to test defenses. Which team is this?
 - A) Red team
 - B) Blue team
 - C) Help desk
 - D) Legal
3. Which role monitors alerts in a SIEM around the clock?
 - A) GRC auditor
 - B) SOC analyst
 - C) Developer
 - D) Cabling technician
4. What is a purple team approach?
 - A) Red and blue collaborating to tune detections
 - B) A third attacker group
 - C) A compliance audit
 - D) A backup method
5. What allowed the Target attackers to get in?
 - A) A vendor's stolen credentials
 - B) A broken cable
 - C) A DNS typo
 - D) A lost USB stick
6. What was the key weakness in the Equifax breach?
 - A) An unpatched web framework flaw
 - B) No firewall at all
 - C) Weak Wi-Fi
 - D) A stolen badge
7. What makes the SolarWinds attack a supply chain attack?
 - A) It exploited a trusted software update
 - B) It used ransomware
 - C) It targeted home users
 - D) It used SMS
8. What is the best definition of scope?
 - A) The written list of authorized targets, methods and times
 - B) A type of malware
 - C) A firewall rule
 - D) A report format
9. In the US, which law is the main federal statute on unauthorized computer access?
 - A) CFAA
 - B) HIPAA only
 - C) PCI only
 - D) SOX
10. You discover a flaw in a stranger's website by accident. What is the best action?
 - A) Report privately to the owner
 - B) Post details publicly
 - C) Exploit it further
 - D) Sell it

Answers: 1-B 2-A 3-B 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. CIA stands for confidentiality, integrity and availability.
2. Authorized attacker emulation is red team work.
3. SOC analysts watch and triage alerts.
4. Purple teaming is shared learning between attackers and defenders.
5. A third-party HVAC vendor account was the entry point.
6. A known Apache Struts flaw was not patched.
7. Attackers poisoned a legitimate update.
8. Scope states exactly what testing is allowed.
9. The Computer Fraud and Abuse Act covers it.
10. Coordinated private reporting is the accepted approach.

How Computers Work

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

Before you can secure a computer you need a clear picture of what is inside one. This module covers the CPU, memory and storage, the operating system kernel, and how numbers and text are represented in binary, hex and ASCII. You will learn how files identify themselves, how running programs become processes, and how virtual machines isolate work. These ideas explain why malware behaves as it does and why analysts do their work inside disposable virtual labs.

BY THE END YOU CAN

- + Convert between binary, decimal, hex and ASCII
- + Identify the roles of CPU, RAM, storage and the OS kernel
- + Describe processes, file types and virtual machine isolation

IN THIS MODULE

Lesson 1 CPU, RAM and Storage

Lesson 2 Operating Systems and the Kernel

Lesson 3 Binary, Hex and ASCII

Lesson 4 Files, Extensions and Processes

Lesson 5 Virtual Machines and Hypervisors

Lab 1 Binary and Hex Converter Drill (Scenario)

Lab 2 Spot the Fake Extension (File explorer)

Lab 3 Process Tree Walkthrough (Terminal)

Lab 4 Design a Safe Analysis Lab (Diagram builder)

Checkpoint 10 questions, 80% to pass

CPU, RAM and Storage

Every attack and every defense plays out on three resources: processing, memory and storage.

The CPU executes instructions

The central processing unit, or CPU, is the component that executes instructions. A program is ultimately a long list of simple operations such as add, compare, move data and jump to another instruction. The CPU fetches an instruction, decodes it, executes it and repeats this cycle billions of times per second. Speed is measured in gigahertz, and modern chips contain several cores, each able to run its own stream of instructions. Cores let a computer run many programs at once. For security, the key point is that the CPU runs whatever instructions it is given, which is why untrusted code is dangerous.

RAM: fast but temporary

Random access memory, or RAM, is the working space where the CPU keeps the programs and data it is using right now. It is fast, measured in nanoseconds, but volatile, meaning its contents vanish when power is lost. A running program, a document being edited and a password typed into a login form all exist in RAM. This has a security side: sensitive data in memory can be read by other code with enough privilege, and investigators capture a memory image from a live machine because it holds evidence, such as running malware and network connections, that never reaches disk.

Storage: persistent but slower

Storage keeps data when the power is off. A hard disk drive, or HDD, stores bits magnetically on spinning platters and is cheap but slow. A solid state drive, or SSD, uses flash memory with no moving parts and is much faster. Both hold the operating system, applications and user files. Storage is divided into partitions and organized by a file system such as NTFS, ext4 or APFS. Because storage persists, it is where attackers plant malware that must survive a reboot, and where defenders look for logs. Full disk encryption protects stored data if a laptop is stolen.

How they cooperate

When you launch a program, the operating system copies its instructions from storage into RAM. The CPU then fetches those instructions from RAM, often via small, very fast caches built into the chip. Results are held in RAM until you save, which writes them back to storage. If RAM fills up, the system may use swap or a page file on disk, which is much slower and can leave fragments of memory on disk. Remember the hierarchy: registers and cache are fastest and smallest, RAM is next, and storage is the slowest and largest.

MEMORY HIERARCHY

```
Registers/cache : fastest, tiny, volatile
RAM : fast, GBs, volatile
SSD : slower, 100s of GBs, persistent
HDD : slowest, TBs, persistent
```

WORKED EXAMPLE

You open a spreadsheet on a laptop. The operating system reads the file from the SSD into RAM, and the CPU runs the spreadsheet program's instructions against that data. You type new numbers, which exist only in RAM until you press save. Suddenly the battery dies. The unsaved edits are gone because RAM is volatile, but the older saved file remains on the SSD. A forensic examiner would find a different story on a live, infected machine: the malware process, visible only in RAM, would vanish on shutdown, so capturing memory first preserves that evidence.

KEY TERMS

CPU: The processor that fetches, decodes and executes program instructions.

RAM: Fast, volatile memory holding data and programs currently in use.

Volatile: Losing contents when power is removed.

SSD: Solid state drive using flash memory, faster than an HDD.

Swap: Disk space used as overflow when RAM is full.

REMEMBER

- The CPU executes instructions; RAM holds what is in use; storage persists data.
- RAM is fast and volatile, while storage is slower and persistent.
- Memory holds evidence that disk may not.

QUICK CHECK

1. Which component loses its data when power is cut?
 - A) SSD
 - B) HDD
 - C) RAM
 - D) USB stick
2. Why do forensic investigators often capture RAM from a live machine?
 - A) It is cheap
 - B) It holds evidence that never reaches disk
 - C) It is permanent
 - D) It encrypts files
3. What is the main difference between HDD and SSD?
 - A) HDD uses flash memory
 - B) SSD has no moving parts and is faster
 - C) SSD is volatile
 - D) HDD is faster
4. When you open a program, where does the OS first load its instructions to run them?
 - A) Into RAM
 - B) Into the monitor
 - C) Into the network card
 - D) Into the BIOS

Answers: 1-C 2-B 3-B 4-A

1. RAM is volatile.
2. Running processes and connections exist only in memory.
3. SSDs use flash memory without spinning platters.
4. Programs are copied from storage into RAM for the CPU.

Operating Systems and the Kernel

The operating system decides what every program may do, so it is the main security boundary on a computer.

What an operating system does

An operating system, or OS, manages the hardware and gives programs a safe, consistent way to use it. It schedules the CPU among many programs, allocates RAM, manages files on storage, controls devices and enforces which users may do what. Without an OS each application would need to talk to every disk and network card directly. The OS also provides the user interface, whether a graphical desktop or a command line. The main families are Windows, macOS, Linux and Unix variants, and the mobile systems Android, which is Linux based, and iOS.

Kernel space and user space

The kernel is the core of the OS and runs with the highest privilege, directly controlling hardware and memory. Ordinary applications run in user space with restricted privilege. Modern CPUs enforce this split with privilege rings: the kernel runs in ring 0 and applications in ring 3. A program in user space cannot read another program's memory or touch hardware directly. This separation means a crash or compromise in one application should not take down the whole system. It also explains why attackers prize kernel-level access: code running in the kernel can hide files, processes and network connections from the rest of the system.

System calls and drivers

When an application needs something privileged, such as opening a file or sending a packet, it asks the kernel through a system call. On Linux examples include open, read, write and fork. The kernel checks permissions, performs the action and returns the result. Drivers are kernel components that know how to talk to a specific device such as a graphics card or network adapter. Because drivers run with high privilege, a flawed or malicious driver is dangerous, which is why modern Windows requires signed drivers. Monitoring system calls is a common technique defenders use to observe what a program is really doing.

Main OS families

Windows dominates corporate desktops and uses the NT kernel with the NTFS file system and a central configuration database called the registry. Linux is a free kernel combined with tools in distributions such as Debian, Ubuntu and Red Hat, and it powers most servers, cloud workloads and network devices. macOS is built on a Unix foundation called Darwin. Each family has its own file layout, user model and tools, so a security professional learns the basics of all three. Later modules in this path cover Linux and Windows in detail, starting from the concepts you learned here.

PRIVILEGE RINGS

```
Ring 3 user space : browsers, editors, scripts
Ring 0 kernel : scheduler, memory, drivers

App -> system call (open, read, write) -> kernel -> hardware
```

WORKED EXAMPLE

A text editor wants to save a file. It cannot write to the disk itself because it runs in user space. It issues a write system call. The kernel checks that the logged-in user has permission to modify that folder. If the check passes, the kernel instructs the disk driver to store the bytes. If the user lacks permission, the kernel returns an error such as permission denied and the editor shows a message. This checkpoint, repeated for every file, network and device request, is the foundation of OS security.

KEY TERMS

Kernel: The core of the OS that controls hardware, memory and processes at the highest privilege.

User space: The restricted environment where ordinary applications run.

System call: A request from an application asking the kernel to perform a privileged action.

Driver: Kernel-level software that lets the OS communicate with a hardware device.

Distribution: A packaged Linux system combining the kernel with tools and software.

REMEMBER

- The OS schedules resources and enforces permissions.

- Kernel space is privileged; user space is restricted.
- System calls are the controlled gateway between apps and the kernel.

QUICK CHECK

1. Which runs with the highest privilege?
 - A) A web browser
 - B) The kernel
 - C) A text editor
 - D) A game
2. How does an app ask the OS to open a file?
 - A) By a system call
 - B) By editing RAM directly
 - C) By rebooting
 - D) By overwriting the BIOS
3. Why are drivers a security concern?
 - A) They run with high privilege
 - B) They are slow
 - C) They are visual
 - D) They use no memory
4. Android is based on which kernel?
 - A) Windows NT
 - B) Linux
 - C) Darwin
 - D) DOS

Answers: 1-B 2-A 3-A 4-B

1. The kernel runs in ring 0.
2. System calls are the controlled gateway to the kernel.
3. Drivers run in kernel mode, so flaws or malicious drivers are powerful.
4. Android uses the Linux kernel.

Binary, Hex and ASCII

Analysts read addresses, packet bytes and file headers in hex, so converting by hand is a daily skill.

Binary and decimal

Computers store everything as bits, each either 0 or 1. Binary is base 2, so each position is a power of two. Reading from the right, the places are 1, 2, 4, 8, 16, 32, 64 and 128 for one byte of eight bits. To convert 01000001 to decimal, add the places that hold a 1: 64 plus 1 equals 65. To convert decimal to binary, subtract the largest power of two that fits and repeat. For example 200 is 128 plus 64 plus 8, which gives 11001000. One byte can represent 256 values, from 0 to 255.

Hexadecimal

Hexadecimal is base 16 and uses digits 0 to 9 and letters A to F, where A is 10 and F is 15. It is popular because one hex digit equals exactly four bits, so one byte is two hex digits. Binary 1111 0000 is hex F0. Hex is written with a prefix 0x or a suffix h in many tools. To convert hex to decimal multiply each digit by its place value: 0x2F is 2 times 16 plus 15, which equals 47. MAC addresses, IPv6 addresses, color codes, memory addresses and file signatures are all shown in hex, so fluency pays off immediately.

ASCII and text

ASCII is a table that maps numbers to characters. Capital A is 65, which is 0x41, lowercase a is 97, which is 0x61, and the digit 0 is 48, or 0x30. A space is 32, or 0x20, and a newline is 10, or 0x0A. To decode bytes 48 69 21 in hex, look up each: 0x48 is H, 0x69 is i and 0x21 is an exclamation mark, giving Hi!. Modern text often uses UTF-8, which is compatible with ASCII for the first 128 values but can use several bytes for other characters. Encoding is not encryption: anyone can reverse it.

Why this matters for security

Security tools constantly show raw bytes. A packet capture displays payloads in hex beside their ASCII rendering, and an analyst can spot plain text such as a username in the right-hand column. Malware authors may hide strings by encoding them, so recognizing that 0x41 is A helps you decode simple tricks. Network masks, IP addresses and permissions also rely on binary reasoning. A subnet mask of 255.255.255.0 is simply 24 ones followed by 8 zeros. Practice until the powers of two up to 128 and the hex digits A to F feel automatic.

HEX DUMP

```
Offset Hex ASCII
0000 48 65 6C 6C 6F 20 53 65 Hello Se
0008 63 21 c!

0x48 = 0100 1000 = 72 = H
```

WORKED EXAMPLE

Decode the byte string 53 45 43 with a simple method. Each pair is hex. 0x53 is 5 times 16 plus 3, which equals 83, and ASCII 83 is S. 0x45 is 4 times 16 plus 5, which equals 69, the letter E. 0x43 is 67, the letter C. The result is SEC. Now convert in the other direction: the letter a is decimal 97. Subtract 64 to get 33, subtract 32 to get 1, and so 97 is 64 plus 32 plus 1, which is binary 01100001, or 0x61.

KEY TERMS

Bit: A single binary digit, 0 or 1.

Byte: Eight bits, able to hold 256 values.

Hexadecimal: Base 16 notation where one digit represents four bits.

ASCII: A character encoding table mapping numbers 0 to 127 to characters.

UTF-8: A variable-length text encoding compatible with ASCII for the first 128 values.

REMEMBER

- Binary is base 2, hex is base 16, and one hex digit equals four bits.
- ASCII maps numbers to characters, for example 0x41 is A.
- Encoding is reversible and is not encryption.

QUICK CHECK

1. What is binary 00001010 in decimal?
 - A) 8
 - B) 10
 - C) 12
 - D) 5
2. What is 0xFF in decimal?
 - A) 155
 - B) 256
 - C) 255
 - D) 15
3. Which ASCII character has hex value 0x41?
 - A) a
 - B) A
 - C) 1
 - D) !
4. How many hex digits represent one byte?
 - A) 1
 - B) 2
 - C) 4
 - D) 8

Answers: 1-B 2-C 3-B 4-B

1. 8 plus 2 equals 10.
2. 15 times 16 plus 15 equals 255.
3. 0x41 is decimal 65, capital A.
4. Each hex digit is four bits, so a byte is two.

Files, Extensions and Processes

Attackers disguise files and hide in processes, so you must know how to check what a file or a running program really is.

File extensions versus signatures

A file extension such as .pdf or .jpg is just part of the name, and anyone can change it. The real type of a file is indicated by its signature, also called magic bytes, a short fixed pattern at the start of the content. A PDF begins with 25 50 44 46, which is the text %PDF. A PNG begins with 89 50 4E 47. A ZIP archive, and therefore modern Office documents, begins with 50 4B. A Windows executable begins with 4D 5A, the letters MZ, and a Linux executable begins with 7F 45 4C 46, which spells ELF after a control byte. The Linux file command reads these signatures.

Why mismatches matter

If a file named invoice.pdf begins with 4D 5A, it is a Windows program wearing a disguise. Attackers rename executables, or use double extensions such as report.pdf.exe, hoping users trust the visible name. Windows may hide known extensions by default, which makes this trick work. Defenders check signatures with tools, and mail gateways and scanners compare the claimed type with the real one. When examining a suspicious file, never rely on the icon or extension. Check the header, compute a cryptographic hash such as SHA-256 and look up that hash in a reputation service rather than opening the file.

Programs and processes

A program is a file on disk containing instructions. A process is a running instance of that program, with its own memory, open files and an identity. Starting a program twice creates two processes. Each has a process ID, or PID, a number assigned by the kernel. Processes also have an owner, the user account that started them, and that owner determines what the process may access. Tools such as ps and top on Linux, and Task Manager on Windows, list processes. Seeing which process owns a network connection is a core investigative step.

Parent and child processes

Every process except the first is started by another process, its parent, and the child records the parent's PID, or PPID. On Linux the first process is init or systemd with PID 1. When you open a terminal and run a command, the shell is the parent and the command is the child. Parent-child relationships are a powerful detection clue. A word processor launching a command shell is unusual and often signals a malicious document. Defenders learn the normal process tree, for instance services spawning from the service manager, and investigate deviations.

TERMINAL OUTPUT

```
$ file invoice.pdf
invoice.pdf: PE32 executable (GUI) Intel 80386, for MS Windows
$ xxd -l 4 invoice.pdf
00000000: 4d5a 9000 MZ..
$ ps -o pid,ppid,user,comm
  PID PPID USER COMMAND
 2210 2190 alice  bash
 2301 2210 alice  sleep
```

WORKED EXAMPLE

An employee receives report.pdf and reports that it opens nothing. An analyst checks the first bytes and sees 4D 5A, the MZ signature, so the file is a Windows executable renamed as a PDF. The analyst records the SHA-256 hash, which matches a known downloader in a reputation database. On the employee's PC, the process list shows report.pdf with parent PID belonging to the mail client, an unexpected relationship. The analyst isolates the host, kills the process and searches the mail server for other recipients of the same attachment.

KEY TERMS

Magic bytes: A fixed byte pattern at the start of a file that identifies its real format.

Process: A running instance of a program with its own memory and identity.

PID: The numeric process ID assigned by the kernel.

PPID: The process ID of the parent that started a process.

Hash: A fixed-length fingerprint of data used to identify identical files.

REMEMBER

- Extensions can be faked; file signatures reveal the real type.
- A process is a running program with a PID and a parent PPID.
- Unusual parent-child relationships are strong detection clues.

QUICK CHECK

1. Which bytes begin a Windows executable?
 - A) 25 50 44 46
 - B) 4D 5A
 - C) 89 50 4E 47
 - D) 7F 45 4C 46
2. What best identifies the real type of a file?
 - A) Its extension
 - B) Its icon
 - C) Its signature bytes
 - D) Its size in MB
3. What is the difference between a program and a process?
 - A) None
 - B) A program is a file; a process is a running instance
 - C) A process is a file
 - D) A program only runs once
4. What is unusual about a word processor spawning a command shell?
 - A) It is normal
 - B) It may indicate a malicious document
 - C) It means a hardware fault
 - D) It is a printer driver

Answers: 1-B 2-C 3-B 4-B

1. MZ, hex 4D 5A, starts a Windows PE file.
2. Magic bytes show the true format.
3. A process is a program in execution.
4. Office apps rarely launch shells, so this is a detection clue.

Virtual Machines and Hypervisors

Virtual machines let you run risky samples and build labs without endangering your real computer.

What a virtual machine is

A virtual machine, or VM, is a complete software-defined computer that runs inside a physical one. It has its own virtual CPU, RAM, disk and network card, and runs its own guest operating system. The physical computer is the host. Several VMs can share one host, each isolated from the others. Because the VM disk is usually just a file, you can copy, move or delete it easily. Cloud providers rent VMs by the hour, and organizations use them to consolidate servers. For security learners, VMs are the safe practice room: you can break a system and rebuild it in minutes.

Hypervisors: Type 1 and Type 2

The software that creates and runs VMs is the hypervisor. A Type 1 hypervisor, also called bare metal, runs directly on the hardware and is used in data centers. Examples are VMware ESXi, Microsoft Hyper-V and KVM. A Type 2 hypervisor runs as an application on top of a normal operating system and is common on laptops. Examples are VirtualBox and VMware Workstation. Type 1 generally offers better performance and a smaller attack surface, while Type 2 is convenient for learning. Both share the host CPU and RAM among guests by allocating slices.

Snapshots and networking

A snapshot saves a VM's state at a moment in time, including its disk and optionally its memory. You can take a snapshot before running a risky test and revert to the clean state afterward, which undoes any damage. Virtual networks can be configured in modes. NAT lets the VM reach the internet through the host. Bridged mode places the VM directly on your real network. Host-only mode connects the VM only to the host, and internal mode connects VMs to each other only. For malware analysis, a host-only or internal network with no route to your home network is the safe choice.

Isolation and its limits

Hypervisors enforce isolation so that a guest cannot read the memory of the host or another guest. Isolation is strong but not perfect. Rare flaws called VM escapes have allowed code in a guest to break out, and shared folders, clipboard sharing and drag and drop can leak data or malware between guest and host. Good practice for analysis labs is to disable shared folders and clipboard, use a dedicated lab network, keep the hypervisor patched, take a clean snapshot first and avoid placing personal data in the guest. Never assume a VM makes a dangerous sample harmless.

VM LAB SETTINGS

```
VM name: analysis-01
Network mode: Host-only (no NAT, no bridge)
Shared folders: disabled
Clipboard: disabled
Snapshot: clean-baseline (taken before sample)
```

WORKED EXAMPLE

A student wants to examine a suspicious file. She creates a VM, installs a fresh guest OS and takes a snapshot called clean-baseline. She sets the network to host-only so the sample cannot reach her home router, and disables shared folders and the clipboard. She copies the file in through a controlled method and runs it. Afterward she reverts to the snapshot, which restores the clean state in seconds. She also keeps the hypervisor updated, because the isolation she depends on is only as strong as that software.

KEY TERMS

Hypervisor: Software that creates and runs virtual machines.

Type 1 hypervisor: A bare-metal hypervisor that runs directly on hardware.

Type 2 hypervisor: A hypervisor that runs as an application inside a host operating system.

Snapshot: A saved VM state that can be restored later.

VM escape: A flaw that lets code in a guest break out to the host.

REMEMBER

- VMs are isolated virtual computers running on a host.

- Type 1 runs on hardware; Type 2 runs on an OS.
- Use snapshots and host-only networks, and disable sharing, for safe analysis.

QUICK CHECK

1. Which is a Type 1 hypervisor?
 - A) VirtualBox
 - B) VMware ESXi
 - C) A web browser
 - D) Notepad
2. What does a snapshot allow you to do?
 - A) Speed up the CPU
 - B) Revert a VM to an earlier state
 - C) Encrypt the host
 - D) Delete the hypervisor
3. Which network mode best keeps a malware sample away from your home network?
 - A) Bridged
 - B) Host-only
 - C) NAT with sharing
 - D) Public
4. Which setting should be disabled in a malware lab?
 - A) Shared folders and clipboard
 - B) The screen
 - C) The keyboard
 - D) The snapshot

Answers: 1-B 2-B 3-B 4-A

1. ESXi runs directly on hardware.
2. Snapshots restore a known-good state.
3. Host-only gives no route to other networks.
4. These channels can leak data or malware to the host.

Binary and Hex Converter Drill

SCENARIO

You are working through a conversion worksheet in a practice simulator for a junior analyst. The worksheet gives you ten values to convert among binary, decimal and hex, then a short string of hex bytes that you must decode character by character using an ASCII table shown on the side. The simulator checks every entry and shows your running score, so you can see where your method needs correction before moving to harder material such as packet payloads.

OBJECTIVE

Convert values among binary, decimal and hex, and decode a short ASCII string.

STEPS IN THE SIMULATOR

- 1 **Do:** Enter the decimal value of binary 00001101.

Simulator shows:
Correct: 13.

- 2 **Do:** Enter the binary form of decimal 200.

Simulator shows:
Correct: 11001000.

- 3 **Do:** Enter the decimal value of hex 0x2F.

Simulator shows:
Correct: 47.

- 4 **Do:** Enter the hex form of decimal 255.

Simulator shows:
Correct: 0xFF.

- 5 **Do:** Enter the decimal form of binary 11110000, then its hex form.

Simulator shows:
Correct: 240 and 0xF0. Score: 5 of 5 conversions.

- 6 **Do:** Open the ASCII table and look up 0x53, 0x45 and 0x43.

Simulator shows:
Table shows S, E and C.

- 7 **Do:** Type the decoded string for 53 45 43 20 31.

Simulator shows:
Correct: SEC 1 (0x20 is a space, 0x31 is the digit 1). Worksheet complete.

ANSWER

13, 11001000, 47, 0xFF, 240 and 0xF0 for the conversions. The byte string 53 45 43 20 31 decodes to SEC 1, because 0x20 is a space and 0x31 is the digit 1. Each hex digit is four bits, and a byte is two hex digits.

Hint shown in the app: Convert hex pairs to decimal first, then look up the number in the ASCII table.

Spot the Fake Extension

SCENARIO

A fictional user, bob, reports odd files in his Downloads folder at Example Corp. The file explorer lets you view each file's first bytes in a header panel beside its name. Four files are listed: invoice.pdf, photo.jpg, notes.zip and setup.png. Some extensions do not match the real signature. Use the magic-byte reference chart shown in the simulator to decide which files are disguised and what they really are.

OBJECTIVE

Identify which files have extensions that do not match their magic bytes.

STEPS IN THE SIMULATOR

- 1 **Do:** Select invoice.pdf and open the header panel.

Simulator shows:

Header: 25 50 44 46 2D 31 2E 34 (%PDF-1.4).

- 2 **Do:** Compare to the reference chart.

Simulator shows:

25 50 44 46 matches PDF. Extension is consistent.

- 3 **Do:** Select photo.jpg and open its header.

Simulator shows:

Header: 4D 5A 90 00 03 00 00 00 (MZ...).

- 4 **Do:** Compare to the chart.

Simulator shows:

4D 5A is a Windows executable. MISMATCH: photo.jpg is not an image.

- 5 **Do:** Select notes.zip and open its header.

Simulator shows:

Header: 50 4B 03 04. Matches ZIP.

- 6 **Do:** Select setup.png and open its header.

Simulator shows:

Header: 7F 45 4C 46 02 01 01 00 (.ELF...).

- 7 **Do:** Compare to the chart and flag mismatches.

Simulator shows:

7F 45 4C 46 is a Linux ELF executable. MISMATCH. Flagged: photo.jpg, setup.png.

ANSWER

Two files are disguised. photo.jpg starts with 4D 5A, so it is a Windows executable. setup.png starts with 7F 45 4C 46, so it is a Linux ELF executable. invoice.pdf and notes.zip match their signatures. Neither disguised file should be opened; hash them and quarantine.

Hint shown in the app: Compare the first two to four bytes with the chart rather than the file name.

Process Tree Walkthrough

SCENARIO

You have a shell on a fictional Linux training machine, host lab01. A user named alice complains that a file called /tmp/backup.sh seems to be in use and that something is slowing the system. You will list processes with ps, trace a child process back to its parent using the PPID column, and find which process holds a given file open, in order to identify what started it.

OBJECTIVE

Find a process parent and the process that owns a given file.

STEPS IN THE SIMULATOR

1 Do: Run: ps -ef

Simulator shows:

Lists UID PID PPID CMD. Shows 14 processes including bash (PID 2190), backup.sh (PID 2301, PPID 2190) and sleep (PID 2305, PPID 2301).

2 Do: Locate backup.sh in the list.

Simulator shows:

PID 2301, PPID 2190, owner alice.

3 Do: Find the process with PID 2190.

Simulator shows:

2190 alice bash, PPID 2150.

4 Do: Run: ps -p 2150 -o pid,comm

Simulator shows:

PID 2150 sshd. The shell was started by an SSH session.

5 Do: Run: lsof /tmp/backup.sh

Simulator shows:

COMMAND bash, PID 2301, USER alice, FD 255r, NAME /tmp/backup.sh.

6 Do: Run: ps -o pid,ppid,user,comm -p 2305

Simulator shows:

2305 2301 alice sleep: a child of backup.sh.

7 Do: Summarize the tree.

Simulator shows:

sshd (2150) -> bash (2190) -> backup.sh (2301) -> sleep (2305).

ANSWER

The file /tmp/backup.sh is held by process 2301, run by alice. The tree is sshd (2150) -> bash (2190) -> backup.sh (2301) -> sleep (2305). The script started from an SSH login shell, and sleep is just its child. No unexpected parent was found.

Hint shown in the app: Use the PPID column to climb the tree one step at a time.

Design a Safe Analysis Lab

SCENARIO

You are building a malware analysis lab on a canvas for a fictional training team. The palette includes a physical host, a VM named analysis-01, a host-only virtual switch, a bridged switch, a NAT switch, a snapshot icon and the home network. The objective is a layout in which a malware sample cannot reach the host network or the internet, and the VM can be restored to a clean state.

OBJECTIVE

Place components so the sample cannot reach the host network and the VM can be reverted.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag the Host computer onto the canvas.

Simulator shows:

Host placed. Hypervisor node attached automatically.

- 2 **Do:** Drag a VM named analysis-01 into the hypervisor.

Simulator shows:

VM added with a default NAT adapter.

- 3 **Do:** Click the VM adapter and view network options.

Simulator shows:

Options: NAT, Bridged, Host-only, Internal. Warning shown: NAT and Bridged reach other networks.

- 4 **Do:** Drag a Host-only switch onto the canvas and connect analysis-01 to it.

Simulator shows:

Connected. Route to home network: none. Route to internet: none.

- 5 **Do:** Remove the default NAT adapter.

Simulator shows:

Adapter removed. Checker: no outbound path.

- 6 **Do:** Attach a snapshot named clean-baseline to analysis-01.

Simulator shows:

Snapshot attached. Revert available.

- 7 **Do:** Toggle shared folders and clipboard to Off, then run the checker.

Simulator shows:

Checker: PASS. Isolated, revertible, no sharing channels.

ANSWER

The VM analysis-01 connects only to a host-only switch, with the NAT and bridged adapters removed, shared folders and clipboard disabled, and a clean-baseline snapshot attached. This blocks any route to the home network or internet and lets you revert after each run.

Hint shown in the app: Use host-only networking and remove every adapter that can reach outside.

How Computers Work: 10 questions, 80% to pass

1. Which statement about RAM is correct?
 - A) It keeps data without power
 - B) It is volatile working memory
 - C) It is slower than a hard disk
 - D) It stores the BIOS only
2. Which runs with the highest privilege?
 - A) User application
 - B) Kernel
 - C) Browser tab
 - D) Script
3. A user program needs to read a file. How does it request this?
 - A) System call
 - B) Direct disk write
 - C) Pinging the router
 - D) Editing the registry
4. What is binary 11111111 in decimal?
 - A) 127
 - B) 255
 - C) 256
 - D) 128
5. What is the ASCII character for 0x61?
 - A) A
 - B) a
 - C) 1
 - D) !
6. How many bits does one hex digit represent?
 - A) 2
 - B) 4
 - C) 8
 - D) 16
7. A file named photo.jpg begins with 4D 5A. What is it most likely?
 - A) A real JPEG
 - B) A Windows executable
 - C) A PDF
 - D) A text file
8. What does PPID identify?
 - A) The parent process
 - B) The program size
 - C) The port
 - D) The user password
9. Which is a Type 1 hypervisor?
 - A) VirtualBox
 - B) Hyper-V
 - C) Excel
 - D) Chrome
10. Which VM network mode best isolates a malware sample?
 - A) Bridged
 - B) Host-only
 - C) NAT
 - D) Public bridged

Answers: 1-B 2-B 3-A 4-B 5-B 6-B 7-B 8-A 9-B 10-B

1. RAM is fast, volatile working memory.
2. The kernel runs in ring 0.
3. System calls are the controlled gateway to the kernel.
4. All eight bits set equals 255.
5. 0x61 is decimal 97, lowercase a.
6. One hex digit is four bits.
7. 4D 5A is the MZ header of a Windows PE file.
8. PPID is the parent's process ID.
9. Hyper-V runs on bare metal.
10. Host-only restricts traffic to the host.

Networking Fundamentals

Beginner | 8 lessons | 6 simulated labs | checkpoint | about 5h 30m

Networks connect every computer you will ever defend or attack, so this is the longest module in the stage. You will read IPv4, IPv6 and MAC addresses, calculate subnets, and map protocols to the OSI and TCP/IP layers. You will study TCP and UDP, the three-way handshake, well-known ports, DNS, DHCP and NAT, then switches, routers and Wi-Fi. The module ends with ping, traceroute, nslookup and ARP, the first tools you will use to find where a connection fails.

BY THE END YOU CAN

- + Calculate subnets and distinguish IP from MAC addressing
- + Trace a packet through the OSI and TCP/IP layers
- + Troubleshoot connectivity using ping, traceroute and DNS tools

IN THIS MODULE

Lesson 1 IPv4, IPv6 and MAC Addresses

Lesson 2 Subnet Masks and CIDR

Lesson 3 OSI and TCP/IP Models

Lesson 4 TCP vs UDP and the Three-Way Handshake

Lesson 5 Ports and Common Services

Lesson 6 DNS, DHCP and NAT

Lesson 7 Routers, Switches and Wi-Fi

Lesson 8 ping, traceroute, nslookup and ARP

Lab 1 Subnet Calculator Practice (Scenario)

Lab 2 Read the Handshake (Packet viewer)

Lab 3 Layer the Packet (Diagram builder)

Lab 4 DNS Lookup Troubleshoot (Terminal)

Lab 5 DHCP Lease Debug (Log viewer)

Lab 6 Where Does the Packet Die? (Terminal)

Checkpoint 10 questions, 80% to pass

IPv4, IPv6 and MAC Addresses

Addresses identify devices on a network, and reading them quickly is the foundation of every later networking and log analysis task.

IPv4 addresses

An IPv4 address is a 32-bit number written as four decimal octets separated by dots, such as 192.0.2.10. Each octet is eight bits, so it ranges from 0 to 255. The address has two parts: a network portion that identifies the network and a host portion that identifies the device on it. IPv4 offers about 4.3 billion addresses, which is fewer than the number of connected devices today. That shortage drove the use of private addressing and NAT, covered later. Every device that communicates over IP, whether a laptop, a server or a printer, needs an address, which may be assigned manually or by DHCP.

Private, public and special ranges

RFC 1918 reserves three private ranges that are not routed on the public internet: 10.0.0.0 to 10.255.255.255 (10.0.0.0/8), 172.16.0.0 to 172.31.255.255 (172.16.0.0/12) and 192.168.0.0 to 192.168.255.255 (192.168.0.0/16). Home and office networks use them internally. Other special blocks include 127.0.0.1, the loopback address that always means this computer, and 169.254.0.0/16, used for automatic link-local addresses when DHCP fails. Documentation blocks 192.0.2.0/24, 198.51.100.0/24 and 203.0.113.0/24 are reserved for examples, which is why this book uses them.

IPv6 addresses

IPv6 uses 128-bit addresses written as eight groups of four hex digits separated by colons, for example 2001:db8:0:0:0:0:0:1. Leading zeros in a group can be dropped and one run of all-zero groups can be replaced by a double colon, so the same address becomes 2001:db8::1. The prefix 2001:db8::/32 is reserved for documentation. IPv6 provides an enormous address space, so NAT is not needed for scarcity. Link-local addresses begin with fe80::, and ::1 is loopback. Many networks run IPv4 and IPv6 together, so defenders must monitor and secure both, because ignoring IPv6 leaves a blind spot.

MAC addresses

A Media Access Control, or MAC, address is a 48-bit hardware identifier for a network interface, written as six hex pairs such as 00:00:5E:00:53:01. The first three bytes are the Organizationally Unique Identifier, or OUI, assigned to the manufacturer, and the last three are chosen by that manufacturer. MAC addresses work at the local network segment, while IP addresses work across networks. A MAC address can be changed in software, so it should not be treated as proof of identity. Randomized MAC addresses are now common on phones to reduce tracking, which can complicate device inventories.

ADDRESS FORMATS

```
IPv4 : 192.0.2.10 (32 bits)
IPv6 : 2001:db8::1 (128 bits)
MAC : 00:00:5E:00:53:01 (48 bits)
      OUI=00:00:5E device=00:53:01
Loop : 127.0.0.1 and ::1
```

WORKED EXAMPLE

Your laptop reports the address 169.254.17.9 and cannot reach the internet. A learner who knows the special ranges recognizes this immediately: it is a link-local address that the operating system assigned itself because it received no answer from a DHCP server. The fault is not the website but the local connection to DHCP, so the next step is to check the cable or Wi-Fi association and the DHCP service. Meanwhile, a printer shows 192.168.1.40, an RFC 1918 private address, so it is reachable only inside the local network.

KEY TERMS

IPv4: A 32-bit addressing scheme written as four decimal octets.

RFC 1918: The standard defining private IPv4 ranges not routed on the internet.

IPv6: A 128-bit addressing scheme written in hexadecimal groups.

MAC address: A 48-bit hardware identifier for a network interface.

OUI: The first three bytes of a MAC address identifying the manufacturer.

REMEMBER

- IPv4 is 32 bits, IPv6 is 128 bits and a MAC is 48 bits.
- RFC 1918 ranges are private; 169.254.x.x means DHCP failed.
- MAC addresses work locally and can be spoofed.

QUICK CHECK

1. Which is a private RFC 1918 address?
 - A) 8.8.8.8
 - B) 172.20.4.5
 - C) 192.0.2.1
 - D) 169.254.1.1
2. What does 169.254.x.x usually indicate?
 - A) A working DHCP lease
 - B) The device could not reach a DHCP server
 - C) A public server
 - D) A VPN
3. What is the shortened form of 2001:0db8:0000:0000:0000:0000:0000:0001?
 - A) 2001:db8::1
 - B) 2001::db8::1
 - C) 2001:db8:1
 - D) ::2001:db8
4. What do the first three bytes of a MAC address identify?
 - A) The VLAN
 - B) The manufacturer
 - C) The IP network
 - D) The port

Answers: 1-B 2-B 3-A 4-B

1. 172.16.0.0/12 spans 172.16 to 172.31.
2. It is an automatic link-local address assigned after DHCP failure.
3. Zero runs collapse once into a double colon.
4. That portion is the OUI assigned to the vendor.

Subnet Masks and CIDR

Subnetting tells you which devices can talk directly and how big a network is, and it appears in firewall rules and scan scopes.

What a mask does

A subnet mask divides an IPv4 address into network and host parts. Where the mask has a 1 bit, the address bit belongs to the network, and where it has a 0 bit, it belongs to the host. The mask 255.255.255.0 is 24 ones followed by 8 zeros, so the first three octets are the network and the last is the host. Two devices are on the same subnet if their addresses match under the mask. If not, traffic must go through a router, also called the default gateway. Getting the mask wrong is a common cause of partial connectivity.

CIDR notation

Classless Inter-Domain Routing, or CIDR, writes the mask as a slash and the number of network bits, so 255.255.255.0 becomes /24 and 255.255.255.192 becomes /26. The number of host bits is 32 minus the prefix length. A /24 has 8 host bits, giving 256 addresses, and a /26 has 6 host bits, giving 64. Of these, two are reserved: the first is the network address and the last is the broadcast address. Usable hosts are therefore two to the power of the host bits, minus two. A /24 offers 254 hosts, a /26 offers 62, a /30 offers 2.

Working an example

Take 192.168.1.0/26. The prefix is 26, so 6 host bits remain and the block size is 64. The network address is 192.168.1.0. Blocks start at 0, 64, 128 and 192. The broadcast address is the last address in the block, 192.168.1.63. Usable hosts run from 192.168.1.1 to 192.168.1.62, which is 62 addresses. For 192.168.1.70/26, it falls in the block starting at 64, so the network is 192.168.1.64 and the broadcast is 192.168.1.127. A quick method is to find the block size from the last mask octet: 256 minus 192 equals 64.

Why defenders care

Subnets are how networks are segmented. Placing servers, staff computers and guest Wi-Fi in different subnets lets firewalls control traffic between them. Security teams write rules and scan scopes in CIDR, for example allowing management access only from 192.0.2.0/28. Reading a log entry from a source in 198.51.100.64/26 tells you the range the address belongs to. Mistakes cause either open access, if a prefix is too short and covers too many hosts, or outages, if too long. Remember also that a larger prefix number means a smaller network, which often confuses beginners.

SUBNET WORKSHEET

```
Network : 192.168.1.0/26
Mask : 255.255.255.192
Block size: 64
Network : 192.168.1.0
Broadcast : 192.168.1.63
Hosts : 192.168.1.1 - 192.168.1.62 (62)
```

WORKED EXAMPLE

A branch office is given 203.0.113.0/24 and needs four equal segments for servers, staff, guests and cameras. Borrowing two bits creates four /26 subnets: 203.0.113.0/26, 203.0.113.64/26, 203.0.113.128/26 and 203.0.113.192/26. Each has 62 usable hosts. The firewall rule for guests then reads: deny 203.0.113.128/26 to 203.0.113.0/26. The administrator confirms that a guest laptop at 203.0.113.140 falls inside the guest block, because 140 is between 128 and 191, and so is blocked from reaching servers.

KEY TERMS

Subnet mask: A value that separates the network part of an IP address from the host part.

CIDR: Notation giving the number of network bits after a slash, such as /24.

Default gateway: The router address used to reach other networks.

Broadcast address: The last address in a subnet, used to reach all hosts in it.

Subnetting: Dividing a network into smaller networks.

REMEMBER

- Hosts per subnet equal 2 to the host bits, minus 2.
- The block size comes from the last mask octet: 256 minus the octet value.
- Segmentation by subnet enables firewall control.

QUICK CHECK

1. How many usable hosts are in a /26?
 - A) 64
 - B) 62
 - C) 30
 - D) 126
2. What is the broadcast address of 192.168.1.0/26?
 - A) 192.168.1.31
 - B) 192.168.1.63
 - C) 192.168.1.64
 - D) 192.168.1.255
3. What is the CIDR for mask 255.255.255.0?
 - A) /16
 - B) /24
 - C) /26
 - D) /30
4. A larger prefix number means what?
 - A) A larger network
 - B) A smaller network
 - C) No change
 - D) IPv6

Answers: 1-B 2-B 3-B 4-B

1. 64 addresses minus network and broadcast leaves 62.
2. The block spans .0 to .63.
3. Three full octets equal 24 bits.
4. More network bits leave fewer host bits.

OSI and TCP/IP Models

Layer models give you a shared language to locate where a problem or an attack occurs.

The seven OSI layers

The OSI model splits network communication into seven layers. From bottom to top: Physical (bits on cables or radio), Data Link (frames, MAC addresses, switches), Network (packets, IP addresses, routers), Transport (TCP and UDP, ports), Session (managing conversations), Presentation (formatting, encryption, encoding) and Application (protocols such as HTTP, DNS and SMTP). A memory aid from the top is All People Seem To Need Data Processing. The model is a teaching tool rather than an exact description of real software, but it is used universally in job interviews and in troubleshooting talk.

The TCP/IP model

The TCP/IP model, which the internet actually uses, has four layers. The Link layer covers physical and data link functions. The Internet layer handles IP addressing and routing. The Transport layer provides TCP and UDP. The Application layer combines OSI layers 5 to 7. When people say layer 3 they usually mean the OSI Network layer, which is IP, and layer 2 means the Data Link layer, which is Ethernet and MAC addresses. Layer 4 is transport and layer 7 is application, as in a layer 7 firewall that understands HTTP.

Encapsulation

Data moves down the stack at the sender and each layer adds its own header, a process called encapsulation. An application hands HTTP data to TCP, which adds a header with ports and sequence numbers to form a segment. IP adds source and destination addresses to form a packet. Ethernet adds MAC addresses and a trailer to form a frame. The receiver reverses the process, removing headers going up, which is de-encapsulation. Data units have names: bits, frames, packets, segments and data. Understanding this nesting is essential for reading packet captures, which display each layer separately.

Devices and attacks by layer

Devices map to layers. Hubs and cables are layer 1, switches are layer 2, routers are layer 3, and firewalls work from layer 3 to 7. Attacks also map to layers. Cable tapping is layer 1, ARP spoofing and MAC flooding are layer 2, IP spoofing is layer 3, SYN floods are layer 4 and SQL injection or phishing pages are layer 7. When you hear an alert description, placing it on a layer tells you which control is relevant and which team to ask. A ping that fails points to layers 1 to 3, while a working ping with a broken website points higher.

LAYER MAP

```
7 Application HTTP, DNS, SMTP
6 Presentation TLS, encoding
5 Session
4 Transport TCP, UDP segment
3 Network IP, ICMP packet router
2 Data Link Ethernet frame switch
1 Physical bits cable, radio
```

WORKED EXAMPLE

A user says the intranet page will not load. The technician pings the server's IP and gets replies, so layers 1 to 3 are healthy. A test to port 443 fails, pointing at layer 4 or above. Checking the server shows the web service stopped, a layer 7 problem. Contrast this with a case where ping fails and the cable is unplugged: that is layer 1. Working upward from the bottom and ruling out layers quickly narrows the cause, and the same approach helps analysts decide whether an alert is network noise or a real application attack.

KEY TERMS

OSI model: A seven-layer reference model for network communication.

Encapsulation: Adding headers at each layer as data moves down the stack.

Frame: The layer 2 data unit containing MAC addresses.

Packet: The layer 3 data unit containing IP addresses.

Segment: The layer 4 data unit used by TCP.

REMEMBER

- OSI has seven layers; TCP/IP has four.
- Each layer wraps data with its own header during encapsulation.
- Devices and attacks can be mapped to layers to speed troubleshooting.

QUICK CHECK

1. Which OSI layer handles IP addresses and routing?

- A) Data Link
- B) Network
- C) Transport
- D) Session

2. A switch primarily operates at which layer?

- A) 1
- B) 2
- C) 3
- D) 7

3. What is the data unit at the Transport layer for TCP?

- A) Frame
- B) Segment
- C) Bit
- D) Packet

4. ARP spoofing is an attack at which layer?

- A) Layer 2
- B) Layer 4
- C) Layer 7
- D) Layer 6

Answers: 1-B 2-B 3-B 4-A

1. Layer 3 is the Network layer.
2. Switches forward frames using MAC addresses at layer 2.
3. TCP data units are segments.
4. ARP maps IP to MAC at the data link layer.

TCP vs UDP and the Three-Way Handshake

Almost all network traffic rides on TCP or UDP, and their behavior explains scans, floods and many log lines.

TCP: reliable delivery

The Transmission Control Protocol, or TCP, provides reliable, ordered delivery. It first establishes a connection, numbers every byte with sequence numbers, acknowledges received data, retransmits what is lost and slows down when the network is congested. Applications that cannot tolerate missing data use it: web pages over HTTP and HTTPS, email, file transfer and remote login with SSH. The cost is overhead and delay from the setup and acknowledgements. TCP is called connection-oriented because both ends keep state about the conversation, and this state is what firewalls and intrusion detection systems track.

The three-way handshake

A TCP connection starts with three packets. The client sends SYN, meaning synchronize, with its initial sequence number. The server replies SYN-ACK, acknowledging the client and offering its own sequence number. The client then sends ACK, and the connection is established. TCP flags in the header include SYN, ACK, FIN (finish), RST (reset, abrupt close) and PSH and URG. A normal close uses FIN and ACK exchanges. Seeing a SYN answered with RST means the port is closed, while no reply at all often means a firewall is dropping the packet. This difference is how port scanners classify ports.

UDP: fast and connectionless

The User Datagram Protocol, or UDP, sends independent datagrams with no handshake, no acknowledgements and no guarantee of order or delivery. Its header is only 8 bytes. That makes it quick and light, suited to traffic where speed matters more than perfection or where the application handles loss itself: DNS queries, DHCP, voice and video calls, streaming, online games and many VPN protocols. Because UDP has no handshake, source addresses are easy to forge, which is why UDP services are abused for reflection and amplification denial of service attacks. Scanning UDP is slower because silence is ambiguous.

Choosing and recognizing

Choose TCP when correctness matters and UDP when timeliness matters. A dropped frame in a video call is better skipped than resent late, whereas a dropped byte in a bank transfer is unacceptable. In security, a flood of SYN packets that never complete the handshake is a SYN flood, which exhausts the server's table of half-open connections. Many SYN packets from one source to many ports suggests a port scan. Knowing the normal handshake gives you the pattern to compare against when you read a capture or firewall log.

PACKET CAPTURE LINES

```
1 192.0.2.10:51514 -> 198.51.100.5:443 [SYN] seq=100
2 198.51.100.5:443 -> 192.0.2.10:51514 [SYN,ACK] seq=900 ack=101
3 192.0.2.10:51514 -> 198.51.100.5:443 [ACK] seq=101 ack=901
```

WORKED EXAMPLE

A capture shows 192.0.2.10 sending a SYN from port 51514 to 198.51.100.5 on port 443. The server replies SYN-ACK with acknowledgment 101, one more than the client's sequence number 100, and the client completes with ACK. The three packets prove a successful connection to an HTTPS service. If the server had instead returned RST, the analyst would conclude that nothing was listening on 443. If a hundred SYNs from one source arrived in a second with no final ACKs, the pattern would suggest a SYN flood or scan.

KEY TERMS

TCP: A connection-oriented transport protocol providing reliable, ordered delivery.

UDP: A connectionless transport protocol with no delivery guarantee.

SYN: The TCP flag used to start a connection.

RST: The TCP flag that abruptly resets a connection.

SYN flood: A denial of service attack sending many SYNs without completing the handshake.

REMEMBER

- TCP is reliable and connection-oriented; UDP is fast and connectionless.
- The handshake is SYN, SYN-ACK, ACK.
- Abnormal flag patterns reveal scans and floods.

QUICK CHECK

1. What is the correct order of the TCP handshake?
 - A) SYN, SYN-ACK, ACK
 - B) ACK, SYN, FIN
 - C) SYN, ACK, SYN-ACK
 - D) FIN, ACK, SYN
2. Which protocol is typically chosen for a live voice call?
 - A) TCP
 - B) UDP
 - C) FTP
 - D) SMTP
3. A SYN answered with RST suggests what?
 - A) Port is closed
 - B) Port is open
 - C) Firewall allowed it
 - D) DNS failed
4. Why are UDP services abused in amplification attacks?
 - A) They require handshakes
 - B) Source addresses are easy to forge
 - C) They encrypt data
 - D) They are slow

Answers: 1-A 2-B 3-A 4-B

1. The client sends SYN, server SYN-ACK, client ACK.
2. UDP favors low delay over reliability.
3. RST means no service is listening.
4. No handshake means spoofed sources are not validated.

Ports and Common Services

Ports tell you which service a connection is aiming at, so they are central to firewall rules, scans and log reading.

What a port is

An IP address finds a host, and a port number finds the service on it. A port is a 16-bit number from 0 to 65535 used by TCP and UDP. A server application listens on a port, waiting for connections, and a client picks a temporary high source port, usually in the range 49152 to 65535, for its side of each conversation. The combination of IP address and port is a socket, and a connection is identified by the full set of source IP, source port, destination IP, destination port and protocol. This is why one server can handle thousands of clients on port 443.

Well-known ports

Ports 0 to 1023 are well known and assigned by IANA to standard services. Learn these first: 20 and 21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 67 and 68 DHCP, 80 HTTP, 110 POP3, 123 NTP, 143 IMAP, 161 SNMP, 389 LDAP, 443 HTTPS, 445 SMB, 3306 MySQL, 3389 RDP and 5432 PostgreSQL, the last three being registered ports above 1023. Ports 1024 to 49151 are registered, and the rest are dynamic. A service can run on a nonstandard port, so the port number is a strong hint but not proof of what is listening.

Listening states and exposure

A port on a server can be listening, meaning a service is waiting for connections, or closed, meaning nothing is there. From the network a port may be open (a service answers), closed (the host replies with a reset) or filtered (a firewall silently drops probes). Tools such as netstat and ss list local listening sockets, and a scanner such as Nmap probes from outside. Every listening port is part of the attack surface. A database listening on all interfaces, written as 0.0.0.0:3306, is reachable from anywhere the network allows, while 127.0.0.1:3306 is reachable only from the machine itself.

Security habits

Defenders reduce exposure by disabling unneeded services and blocking unused ports at the firewall. Insecure legacy services such as Telnet on 23 and FTP on 21 send passwords in clear text and should be replaced by SSH on 22 and SFTP. Administrative ports such as SSH and RDP on 3389 are heavily attacked, so they should not face the internet directly and should require strong authentication or a VPN. When reading logs, a burst of attempts to port 22 or 3389 across many addresses is common background scanning, while an unexpected outbound connection to an unusual port deserves a closer look.

SS OUTPUT

```
$ ss -tln
Netid State Local Address:Port
tcp LISTEN 0.0.0.0:22
tcp LISTEN 127.0.0.1:3306
tcp LISTEN 0.0.0.0:80
udp UNCONN 0.0.0.0:68
```

WORKED EXAMPLE

A small office server at 192.0.2.20 shows the listening sockets above. SSH on 22 and a web server on 80 listen on all interfaces, so they are reachable from the network. MySQL on 3306 listens only on 127.0.0.1, so remote users cannot connect to it directly, which is the safer design. The administrator notes that port 80 carries unencrypted HTTP and plans to add TLS on 443 and redirect. If a later scan found 3306 open on 0.0.0.0, that change in exposure would be investigated immediately.

KEY TERMS

Port: A 16-bit number identifying a service endpoint on a host.

Socket: The combination of an IP address and port.

Listening: A state where a service waits for incoming connections.

Filtered: A port whose probes are dropped, usually by a firewall.

Well-known ports: Ports 0 to 1023 assigned to standard services.

REMEMBER

- Ports identify services; IP plus port forms a socket.
- Learn 22, 53, 80, 443 and 3389 first.
- Every listening port widens the attack surface.

QUICK CHECK

1. Which port does SSH use by default?
 - A) 21
 - B) 22
 - C) 23
 - D) 25
2. Which port is HTTPS?
 - A) 80
 - B) 443
 - C) 3389
 - D) 53
3. What does 127.0.0.1:3306 listening mean?
 - A) Reachable from anywhere
 - B) Reachable only from the same machine
 - C) Closed
 - D) Filtered
4. Why is Telnet discouraged?
 - A) It is fast
 - B) It sends data including passwords in clear text
 - C) It uses UDP
 - D) It is a database

Answers: 1-B 2-B 3-B 4-B

1. SSH uses TCP 22.
2. HTTPS uses TCP 443.
3. Loopback binding limits access to the local host.
4. Telnet has no encryption.

DNS, DHCP and NAT

These three services quietly make a network usable, and each is a frequent target or source of evidence.

DNS: names to addresses

The Domain Name System translates names like `www.example.com` into IP addresses. A client asks a recursive resolver, which queries the hierarchy of root, top-level domain and authoritative servers and caches the answer for a period called the TTL. DNS uses UDP port 53 for most queries and TCP 53 for large responses and zone transfers. Key record types are A (name to IPv4), AAAA (name to IPv6), CNAME (alias to another name), MX (mail server), NS (authoritative name server), TXT (free text, used for SPF and verification) and PTR (address to name). DNS logs reveal which names a host looks up, which is valuable evidence.

DHCP: automatic addressing

The Dynamic Host Configuration Protocol assigns IP settings automatically. It uses UDP ports 67 for the server and 68 for the client, and a four-step exchange known as DORA. The client broadcasts a Discover, a server answers with an Offer of an address, the client sends a Request for that offer, and the server finishes with an Acknowledgment. The lease includes address, mask, default gateway, DNS servers and a lease time. If no server answers, Windows and many systems fall back to a `169.254.x.x` link-local address. A rogue DHCP server on a network can hand out malicious gateway or DNS settings, which is why switches offer DHCP snooping.

NAT and PAT

Network Address Translation lets many private hosts share public addresses. A router at the edge rewrites the source address of outbound packets. The common form, Port Address Translation or PAT, maps each internal address and port to the router's one public address with a unique port, and keeps a translation table so replies return to the right host. Internal machines with RFC 1918 addresses such as `192.168.1.20` therefore appear as one public address, for example `203.0.113.9`. NAT also blocks unsolicited inbound connections by default, which gives a side benefit, but it is not a replacement for a firewall.

Security implications

DNS is trusted by everything, so attacks on it matter. Cache poisoning tries to plant false answers, and attackers use DNS for command and control or to smuggle data out inside queries, called DNS tunneling. Defenders log queries and look for odd patterns such as very long random-looking names. DHCP logs map an IP address to a device at a particular time, which is essential for tracing an alert back to a machine. NAT complicates investigation because many users share one public address, so logs from both sides and timestamps are needed to identify the actual host.

DNS AND DHCP SAMPLES

```
$ nslookup -type=MX example.com
example.com mail exchanger = 10 mail.example.com.
```

```
DHCP: DISCOVER (broadcast) -> OFFER 192.168.1.50
      REQUEST 192.168.1.50 -> ACK, lease 24h
```

WORKED EXAMPLE

A laptop joins the office Wi-Fi. It broadcasts a DHCP Discover and receives an Offer of `192.168.1.50` with gateway `192.168.1.1` and DNS `192.168.1.2`. After Request and Ack it can browse. It asks DNS for `www.example.com`, receives an A record for `198.51.100.5` and connects to port 443. At the edge router, PAT rewrites the source to `203.0.113.9` with a unique port. A log at the web server shows only `203.0.113.9`, so the analyst needs the router's translation log to identify which internal laptop made the request.

KEY TERMS

DNS: The system that resolves domain names to IP addresses.

TTL: The time a DNS answer may be cached.

DORA: Discover, Offer, Request, Acknowledge: the DHCP exchange.

NAT: Rewriting addresses so private hosts share public addresses.

PAT: NAT that also maps ports so many hosts use one public address.

REMEMBER

- DNS resolves names, DHCP assigns settings, NAT shares public addresses.
- DNS uses port 53; DHCP uses 67 and 68.
- Logs from DHCP and NAT are needed to map an address to a device.

QUICK CHECK

1. Which DNS record maps a name to an IPv4 address?
 - A) MX
 - B) A
 - C) NS
 - D) PTR
2. What is the correct DHCP sequence?
 - A) Discover, Offer, Request, Acknowledge
 - B) Request, Discover, Offer, Ack
 - C) Offer, Discover, Ack, Request
 - D) Ack, Offer, Request, Discover
3. What does PAT use to separate internal hosts sharing one public address?
 - A) Port numbers
 - B) Cable color
 - C) MAC OUI
 - D) VLAN tags
4. Which DNS record type identifies a domain's mail server?
 - A) MX
 - B) TXT
 - C) AAAA
 - D) SOA only

Answers: 1-B 2-A 3-A 4-A

1. An A record holds an IPv4 address.
2. DORA stands for that order.
3. Unique source ports identify each translation.
4. MX records list mail exchangers.

Routers, Switches and Wi-Fi

Understanding how traffic is forwarded locally and wirelessly shows where it can be intercepted, segmented or blocked.

Switches and CAM tables

A switch connects devices on one local network and forwards Ethernet frames at layer 2. It learns by watching the source MAC address of each frame and storing the MAC and the port in its content addressable memory, the CAM table or MAC address table. When a frame arrives, the switch looks up the destination MAC and sends it out only the matching port. If the destination is unknown it floods the frame to all ports. A hub, by contrast, repeats everything everywhere. Attackers can fill a CAM table with fake entries to make a switch flood, called MAC flooding, which port security settings can limit.

Routers and routing

A router connects different networks and forwards packets at layer 3 using a routing table. Each entry lists a destination network, a next hop and an interface, and the most specific match wins. The default route 0.0.0.0/0 sends everything unknown to the upstream gateway. Hosts send traffic for other networks to their default gateway, which is a router. Routers can also apply access control lists that permit or deny traffic by address and port, and they often perform NAT. Compared to switches, which separate collision domains, routers separate broadcast domains, so broadcasts do not cross them.

VLANs

A virtual LAN splits one physical switch into several logical networks. Ports are assigned to a VLAN, identified by an ID number such as 10 or 20, and traffic stays inside its VLAN unless a router or layer 3 switch forwards it. Between switches, a trunk link carries several VLANs using 802.1Q tags. Typical designs use separate VLANs for staff, servers, guests and cameras so that a compromised device has limited reach. VLANs improve organization and security, but they are not a perfect barrier, since misconfigured trunks can allow attacks known as VLAN hopping.

Wi-Fi basics and security

Wi-Fi follows the IEEE 802.11 standards, such as 802.11n, ac (Wi-Fi 5) and ax (Wi-Fi 6), on 2.4, 5 and 6 GHz bands. A network is named by its SSID, which is only a label and not a security feature. Hiding the SSID adds almost no protection. WEP is broken and must not be used. WPA2 with AES-CCMP and a strong passphrase is the minimum, and WPA3 improves protection against offline password guessing with SAE. Enterprise networks use 802.1X with individual credentials. Guest Wi-Fi belongs on its own VLAN, and attackers sometimes run evil twin access points that copy a legitimate SSID.

SWITCH AND ROUTE TABLES

```
MAC table (VLAN 10) Route table
0050.0000.0001 Fa0/1 192.168.10.0/24 eth1 direct
0050.0000.0002 Fa0/2 192.168.20.0/24 eth2 direct
0050.0000.0003 Fa0/3 0.0.0.0/0 via 203.0.113.1
```

WORKED EXAMPLE

A company puts staff laptops on VLAN 10 and visitors on VLAN 30. A visitor connects to the guest SSID, which uses WPA2 and sits on VLAN 30. The router permits VLAN 30 only to the internet, and an access control list blocks it from 192.168.10.0/24. When the visitor tries to open a staff file server at 192.168.10.5, the packet reaches the router, which checks its ACL and drops it. The switch's CAM table shows the visitor's MAC only on its guest port, confirming layer 2 separation.

KEY TERMS

CAM table: A switch table mapping MAC addresses to ports.

Default gateway: The router a host uses for traffic leaving its network.

VLAN: A logical network created on switch ports, identified by an ID.

SSID: The name of a Wi-Fi network.

WPA3: The newer Wi-Fi security standard using SAE for stronger password protection.

REMEMBER

- Switches forward by MAC; routers forward by IP.

- VLANs segment a network logically.
- Use WPA2 or WPA3 and keep guests on a separate VLAN.

QUICK CHECK

1. What does a switch use to decide which port to send a frame to?
 - A) IP address
 - B) MAC address
 - C) Port number
 - D) Hostname
2. Which device separates broadcast domains?
 - A) Hub
 - B) Router
 - C) Cable
 - D) Repeater
3. Which Wi-Fi security option should not be used?
 - A) WPA2
 - B) WPA3
 - C) WEP
 - D) 802.1X
4. What carries multiple VLANs between switches?
 - A) A trunk
 - B) A hub
 - C) A modem
 - D) An SSID

Answers: 1-B 2-B 3-C 4-A

1. Switches use MAC addresses and the CAM table.
2. Routers do not forward broadcasts.
3. WEP is broken.
4. Trunk links use 802.1Q tagging.

ping, traceroute, nslookup and ARP

These four commands are the first tools for finding where a connection breaks, and you will use them every week.

ping and ICMP

The ping command sends ICMP echo requests and waits for echo replies. ICMP, the Internet Control Message Protocol, is a layer 3 helper protocol that reports errors and tests reachability. A reply shows that the path works both ways. Output includes the round trip time in milliseconds and the TTL, the time to live value. Each router that forwards a packet decreases TTL by one. Replies with TTL 64 often come from Linux and 128 from Windows, though this is only a hint. Loss, such as 4 sent and 0 received, may mean the host is down, the route is broken or a firewall blocks ICMP.

traceroute and the path

Traceroute, called tracert on Windows, shows each router along the path. It sends packets with TTL 1, then 2, then 3 and so on. Each router that sees the TTL expire returns an ICMP Time Exceeded message, revealing its address and the delay. Lines with asterisks mean no reply, which can be a router that does not answer or a drop point. Reading the output from top to bottom, a sudden jump in latency shows a slow link, and a path that stops at a hop and never continues points at the failing router or a filtering firewall beyond it.

nslookup, dig and ARP

Use nslookup or dig to test DNS. nslookup www.example.com returns the resolver used and the address found, and nslookup -type=MX example.com returns mail servers. If an IP address works but a name fails, the fault is in DNS. The arp command shows the ARP cache, which maps IPv4 addresses to MAC addresses on the local segment. ARP, the Address Resolution Protocol, asks who has an IP address and learns the answer. An entry marked incomplete means no one answered. Two IP addresses with the same MAC, or the gateway MAC changing suddenly, may signal ARP spoofing.

A troubleshooting method

Work from the bottom up. First check the link and your own address with ip a or ipconfig. Ping the loopback 127.0.0.1 to test the local stack, then your default gateway, then a public IP such as 192.0.2.1, then a name. A failure at gateway means a local problem. A failure after the gateway means the path or the provider. If the IP works but the name fails, check DNS with nslookup. Recording each result gives a clear picture, and the same method supports incident triage when you need to prove a host is truly unreachable.

COMMAND OUTPUT

```
$ ping -c 2 192.0.2.1
64 bytes from 192.0.2.1: icmp_seq=1 ttl=63 time=4.1 ms
$ traceroute 198.51.100.5
 1 192.168.1.1 1.2 ms
 2 203.0.113.1 8.9 ms
 3 * * *
$ arp -a
? (192.168.1.1) at 00:00:5e:00:53:01 [ether]
```

WORKED EXAMPLE

A user cannot open www.example.com. The technician pings the gateway 192.168.1.1 and gets replies, so the local link is fine. Ping to 198.51.100.5 also succeeds, so routing works. Ping to the name fails with unknown host. nslookup shows the configured DNS server does not answer. The fault is DNS, not connectivity. The technician switches the resolver to a working server, nslookup returns an A record, and the page loads. Four short commands separated three layers of possible faults in under two minutes.

KEY TERMS

ICMP: A layer 3 protocol for error messages and tests such as ping.

TTL: A counter decreased at every router hop that limits packet lifetime.

traceroute: A tool that lists the routers along a path using increasing TTL values.

ARP: A protocol mapping IPv4 addresses to MAC addresses locally.

nslookup: A tool for querying DNS records.

REMEMBER

- Ping tests reachability; traceroute shows the path.
- nslookup tests DNS; arp shows local IP to MAC mappings.
- Troubleshoot from the bottom up and record each result.

QUICK CHECK

1. What does a traceroute line with * * * mean?
 - A) No reply was received for that hop
 - B) The hop is fastest
 - C) The host is a Linux machine
 - D) ARP failed
2. Ping works to an IP but not to a name. What is likely wrong?
 - A) DNS
 - B) Cable
 - C) Switch power
 - D) MAC address
3. Which command shows the IP to MAC mapping cache?
 - A) arp -a
 - B) ping
 - C) nslookup
 - D) tracert
4. What does each router do to TTL?
 - A) Decrease by one
 - B) Double it
 - C) Reset it
 - D) Ignore it

Answers: 1-A 2-A 3-A 4-A

1. Asterisks show missing replies.
2. Name resolution is failing.
3. arp -a lists the ARP cache.
4. TTL decreases per hop, and at zero the packet is dropped.

Subnet Calculator Practice

SCENARIO

A fictional network team at Example Corp has handed you six address blocks to document for a firewall change request. The simulator provides a form for each block with fields for network address, broadcast address and usable host count, and it validates each answer immediately. You should work out the block size from the prefix, find where the block starts, and subtract two from the total addresses to get usable hosts, showing your method before submitting each result.

OBJECTIVE

Enter the network address, broadcast address and usable hosts for six CIDR blocks.

STEPS IN THE SIMULATOR

- 1 **Do:** Block 1: 192.168.1.0/26. Enter network, broadcast and hosts.

Simulator shows:

Accepted: 192.168.1.0, 192.168.1.63, 62 hosts.

- 2 **Do:** Block 2: 192.0.2.0/24.

Simulator shows:

Accepted: 192.0.2.0, 192.0.2.255, 254 hosts.

- 3 **Do:** Block 3: 198.51.100.64/26.

Simulator shows:

Accepted: 198.51.100.64, 198.51.100.127, 62 hosts.

- 4 **Do:** Block 4: 203.0.113.128/25.

Simulator shows:

Accepted: 203.0.113.128, 203.0.113.255, 126 hosts.

- 5 **Do:** Block 5: 192.0.2.16/28.

Simulator shows:

Accepted: 192.0.2.16, 192.0.2.31, 14 hosts.

- 6 **Do:** Block 6: 198.51.100.0/30.

Simulator shows:

Accepted: 198.51.100.0, 198.51.100.3, 2 hosts.

- 7 **Do:** Submit the worksheet.

Simulator shows:

Score 6 of 6. Hint shown: block size = $2^{(32 - \text{prefix})}$.

ANSWER

/26 blocks of 64 give 62 hosts. /24 gives 254, /25 gives 126, /28 gives 14 and /30 gives 2. In each case the network address is the block start, the broadcast address is the block start plus size minus one, and usable hosts are size minus two.

Hint shown in the app: Find the block size first, then the multiple of that size at or below the address.

Read the Handshake

SCENARIO

You open a short capture from a fictional office network in a packet viewer. The file contains eight packets between a client at 192.0.2.10 and a server at 198.51.100.5. The columns show source, destination, protocol, flags and sequence numbers. Your job is to label the three handshake packets, identify which packet begins the data exchange, and determine the destination port of the service the client connected to.

OBJECTIVE

Label SYN, SYN-ACK and ACK packets and find the server port.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the capture and sort by packet number.

Simulator shows:

Eight packets listed, numbered 1 to 8.

- 2 **Do:** Select packet 1.

Simulator shows:

192.0.2.10:51514 -> 198.51.100.5:443 TCP [SYN] seq=100.

- 3 **Do:** Label packet 1 as SYN.

Simulator shows:

Correct.

- 4 **Do:** Select packet 2.

Simulator shows:

198.51.100.5:443 -> 192.0.2.10:51514 [SYN, ACK] seq=900 ack=101.

- 5 **Do:** Label packet 2 as SYN-ACK and packet 3 as ACK.

Simulator shows:

Correct. Packet 3: [ACK] seq=101 ack=901.

- 6 **Do:** Select packet 4 and read the protocol column.

Simulator shows:

TLSv1.2 Client Hello. First data after the handshake.

- 7 **Do:** Enter the server port.

Simulator shows:

Correct: 443 (HTTPS). Client source port 51514 is ephemeral.

ANSWER

Packet 1 is SYN, packet 2 is SYN-ACK and packet 3 is ACK. The server port is 443, so the service is HTTPS. The client port 51514 is a temporary source port, and packet 4 is the TLS Client Hello starting data exchange.

Hint shown in the app: The SYN comes from the client; the server replies from the service port.

Layer the Packet

SCENARIO

A trainee must show how an HTTP request is wrapped for transmission. The diagram builder gives you four header blocks labeled Ethernet, IP, TCP and HTTP, and an empty frame outline with positions for outermost to innermost. Place the headers in the order they would appear on the wire, with the first bytes sent on the left, and then confirm the field each layer uses for addressing so the frame can cross a network to a web server.

OBJECTIVE

Arrange Ethernet, IP, TCP and HTTP headers in the correct order and name their addressing fields.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag the Ethernet block to position 1.

Simulator shows:

Accepted. Outermost header goes first on the wire.

- 2 **Do:** Drag the IP block to position 2.

Simulator shows:

Accepted.

- 3 **Do:** Drag the TCP block to position 3.

Simulator shows:

Accepted.

- 4 **Do:** Drag the HTTP block to position 4.

Simulator shows:

Accepted. Application data is innermost.

- 5 **Do:** Click the Ethernet header.

Simulator shows:

Fields: destination MAC, source MAC, type 0x0800 (IPv4).

- 6 **Do:** Click the IP header.

Simulator shows:

Fields: source IP 192.0.2.10, destination IP 198.51.100.5, protocol 6 (TCP).

- 7 **Do:** Click TCP, then run the check.

Simulator shows:

TCP: source port 51514, destination port 80, flags. Check: PASS.

ANSWER

The order is Ethernet, IP, TCP, HTTP. Ethernet uses MAC addresses, IP uses IP addresses with protocol number 6 for TCP, TCP uses ports and flags, and HTTP carries the request text. Each layer wraps the one above it.

Hint shown in the app: Think of the envelopes: the outermost header is the local delivery label.

DNS Lookup Troubleshoot

SCENARIO

You have a terminal on a fictional help desk workstation. Users report that `www.example.com` works but a newer name, `intranet.example.com`, does not resolve. You will use `nslookup` and `dig` to query A, MX and NS records, compare the answers and decide why one hostname fails while the other works. The resolver for the lab is `192.0.2.53`. Record each result as you go, because the final answer depends on comparing what the resolver says with what the authoritative servers say.

OBJECTIVE

Query A, MX and NS records and find why one hostname fails.

STEPS IN THE SIMULATOR

- 1 Do:** Run: `nslookup www.example.com`
Simulator shows:
`Server 192.0.2.53. Address: 198.51.100.5.`
- 2 Do:** Run: `nslookup intranet.example.com`
Simulator shows:
`** server can't find intranet.example.com: NXDOMAIN`
- 3 Do:** Run: `dig example.com NS +short`
Simulator shows:
`ns1.example.com. ns2.example.com.`
- 4 Do:** Run: `dig example.com MX +short`
Simulator shows:
`10 mail.example.com.`
- 5 Do:** Run: `dig @ns1.example.com intranet.example.com A +short`
Simulator shows:
`(empty). The authoritative server has no A record.`
- 6 Do:** Run: `dig @ns1.example.com intranet-app.example.com A +short`
Simulator shows:
`198.51.100.30. A record exists under a different name.`
- 7 Do:** State the cause.
Simulator shows:
`Recorded: record missing or misspelled in the zone.`

ANSWER

The name `intranet.example.com` returns `NXDOMAIN` and the authoritative server has no A record for it, while a similar record exists as `intranet-app.example.com`. The fault is a missing or misnamed record in the zone, not the resolver.

Hint shown in the app: If the authoritative server also lacks the record, the problem is the zone data.

DHCP Lease Debug

SCENARIO

A fictional laptop named LAPTOP-07 in a branch office shows the address 169.254.33.8 and cannot reach the network. You are given a log from the client and the DHCP relay. The log viewer lets you filter by MAC address 00:00:5E:00:53:07. Find the point in the DORA exchange that never happened and explain why the client picked a link-local address. Read entries in time order and note which messages appear for this MAC.

OBJECTIVE

Identify the missing DHCP step that left the client on a 169.254.x.x address.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter the log by MAC 00:00:5E:00:53:07.

Simulator shows:
Six entries shown.

- 2 **Do:** Read entry 1.

Simulator shows:
10:02:11 client DHCPDISCOVER broadcast sent.

- 3 **Do:** Read entry 2.

Simulator shows:
10:02:15 client DHCPDISCOVER retry 1.

- 4 **Do:** Look for a DHCPOFFER from the server.

Simulator shows:
No DHCPOFFER entry found for this MAC.

- 5 **Do:** Read entry 5.

Simulator shows:
10:02:50 no response; assigning link-local 169.254.33.8.

- 6 **Do:** Check the relay log.

Simulator shows:
Relay 192.168.30.1: link to DHCP server 192.168.1.2 down.

- 7 **Do:** State the missing step.

Simulator shows:
Recorded: Offer never arrived, so Request and Ack never occurred.

ANSWER

The client sent Discover but never received an Offer, so no Request or Acknowledge followed. The relay could not reach the DHCP server, so the client self-assigned 169.254.33.8. Restoring the relay link or server reachability fixes it.

Hint shown in the app: Count which of D, O, R, A appear for that MAC.

Where Does the Packet Die?

SCENARIO

You are in a terminal on a fictional host at 192.168.1.20 and a user cannot reach a server at 198.51.100.5. You will run ping and traceroute to see where traffic stops. The simulated path crosses the local gateway, an ISP edge router and an upstream router. Use the hop output to decide which device is dropping the packets and what to report to the provider.

OBJECTIVE

Locate the hop at which traffic stops using ping and traceroute.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: ping -c 3 192.168.1.1

Simulator shows:
3 replies, ttl=64, 1 ms. Gateway is reachable.

- 2 **Do:** Run: ping -c 3 198.51.100.5

Simulator shows:
3 packets sent, 0 received, 100 percent loss.

- 3 **Do:** Run: traceroute 198.51.100.5

Simulator shows:
Hop 1 192.168.1.1 1.1 ms.

- 4 **Do:** Read hop 2.

Simulator shows:
Hop 2 203.0.113.1 9.8 ms.

- 5 **Do:** Read hop 3.

Simulator shows:
Hop 3 203.0.113.9 14.2 ms.

- 6 **Do:** Read hops 4 to 6.

Simulator shows:
Hops 4, 5, 6: * * *

- 7 **Do:** State the finding.

Simulator shows:
Recorded: last responding hop is 203.0.113.9.

ANSWER

The local network and the first two routers work. Traffic stops after 203.0.113.9, the last responding hop, so the fault lies at or beyond the next router on the provider side. Report 203.0.113.9 as the last good hop.

Hint shown in the app: The failing device is just beyond the last hop that answered.

Networking Fundamentals: 10 questions, 80% to pass

1. Which address is a private RFC 1918 address?
 - A) 198.51.100.4
 - B) 192.168.5.9
 - C) 203.0.113.7
 - D) 169.254.2.2
2. What is the broadcast address of 192.168.1.0/26?
 - A) 192.168.1.63
 - B) 192.168.1.64
 - C) 192.168.1.255
 - D) 192.168.1.31
3. How many usable hosts are in a /28?
 - A) 16
 - B) 14
 - C) 30
 - D) 6
4. At which OSI layer does a router work?
 - A) 1
 - B) 2
 - C) 3
 - D) 7
5. What is the correct handshake order?
 - A) SYN, ACK, SYN-ACK
 - B) SYN, SYN-ACK, ACK
 - C) ACK, SYN, FIN
 - D) SYN-ACK, SYN, ACK
6. Which transport protocol suits a live video call?
 - A) UDP
 - B) FTP
 - C) Telnet
 - D) SMTP
7. Which port does RDP use by default?
 - A) 22
 - B) 443
 - C) 3389
 - D) 25
8. A client self-assigns 169.254.x.x. What probably failed?
 - A) DNS
 - B) DHCP
 - C) NAT
 - D) ARP spoofing
9. What does PAT allow?
 - A) Many private hosts to share one public address
 - B) Encrypt traffic
 - C) Block ICMP
 - D) Assign MAC addresses
10. Ping to an IP works but the hostname fails. Where is the fault?
 - A) DNS
 - B) Cable
 - C) Switch CAM table
 - D) Wi-Fi band

Answers: 1-B 2-A 3-B 4-C 5-B 6-A 7-C 8-B 9-A 10-A

1. 192.168.0.0/16 is private.
2. The block runs .0 to .63.
3. 16 addresses minus two.
4. Routers forward packets at layer 3.
5. Client SYN, server SYN-ACK, client ACK.
6. UDP favors timeliness.
7. RDP uses TCP 3389.
8. No DHCP server answered.
9. Port translation lets hosts share one address.
10. Name resolution is failing.

How the Web Works

Beginner | 6 lessons | 4 simulated labs | checkpoint | about 4h

Most attacks you will study target web applications, so you need to know how the web works before you can see how it fails. This module follows a page load from the URL through DNS, TCP and TLS, then reads HTTP requests, headers, methods and status codes. You will learn how cookies and sessions keep users logged in, what HTML, CSS and JavaScript do in the browser, how APIs exchange JSON, and how databases answer SQL queries safely.

BY THE END YOU CAN

- + Break down a URL and read an HTTP request and response
- + Explain cookies, sessions and the front-end stack
- + Write basic SQL SELECT queries and read JSON from an API

IN THIS MODULE

Lesson 1 URLs and the Request Lifecycle

Lesson 2 HTTP Methods, Headers and Status Codes

Lesson 3 Cookies, Sessions and HTTPS

Lesson 4 HTML, CSS and JavaScript Basics

Lesson 5 APIs, REST and JSON

Lesson 6 Databases and SQL Basics

Lab 1 Inspect a Request (Web proxy)

Lab 2 Status Code Detective (Log viewer)

Lab 3 Cookie Flag Audit (Web proxy)

Lab 4 Query the Customers Table (Terminal)

Checkpoint 10 questions, 80% to pass

URLs and the Request Lifecycle

Every web attack begins with a request, and reading a URL correctly tells you what the browser is asking for and from whom.

Anatomy of a URL

A Uniform Resource Locator names a resource and how to reach it. Take `https://shop.example.com:8443/products/list?cat=books&page=2#reviews`. The scheme, `https`, selects the protocol. The host, `shop.example.com`, names the server. The port, `8443`, overrides the default, which is `80` for `http` and `443` for `https`. The path, `/products/list`, selects the resource. The query string after the question mark holds name and value pairs separated by ampersands. The fragment after the hash, `reviews`, is used only by the browser to jump within the page and is never sent to the server. Optional user information before the host is legacy and often abused in phishing.

From click to connection

When you press enter, the browser first checks its cache and then asks DNS to turn the host into an IP address, for example `198.51.100.5`. It opens a TCP connection to that address on the correct port using the three-way handshake. For `https`, a TLS handshake follows, in which the server presents a certificate, the browser verifies it and both sides agree on encryption keys. Only then does the browser send the HTTP request. Each stage can fail separately: a DNS error, a connection timeout, a certificate warning or an HTTP error. Knowing the stage tells you where to look.

Request and response

HTTP is a request and response protocol. The browser sends a request with a method, a path, headers and sometimes a body. The server replies with a status code, headers and a body, which for a page is HTML. The page references further resources such as stylesheets, scripts and images, and the browser sends a separate request for each. A single page may trigger dozens of requests to several hosts, including advertising and analytics domains. HTTP itself is stateless, meaning each request is independent, which is why cookies were invented. Every one of these requests is a place where input can be tampered with.

Security view of URLs

Because the query string is visible in browser history, proxy logs and referrer headers, secrets such as passwords or tokens should never be placed in a URL. Attackers craft deceptive links, for example `https://example.com.login.invalid`, where the real host is everything between the scheme and the first single slash, and its registered domain is at the right-hand end, not the familiar name at the start. Hover over links and read the host from right to left. URL parameters are also user-controlled input, so servers must validate them. Defenders read URLs in web logs to spot scanning, injection attempts and unusual paths.

URL BREAKDOWN

```
https://shop.example.com:8443/products/list?cat=books&page=2#reviews
scheme https
host shop.example.com
port 8443
path /products/list
query cat=books&page=2
fragment reviews (client only)
```

WORKED EXAMPLE

A user clicks a link in an email: `https://secure-login.example.net.invalid/example.com/account`. A careful reader splits it. The scheme is `https`, the host is `secure-login.example.net.invalid`, and `example.com` appears only in the path, so the link does not go to `example.com` at all. The page load would involve a DNS lookup for the lookalike host, a TCP connection to port `443` and a TLS certificate for that attacker domain. The `https` padlock only says the connection is encrypted, not that the site is trustworthy. The user deletes the email and reports it.

KEY TERMS

URL: A string that names a resource and how to reach it.

Query string: Name and value pairs after the question mark in a URL.

Fragment: The part after # used only by the browser.

TLS: The protocol that encrypts and authenticates HTTPS connections.

Stateless: Treating every request independently, with no memory of earlier ones.

REMEMBER

- A URL has scheme, host, port, path, query and fragment.
- A page load involves DNS, TCP, TLS and then HTTP.
- Read hosts carefully to spot deceptive links.

QUICK CHECK

1. Which URL part is never sent to the server?

- A) Path
- B) Query
- C) Fragment
- D) Host

2. What is the default port for https?

- A) 80
- B) 443
- C) 8080
- D) 22

3. In https://a.example.com/x?id=5, what is the host?

- A) a.example.com
- B) /x
- C) id=5
- D) https

4. Why avoid putting secrets in URLs?

- A) They are slower
- B) URLs appear in logs and history
- C) They are encrypted twice
- D) They break DNS

Answers: 1-C 2-B 3-A 4-B

1. The fragment is handled by the browser only.
2. HTTPS defaults to 443.
3. The host follows the scheme and precedes the path.
4. URLs are logged and shown in many places.

HTTP Methods, Headers and Status Codes

Reading raw HTTP is the core skill for web testing and for understanding web server logs.

Methods

The method says what the client wants done. GET retrieves a resource and should not change data. POST submits data to be processed, such as a login or a form. PUT replaces a resource, PATCH changes part of it, and DELETE removes it. HEAD returns only headers, and OPTIONS asks which methods are allowed. GET and HEAD are called safe because they should not modify anything. A frequent weakness is a server that performs state changes on GET requests or allows methods nobody intended, such as DELETE on a public path. Parameters in a GET travel in the URL, while POST parameters travel in the body.

A request and its headers

A request begins with a line such as GET /index.html HTTP/1.1, followed by headers, one per line, then a blank line and an optional body. Host names the site. User-Agent identifies the client software and is easily forged. Accept lists formats the client wants. Cookie carries stored cookies. Authorization carries credentials such as a bearer token. Content-Type and Content-Length describe a body. Referer states the page that linked here. Response headers include Content-Type, Set-Cookie, Location for redirects, Server, and security headers such as Strict-Transport-Security and Content-Security-Policy that instruct the browser to apply protections.

Status codes

The first digit of the status code gives the class. 1xx is informational. 2xx means success: 200 OK, 201 Created and 204 No Content. 3xx means redirection: 301 Moved Permanently, 302 Found and 304 Not Modified. 4xx means the client made an error: 400 Bad Request, 401 Unauthorized (authentication needed), 403 Forbidden (authenticated but not allowed), 404 Not Found and 429 Too Many Requests. 5xx means the server failed: 500 Internal Server Error, 502 Bad Gateway and 503 Service Unavailable. Many 404 responses in a row from one address is typical of content scanning, and repeated 401 responses suggest password guessing.

Reading logs

Web servers record each request on a line, often in the Common Log Format: client address, time, request line, status and size. Reading these lines tells a story. A 200 on a path like /admin from an unfamiliar address deserves attention. A burst of 500 responses after a specially crafted parameter may show an attack tripping an error. A 403 means the control worked. Because the User-Agent and many other fields can be faked, logs prove what was requested, not who really sent it. Combine status, path, size, timing and source to judge intent.

HTTP EXCHANGE

```
GET /account HTTP/1.1
Host: shop.example.com
Cookie: sid=a1b2c3

HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: theme=dark; Secure

198.51.100.9 - - [05/Oct/2026:10:01:02 +0000] "GET /admin HTTP/1.1" 403 199
```

WORKED EXAMPLE

A log shows 203.0.113.25 requesting /admin, /backup.zip, /login.php and /wp-admin within two seconds, each answered 404. The pattern of many different guessed paths and 404 codes indicates automated content discovery. Later the same address sends 25 POST requests to /login with 401 responses, then a 200 and a redirect 302 to /dashboard. The analyst concludes that a password guessing attempt may have succeeded, checks the account, forces a password reset and blocks the address range while reviewing what the session accessed.

KEY TERMS

GET: A method that retrieves a resource without changing it.

POST: A method that submits data to the server for processing.

401: Status code meaning authentication is required or has failed.

403: Status code meaning the request is understood but not permitted.

Header: A name and value pair carrying metadata with a request or response.

REMEMBER

- Methods say what to do; headers carry metadata; status codes report the result.
- 2xx success, 3xx redirect, 4xx client error, 5xx server error.
- Logs show requests, not proven identity.

QUICK CHECK

1. Which method should not change server data?

- A) GET
- B) POST
- C) DELETE
- D) PUT

2. What does status 403 mean?

- A) Not found
- B) Forbidden
- C) Server error
- D) Redirect

3. Which class of status code indicates a server failure?

- A) 2xx
- B) 3xx
- C) 4xx
- D) 5xx

4. Many 404 responses from one IP across different paths suggests what?

- A) Normal browsing
- B) Content discovery scanning
- C) TLS failure
- D) DNS caching

Answers: 1-A 2-B 3-D 4-B

1. GET is defined as safe.
2. The server refuses access to an identified client.
3. 5xx codes mean a server-side error.
4. Guessing paths yields many 404s.

Cookies, Sessions and HTTPS

Cookies and sessions are how websites remember you, so stealing or abusing them is a major attack goal.

Why cookies exist

Because HTTP is stateless, a site must recognize a returning user some other way. After login, the server creates a session, a record on its side, and sends back a session ID in a Set-Cookie header. The browser stores the cookie and attaches it to every later request to that site through the Cookie header. The server looks up the ID and knows who is calling. The session ID is therefore as powerful as a password for as long as it is valid, and anyone who obtains it can impersonate the user, a technique called session hijacking.

Cookie attributes

Security flags on a cookie reduce risk. HttpOnly prevents JavaScript from reading the cookie, limiting theft through cross-site scripting. Secure restricts the cookie to HTTPS connections so it is never sent in clear text. SameSite controls whether the cookie accompanies cross-site requests: Strict never sends it cross-site, Lax sends it only on top-level navigations, and None allows all but requires Secure. This helps resist cross-site request forgery. Expires or Max-Age sets lifetime, and Domain and Path define scope. A session cookie without HttpOnly, Secure and SameSite is a finding in almost every security review.

Session management

Good session handling generates long random IDs that cannot be guessed, issues a new ID at login to prevent session fixation, expires idle sessions, and destroys the server-side session at logout. Tokens should not appear in URLs. Sensitive actions may require re-authentication. Multi-factor authentication reduces the value of a stolen password but not a stolen session cookie, which is why session protection matters separately. Persistent login cookies, often called remember me, extend risk and should be long random tokens stored hashed on the server and revocable by the user.

HTTPS and TLS

HTTPS is HTTP carried inside a TLS connection. TLS provides three properties: confidentiality through encryption, integrity through message authentication and authentication of the server through a certificate signed by a trusted certificate authority. Without it, anyone on the path, such as someone on shared Wi-Fi, can read or alter traffic. Modern sites use TLS 1.2 or 1.3. The HSTS header tells browsers to use HTTPS only. A certificate warning means the identity check failed and should not be bypassed. HTTPS protects data in transit only; it does not protect against a malicious site or a flaw in the application itself.

SET-COOKIE HEADER

```
HTTP/1.1 200 OK
Set-Cookie: sid=9f8e7d6c5b4a; Path=/; Secure; HttpOnly; SameSite=Lax; Max-Age=1800
Strict-Transport-Security: max-age=31536000
```

WORKED EXAMPLE

A tester reviews the login response for app.example.com and sees Set-Cookie: sid=1234; Path=/. No HttpOnly, no Secure and no SameSite appear, and the ID 1234 looks short and guessable. The tester reports three issues: the cookie can be read by scripts, may be sent over plain HTTP, and may be forged. The recommended fix is a long random value with Secure, HttpOnly and SameSite=Lax, a new ID issued on login, and HSTS enabled so browsers never attempt plain HTTP.

KEY TERMS

Session ID: A value that links a browser to a server-side session.
HttpOnly: A cookie flag blocking JavaScript access.
Secure: A cookie flag restricting the cookie to HTTPS.
SameSite: A cookie attribute controlling cross-site sending.
Session hijacking: Using a stolen session ID to impersonate a user.

REMEMBER

- Cookies carry session IDs; protect them with HttpOnly, Secure and SameSite.
- A stolen session ID equals a stolen login.

- TLS secures data in transit, not the application.

QUICK CHECK

1. Which flag prevents JavaScript from reading a cookie?

- A) Secure
- B) HttpOnly
- C) SameSite
- D) Path

2. What does the Secure flag do?

- A) Hashes the cookie
- B) Sends it only over HTTPS
- C) Encrypts the disk
- D) Extends expiry

3. What does TLS NOT protect against?

- A) Eavesdropping
- B) Tampering in transit
- C) A flaw in the web application
- D) Server impersonation

4. Why issue a new session ID at login?

- A) To prevent session fixation
- B) To slow users
- C) To reduce cookies
- D) To use HTTP

Answers: 1-B 2-B 3-C 4-A

1. HttpOnly hides it from scripts.
2. The cookie is not sent over plain HTTP.
3. TLS protects the channel, not application logic.
4. A fresh ID stops an attacker from pre-setting the session.

HTML, CSS and JavaScript Basics

The browser runs code and displays content from servers you do not control, and that fact underlies injection and client-side attacks.

Three languages, three jobs

A web page is built from three technologies. HTML, HyperText Markup Language, defines structure and content with tags such as headings, paragraphs, links, forms and images. CSS, Cascading Style Sheets, controls appearance, such as color, layout and fonts. JavaScript adds behavior: it responds to clicks, updates the page without reloading and sends background requests. The browser downloads these as files and combines them. All three are plain text that the user can inspect through the browser's developer tools, which means nothing sent to the browser is secret.

The DOM

After parsing the HTML, the browser builds the Document Object Model, a tree of objects representing every element on the page. JavaScript can read and change this tree, for example altering text, adding elements or hiding buttons. Developer tools let a user edit the DOM live, which only changes their own view and not the server. If an application inserts untrusted text into the page without encoding it, an attacker's input may be interpreted as markup or script rather than as text. That flaw is cross-site scripting, and it is prevented by output encoding and content security policies.

Forms and submitted data

A form groups input fields and says where and how to send them. The action attribute gives the URL, and the method attribute gives GET or POST. Each input has a name, and on submission the browser sends name and value pairs, either in the query string or in the request body. Hidden fields and disabled buttons are just HTML, so users can change or bypass them. Whatever the browser sends must be treated as untrusted input by the server, including values chosen from drop-down lists, since the request can be crafted by hand or by a tool rather than through the form at all.

What client-side code can and cannot be trusted with

JavaScript validation, such as checking that an email field looks right, is a convenience for users and gives quick feedback. It is not a security control because attackers can disable scripts, edit the code or send requests directly. Every rule that matters, including authentication, authorization, price calculation and input validation, must be enforced again on the server. Likewise, secrets such as API keys or passwords must not be placed in client code, since anyone can read them. A useful rule is: the client is for display, the server is for decisions.

HTML FORM

```
<form action="/login" method="POST">
  <input type="text" name="user">
  <input type="password" name="pass">
  <input type="hidden" name="role" value="user">
  <button type="submit">Sign in</button>
</form>
```

Body sent: user=alice&pass=...&role=user

WORKED EXAMPLE

A fictional shop page includes a hidden field price=49.99 for a keyboard, and JavaScript that blocks submission if the quantity is negative. A tester uses the developer tools to change the hidden price to 0.01 and sends the form. The server accepts it because it trusts the submitted price. The finding is that pricing and validation were enforced only in the browser. The fix is for the server to look up the price from its own product database, validate the quantity itself and ignore any price sent by the client.

KEY TERMS

HTML: The markup language that structures web page content.

CSS: The language that controls the look and layout of a page.

JavaScript: The scripting language that adds behavior in the browser.

DOM: The tree of page elements the browser exposes to scripts.

XSS: Cross-site scripting, where untrusted input runs as script in a page.

REMEMBER

- HTML structures, CSS styles, JavaScript acts.
- Anything sent to the browser can be inspected and changed.
- Always enforce validation and authorization on the server.

QUICK CHECK

1. Which technology adds behavior to a page?
 - A) HTML
 - B) CSS
 - C) JavaScript
 - D) DNS
2. Why is client-side validation not a security control?
 - A) It is too slow
 - B) Attackers can bypass or modify it
 - C) It uses HTTPS
 - D) It is encrypted
3. A form field name and value reach the server how?
 - A) As a name and value pair in the query or body
 - B) Through DNS
 - C) Via ARP
 - D) As a cookie only
4. What does the DOM represent?
 - A) The server's disk
 - B) The tree of elements in the loaded page
 - C) The network
 - D) A database

Answers: 1-C 2-B 3-A 4-B

1. JavaScript runs logic in the browser.
2. Requests can be crafted without the page's scripts.
3. Browsers send form data as name=value pairs.
4. The DOM is the browser's page object tree.

APIs, REST and JSON

Modern apps talk to APIs, which are the main interface attackers and testers examine.

What an API is

An application programming interface lets one program request data or actions from another in a defined way. Web APIs use HTTP, so the tools you already know apply. A mobile app, a single-page website and a partner company can all call the same API. Unlike a human-facing page, an API returns structured data meant for programs. Because APIs expose the logic and data of an application directly, they are a large part of the attack surface, and they are often less visible than web pages because users never see them in a browser.

REST

Representational State Transfer, or REST, is a common style for web APIs. Resources are identified by URLs, and HTTP methods express actions. For example GET /api/v1/users/42 reads user 42, POST /api/v1/users creates a user, PUT /api/v1/users/42 replaces it and DELETE /api/v1/users/42 removes it. Responses use status codes such as 200, 201, 400, 401, 403, 404 and 429. REST is stateless, so each request must carry what the server needs, usually an authentication token. Plural nouns in the path and version numbers such as v1 are common conventions.

JSON

JavaScript Object Notation is the usual data format. It is plain text built from objects in curly braces holding keys and values, arrays in square brackets, strings in double quotes, numbers, true, false and null. For example {"id": 42, "name": "alice", "roles": ["user"], "active": true}. Keys must be double-quoted and trailing commas are not allowed. The header Content-Type: application/json announces it. Learn to read nested objects, since real responses may contain several levels. JSON is easy to read, so sensitive fields that the client does not need should be removed on the server before sending.

API authentication

APIs must know who is calling. An API key is a long secret string sent in a header such as X-API-Key, identifying an application. A bearer token, sent as Authorization: Bearer followed by the token, represents a logged-in user or app and often has an expiry. JSON Web Tokens, or JWTs, are a common token format containing signed claims. Secrets must be kept out of source code and URLs, stored securely and rotated. Common API weaknesses include missing authorization checks on object IDs, so that changing 42 to 43 reveals another user's record, excessive data in responses and no rate limiting.

API CALL

```
GET /api/v1/users/42 HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJ...redacted

HTTP/1.1 200 OK
Content-Type: application/json

{"id": 42, "name": "alice", "roles": ["user"], "active": true}
```

WORKED EXAMPLE

A tester logged in as user 42 calls GET /api/v1/orders/1001 and receives their own order in JSON. They change the number to 1002, an order belonging to another customer, and the API returns it with status 200. The server checked that the token was valid but never checked that the order belonged to the caller. This broken object level authorization flaw is among the most common API issues. The fix is to verify ownership of every requested object on the server, returning 403 or 404 for others.

KEY TERMS

API: A defined way for programs to request data or actions from each other.

REST: An API style using URLs for resources and HTTP methods for actions.

JSON: A text format of keys, values, objects and arrays for data exchange.

Bearer token: A credential sent in the Authorization header that grants access to its holder.

API key: A secret string identifying the calling application.

REMEMBER

- APIs return structured data over HTTP, usually JSON.
- REST uses URLs for resources and methods for actions.
- Tokens and keys must be protected and each object access authorized.

QUICK CHECK

1. Which method usually creates a new resource in REST?
A) GET
B) POST
C) HEAD
D) TRACE
2. Which is valid JSON?
A) {name: alice}
B) {"name": "alice"}
C) <name>alice</name>
D) name=alice
3. Where is a bearer token usually sent?
A) Authorization header
B) URL fragment
C) DNS query
D) Image
4. Changing an ID in an API URL reveals another user's data. What is missing?
A) TLS
B) An ownership authorization check
C) A bigger key
D) CSS

Answers: 1-B 2-B 3-A 4-B

1. POST submits data to create a resource.
2. JSON keys and strings use double quotes.
3. It goes in the Authorization header.
4. The server must verify the caller owns the object.

Databases and SQL Basics

Databases hold the valuable data, and unsafe queries are one of the most damaging web flaws.

Relational databases

A relational database stores data in tables made of rows and columns. A customers table might have columns id, name, city and balance, and each row is one customer. A primary key, usually id, uniquely identifies each row, and a foreign key in another table points to it to link records. Popular systems include MySQL, PostgreSQL, SQL Server and SQLite. Web applications talk to them using SQL, Structured Query Language. The database server normally listens on its own port, such as 3306 for MySQL and 5432 for PostgreSQL, and should never be directly exposed to the internet.

Reading data with SELECT

The SELECT statement reads data. SELECT name, city FROM customers returns two columns for every row, and SELECT * returns all columns. WHERE filters rows: SELECT name FROM customers WHERE city = 'Leeds'. Combine conditions with AND and OR, and sort with ORDER BY balance DESC. LIMIT 5 restricts the number of rows. Text values are in single quotes and numbers are not. The comparison operators are =, <>, <, >, <= and >=. LIKE with a percent sign matches patterns, for example name LIKE 'A%'. Reading these statements fluently helps you understand both logs and application behavior.

INSERT, UPDATE, DELETE and JOIN

INSERT INTO customers (name, city) VALUES ('Ana Test', 'Leeds') adds a row. UPDATE customers SET city = 'York' WHERE id = 7 changes rows, and DELETE FROM customers WHERE id = 7 removes them. Forgetting the WHERE clause affects every row, a classic and costly mistake. JOIN combines tables: SELECT c.name, o.total FROM customers c JOIN orders o ON o.customer_id = c.id returns each customer with their orders. Applications should connect with an account that has only the permissions it needs, for example read access to some tables, so that a flaw cannot drop everything.

Why input must be parameterized

If an application builds a query by gluing user text into a string, such as "SELECT * FROM users WHERE name = '" + input + "'", then an attacker can type characters that change the meaning of the query. This is SQL injection, and it can expose or alter entire databases. The defense is parameterized queries, also called prepared statements, where the SQL text with placeholders is sent separately from the values, so input is always treated as data and never as code. Additional layers are input validation, least privilege database accounts and error messages that do not reveal database details.

SQL EXAMPLES

```
SELECT name, city FROM customers WHERE city = 'Leeds' ORDER BY name;

SELECT c.name, o.total
FROM customers c JOIN orders o ON o.customer_id = c.id
WHERE o.total > 100;

-- safe pattern: SELECT * FROM users WHERE name = ?
```

WORKED EXAMPLE

A login form builds the query by concatenating the typed name. A tester enters a lone apostrophe, and the page returns a database error, showing that input reaches the SQL statement unmodified. This is a strong sign of injection risk, so the tester stops, documents the page and reports it without extracting any data. The developer rewrites the code to use a prepared statement with a placeholder and changes the application account to read only the users table. The same input now simply fails to match any name and returns an ordinary login failure.

KEY TERMS

Table: A set of rows and columns holding one kind of record.

Primary key: A column that uniquely identifies each row.

SELECT: The SQL statement that reads data.

JOIN: A SQL operation combining rows from two tables.

Parameterized query: A query where values are passed separately so input cannot change the SQL.

REMEMBER

- Tables hold rows; SELECT reads, WHERE filters, JOIN combines.
- Missing WHERE clauses on UPDATE or DELETE affect every row.
- Use parameterized queries and least privilege accounts.

QUICK CHECK

1. Which clause filters rows in SQL?
 - A) ORDER BY
 - B) WHERE
 - C) LIMIT
 - D) FROM
2. What happens if you run DELETE FROM customers with no WHERE?
 - A) Nothing
 - B) All rows are deleted
 - C) One row is deleted
 - D) An error always
3. What is the best defense against SQL injection?
 - A) Parameterized queries
 - B) Hidden fields
 - C) Longer URLs
 - D) Bigger fonts
4. Which statement combines two tables?
 - A) JOIN
 - B) INSERT
 - C) DROP
 - D) LIMIT

Answers: 1-B 2-B 3-A 4-A

1. WHERE selects the rows that match a condition.
2. Without a condition every row is removed.
3. They keep input as data, not code.
4. JOIN links rows by matching columns.

Inspect a Request

SCENARIO

You are testing a fictional training site, `www.example.com`, with explicit permission inside a lab. A web proxy sits between the browser and the site and lists each request. You log in with the demo account `alice` and intercept the login request. Study its raw text and identify the method, the path, important headers, the cookie, and the body parameters, then note which values the browser chose and which the user typed.

OBJECTIVE

Identify method, headers, cookie and body parameters in an intercepted login request.

STEPS IN THE SIMULATOR

- 1 **Do:** Submit the login form and open the intercepted request.

Simulator shows:

Request 14 captured: POST /login HTTP/1.1.

- 2 **Do:** Read the request line.

Simulator shows:

Method POST, path /login, version HTTP/1.1.

- 3 **Do:** Read the headers.

Simulator shows:

Host: `www.example.com`; User-Agent: `LabBrowser/1.0`; Content-Type: `application/x-www-form-urlencoded`; Content-Length: `31`.

- 4 **Do:** Find the Cookie header.

Simulator shows:

Cookie: `sid=anon-7731; theme=light`

- 5 **Do:** Read the body.

Simulator shows:

`user=alice&pass=Demo123&csrf=ab12`

- 6 **Do:** Forward the request and read the response.

Simulator shows:

HTTP/1.1 302 Found, Location: `/dashboard`, Set-Cookie: `sid=9f8e7d6c; Path=/`

- 7 **Do:** Note what changed.

Simulator shows:

Session ID changed from `anon-7731` to `9f8e7d6c` after login.

ANSWER

The request is POST /login with Content-Type urlencoded. The body has user, pass and csrf parameters. The pre-login cookie `sid=anon-7731` was replaced by a new session ID on login, which is good practice, and the 302 redirects to /dashboard.

Hint shown in the app: Method and path are on the first line; parameters are after the blank line.

Status Code Detective

SCENARIO

A fictional web server for `www.example.com` has recorded several lines in its access log. The log viewer highlights the status field. Your task is to read each line for 301, 403, 404 and 500, explain what the user would have experienced and decide which line could point to an attack or to a bug in the application that engineers should investigate first.

OBJECTIVE

Explain what each status code in the log means for the user and spot the suspicious one.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the access log.

Simulator shows:

Shows 8 lines; status column highlighted.

- 2 **Do:** Read line 2: GET /old-page 301 0.

Simulator shows:

301 Moved Permanently. The user is redirected to the new address.

- 3 **Do:** Read line 4: GET /admin 403 199.

Simulator shows:

403 Forbidden. The user is blocked from the admin page.

- 4 **Do:** Read line 5: GET /missing.png 404 153.

Simulator shows:

404 Not Found. A broken link or image.

- 5 **Do:** Read line 6: GET /search?q=%27 500 721.

Simulator shows:

500 Internal Server Error after an odd quote character in input.

- 6 **Do:** Check source of lines 4 and 6.

Simulator shows:

Both from 203.0.113.25 within 3 seconds.

- 7 **Do:** Flag the line most worth escalating.

Simulator shows:

Line 6 flagged: input likely reaching a database query.

ANSWER

A 301 redirects, a 403 denies, a 404 reports a missing resource and a 500 reports a server fault. Line 6 is the most important: a lone quote in the search parameter caused a 500 from a source that also probed /admin, suggesting testing for injection.

Hint shown in the app: Compare the status code with the request path and the parameter values.

Cookie Flag Audit

SCENARIO

You are auditing three responses captured from a fictional site, shop.example.com, in a lab proxy. Each response sets a cookie. For every cookie you must check three attributes: HttpOnly, Secure and SameSite. Record which ones are missing, rate the cookie by its purpose, and recommend which response needs the most urgent fix before the site goes live. Treat any cookie that proves identity as more sensitive than one that stores a preference.

OBJECTIVE

Find which cookies lack HttpOnly, Secure or SameSite.

STEPS IN THE SIMULATOR

- 1 **Do:** Open response 1.

Simulator shows:

Set-Cookie: sid=9f8e7d6c5b4a; Path=/; Secure; HttpOnly; SameSite=Lax

- 2 **Do:** Mark response 1.

Simulator shows:

All three flags present. Pass.

- 3 **Do:** Open response 2.

Simulator shows:

Set-Cookie: cart=3|7|9; Path=/; SameSite=Lax

- 4 **Do:** Mark response 2.

Simulator shows:

Missing HttpOnly and Secure.

- 5 **Do:** Open response 3.

Simulator shows:

Set-Cookie: auth=eyJ...; Path=

- 6 **Do:** Mark response 3.

Simulator shows:

Missing HttpOnly, Secure and SameSite. Auth cookie.

- 7 **Do:** Rank the fixes.

Simulator shows:

Priority: response 3 first (auth), response 2 second.

ANSWER

Response 1 is correct. Response 2 lacks HttpOnly and Secure, lower impact because it only stores cart items. Response 3 is an authentication cookie missing HttpOnly, Secure and SameSite, the most urgent fix.

Hint shown in the app: Weigh the missing flags by what the cookie protects.

Query the Customers Table

SCENARIO

You have a SQL terminal on a fictional training database named shopdb. The customers table has columns id, name, city and balance, with twelve sample rows. A manager sends four questions that you must answer using SELECT queries with WHERE and ORDER BY. Use only read statements and check each result before reporting it. The terminal prints rows as pipe-separated text and reports a row count after each query, so you can confirm that each answer matches the question asked.

OBJECTIVE

Answer four lookup questions using SELECT with WHERE and ORDER BY.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: SELECT name, city FROM customers LIMIT 3;

Simulator shows:

Ana Test|Leeds; Ben Demo|York; Cara Sample|Leeds

- 2 **Do:** Q1: who lives in Leeds? Run: SELECT name FROM customers WHERE city = 'Leeds';

Simulator shows:

Ana Test; Cara Sample; Dev Example (3 rows)

- 3 **Do:** Q2: balances over 500, highest first. Run: SELECT name, balance FROM customers WHERE balance > 500 ORDER BY balance DESC;

Simulator shows:

Eli Fake 920; Cara Sample 640; Ben Demo 510 (3 rows)

- 4 **Do:** Q3: how many customers in York? Run: SELECT COUNT(*) FROM customers WHERE city = 'York';

Simulator shows:

4

- 5 **Do:** Q4: lowest balance. Run: SELECT name, balance FROM customers ORDER BY balance ASC LIMIT 1;

Simulator shows:

Fay Placeholder 12

- 6 **Do:** Try: SELECT name FROM customers WHERE city = Leeds;

Simulator shows:

ERROR: unknown column Leeds. Text needs single quotes.

- 7 **Do:** Report all answers.

Simulator shows:

All four answers accepted.

ANSWER

Leeds customers are Ana Test, Cara Sample and Dev Example. Balances over 500 in order are Eli Fake 920, Cara Sample 640 and Ben Demo 510. Four customers live in York. The lowest balance is Fay Placeholder with 12.

Hint shown in the app: Text values need single quotes, and ORDER BY ... DESC puts the highest first.

How the Web Works: 10 questions, 80% to pass

1. Which part of a URL is not sent to the server?
 - A) Path
 - B) Query
 - C) Fragment
 - D) Host
2. Which method is intended only to retrieve data?
 - A) GET
 - B) POST
 - C) PUT
 - D) DELETE
3. What does a 401 status mean?
 - A) Authentication required or failed
 - B) Server crashed
 - C) Moved permanently
 - D) Created
4. Many 404 responses for different paths from one IP suggest what?
 - A) Scanning for content
 - B) A healthy site
 - C) A TLS upgrade
 - D) A database backup
5. Which cookie flag blocks JavaScript from reading the cookie?
 - A) HttpOnly
 - B) Secure
 - C) Path
 - D) Domain
6. Which flag keeps a cookie off plain HTTP connections?
 - A) Secure
 - B) SameSite
 - C) HttpOnly
 - D) Max-Age
7. Why must validation also happen on the server?
 - A) Clients can be bypassed or modified
 - B) Servers are faster
 - C) HTML forbids it
 - D) It saves bandwidth
8. Which JSON snippet is valid?
 - A) {"a": 1}
 - B) {a: 1}
 - C) {'a': 1}
 - D) a=1
9. Which SQL clause filters rows?
 - A) WHERE
 - B) ORDER BY
 - C) JOIN
 - D) INSERT
10. What best prevents SQL injection?
 - A) Parameterized queries
 - B) Client-side checks only
 - C) Hiding the form
 - D) Longer passwords

Answers: 1-C 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. The fragment stays in the browser.
2. GET is a safe, read-only method.
3. 401 means credentials are missing or invalid.
4. Path guessing yields many 404s.
5. HttpOnly hides it from scripts.
6. Secure restricts it to HTTPS.
7. Attackers control what the client sends.
8. JSON requires double-quoted keys.
9. WHERE sets the condition.
10. They separate code from data.

Linux I: Meet the Terminal

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

Linux runs most servers, cloud workloads, routers and security tools, so every defender needs to be comfortable in its terminal. This module explains what Linux actually is, how to read a prompt and break a command into parts, and how to move around with `pwd`, `ls` and `cd`. You will learn the standard directory layout, the difference between absolute and relative paths, and how to find answers yourself with man pages and help flags, a skill that never stops being useful.

BY THE END YOU CAN

- + Navigate the Linux file system with `pwd`, `ls` and `cd`
- + Use absolute and relative paths correctly
- + Look up command options with `man` and `--help`

IN THIS MODULE

Lesson 1 What Linux Is: Kernel, Distros, Shells

Lesson 2 The Prompt and Command Anatomy

Lesson 3 `pwd`, `ls` and `cd`

Lesson 4 The Filesystem Hierarchy and Paths

Lesson 5 man Pages and Getting Help

Lab 1 First Terminal Session (Terminal)

Lab 2 Path Puzzle (Terminal)

Lab 3 Find Where Logs Live (File explorer)

Lab 4 Decode the Flags (Terminal)

Checkpoint 10 questions, 80% to pass

What Linux Is: Kernel, Distros, Shells

Knowing the pieces that make up a Linux system tells you what you are actually running and where to look when something goes wrong.

The kernel

Strictly speaking, Linux is a kernel, the core program that manages hardware, memory, processes and files. Linus Torvalds began it in 1991, and thousands of contributors now maintain it as open source software under the GNU General Public License. Open source means anyone may read, modify and share the code, which allows broad review and rapid fixes, though it does not by itself guarantee security. The kernel alone is not a usable system. It needs a set of tools around it, such as the GNU utilities for files and text, libraries and a way for users to give commands.

Distributions

A Linux distribution, or distro, packages the kernel with tools, a package manager, default settings and often a desktop. Debian is a long-established, stable community distribution. Ubuntu is based on Debian and widely used on servers and laptops. Red Hat Enterprise Linux, Rocky Linux and Fedora form another family using the rpm and dnf tools. Kali Linux is a Debian-based distribution bundling security testing tools, and Parrot is similar. Distros differ mainly in package management, release cycles and defaults, but the core commands you learn in this path work on nearly all of them. Check your version with `cat /etc/os-release`.

Shells and bash

A shell is a program that reads commands you type, runs them and shows the results. Bash, the Bourne Again Shell, is the default on most distributions. Others include zsh and sh, a minimal standard shell often used for scripts. The shell is separate from the terminal, which is the window that displays text. A shell can run single commands and also scripts, text files of commands, which is how administrators automate work. Typing is faster and more repeatable than clicking, and servers often have no graphical interface at all, so the command line is the primary way to work.

Why Linux matters for security

Many security tools, such as Nmap, Wireshark and Metasploit, are built for or run best on Linux, and most web servers, containers and cloud instances run it. Attackers and defenders both meet Linux constantly. Logs, configuration files and permissions are all plain files that you can read and search with simple commands, which makes investigations transparent. Everything in Linux is treated as a file or a process, a design idea that you will use repeatedly. Practice safely in a virtual machine, where a mistake costs nothing, and learn to be careful with any command run as the administrator account called root.

IDENTIFY THE SYSTEM

```
$ uname -sr
Linux 6.1.0-18-amd64
$ cat /etc/os-release | head -2
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
NAME="Debian GNU/Linux"
$ echo $SHELL
/bin/bash
```

WORKED EXAMPLE

A junior analyst receives access to an unfamiliar server at 192.0.2.30 and needs to know what it runs before touching anything. She runs `uname -sr`, which prints the kernel name and release, then reads `/etc/os-release` to learn the distribution and version. She checks the shell with `echo $SHELL` and sees `/bin/bash`. With that, she knows to use `apt` rather than `dnf` for packages and where to look for documentation. These three read-only commands changed nothing on the system and gave her the baseline she needs for the investigation.

KEY TERMS

Kernel: The core of the operating system managing hardware, memory and processes.

Distribution: A packaged Linux system with kernel, tools and package manager.

Shell: A program that interprets typed commands.

Bash: The Bourne Again Shell, the default shell on most distributions.

Open source: Software whose source code is available to read, change and share.

REMEMBER

- Linux is a kernel; distributions add tools and a package manager.
- Bash is a shell that runs commands and scripts.
- Most servers and security tools run on Linux.

QUICK CHECK

1. Strictly speaking, what is Linux?

- A) A full desktop
- B) A kernel
- C) A shell
- D) A browser

2. Which is a Debian-based distribution?

- A) Ubuntu
- B) Windows
- C) macOS
- D) Solaris

3. What is bash?

- A) A kernel
- B) A shell
- C) A file system
- D) A firewall

4. Which file shows the distribution name and version?

- A) /etc/os-release
- B) /tmp
- C) /dev/null
- D) /bin/lis

Answers: 1-B 2-A 3-B 4-A

1. Linux is the kernel at the core of the system.
2. Ubuntu is derived from Debian.
3. Bash is the Bourne Again Shell.
4. /etc/os-release holds distro identification.

The Prompt and Command Anatomy

Reading the prompt and a command line correctly prevents mistakes, especially when running as root.

Reading the prompt

When the shell is ready it prints a prompt. A typical one looks like `alice@lab01:~$`. It shows the user name, `alice`, the host name, `lab01`, and the current directory, where the tilde means the home directory. The final character matters. A dollar sign means a normal user, while a hash sign means the root account. Commands run as root can change or delete anything, so a hash prompt calls for extra care. Prompts can be customized, so confirm who you are with `whoami` and where you are with `pwd` before any risky action.

Program, options and arguments

A command line has up to three parts separated by spaces. The program is the first word, such as `ls`. Options, also called flags or switches, modify behavior and usually begin with a dash. A short option uses one dash and one letter, such as `-l`, and short options can be combined, so `-la` means `-l` and `-a`. A long option uses two dashes and a word, such as `--all`. Arguments say what to act on, such as a file or directory name. In `ls -l /var/log`, the program is `ls`, the option is `-l` and the argument is `/var/log`.

Options that take values and quoting

Some options require a value, for example `grep -m 5 error log.txt` where `5` belongs to `-m`. Spaces separate words, so a file name containing a space must be quoted, as in `ls "My Notes"`, or the shell will treat the pieces as two arguments. Single quotes keep text literal, and double quotes allow variables to expand. Commands and file names are case sensitive: `Notes` and `notes` are different files. The shell also expands special characters such as the asterisk before the program runs, which you will study when working with files in the next module.

Working efficiently and safely

Use the Tab key to complete names, which is faster and prevents typos. The up and down arrows recall earlier commands, history lists them and `Ctrl+R` searches them. `Ctrl+C` interrupts a running command, `Ctrl+L` clears the screen and `Ctrl+D` ends input or logs out. The `echo` command prints text, and the exit status of the last command is available in `$?`, where `0` means success. Before running a command you found online, read what it does and avoid pasting commands you do not understand, particularly those starting with `sudo`, because a single character can change the result completely.

PROMPT AND COMMAND

```
alice@lab01:~$ ls -l /var/log
| | | | |
| | | | | +-- argument (what to act on)
| | | | +----- option (how)
| | | +----- program (what)
| | +----- $ = normal user, # = root
| +----- host
+----- user
```

WORKED EXAMPLE

You see the prompt `root@lab01:/etc#` and decide to remove an old file. Before doing so you notice the hash sign and recall that this is the administrator account. You run `whoami`, which prints `root`, and `pwd`, which prints `/etc`, confirming you are in the system configuration directory. Rather than risk an irreversible mistake, you switch to your normal account, practice the command on a copy in your home directory and only then repeat it with the proper privileges. Reading the prompt took two seconds and avoided a potential system outage.

KEY TERMS

Prompt: Text shown by the shell indicating it is ready for a command.

Option: A flag, usually starting with a dash, that changes how a command behaves.

Argument: The item a command acts on, such as a file or directory.

Root: The all-powerful administrator account on Linux.

Exit status: A number returned by a command; 0 means success.

REMEMBER

- A prompt shows user, host, directory and privilege level.
- A command is program, options and arguments.
- Check whoami and pwd before risky commands.

QUICK CHECK

1. What does a # at the end of the prompt usually mean?
 - A) Normal user
 - B) Root user
 - C) Comment
 - D) Error
2. In ls -l /var/log, what is /var/log?
 - A) Program
 - B) Option
 - C) Argument
 - D) Prompt
3. What does -la mean?
 - A) The option -l and the option -a combined
 - B) A file named la
 - C) An error
 - D) Long listing only
4. Which key combination interrupts a running command?
 - A) Ctrl+C
 - B) Ctrl+L
 - C) Tab
 - D) Ctrl+D

Answers: 1-B 2-C 3-A 4-A

1. A hash prompt indicates the root account.
2. It is the target the command acts on.
3. Short options can be combined.
4. Ctrl+C sends an interrupt.

pwd, ls and cd

These three commands are how you see where you are, what is around you and how to move.

pwd: where am I

The `pwd` command, short for print working directory, shows your current location in the file system as a full path. A freshly opened terminal usually starts in your home directory, such as `/home/alice`. Knowing your location matters because many commands act on the current directory by default. If you run a command in the wrong place, you may read or change the wrong files. Get in the habit of running `pwd` whenever you are unsure, particularly before deleting or copying files. The output is a single line and, like most Linux tools, it prints nothing extra when successful.

ls: what is here

The `ls` command lists the contents of a directory. With no argument it lists the current directory. Useful options include `-l` for the long format showing permissions, owner, size and modification date, `-a` for all files including hidden ones whose names begin with a dot, and `-h` for human-readable sizes such as 4.0K or 2.3M when combined with `-l`. The options combine: `ls -lah` is common. Add `-t` to sort by modification time, newest first, and `-r` to reverse the order. You can also give a path, as in `ls -l /etc`, to list a directory other than the current one.

cd: moving around

The `cd` command changes the current directory. `cd /var/log` moves to that absolute path. `cd` with no argument, or `cd ~`, goes to your home directory. `cd ..` goes up one level, since the double dot means the parent directory, and a single dot means the current directory. `cd -` returns to the previous directory, handy for toggling between two places. If a directory name contains spaces, quote it. A message such as No such file or directory means the path is wrong or you have a typo, and Permission denied means you are not allowed to enter it. Tab completion avoids most errors.

Hidden files and a working rhythm

Hidden files start with a dot, such as `.bashrc`, which configures your shell, and `.ssh`, which holds keys. They are hidden only to reduce clutter and are not secret or protected, but attackers like to place files there, so security staff always list with `-a`. A good rhythm is to run `pwd` to see where you are, `ls` to see what is there, then `cd` to move, and repeat. In long listings the first character shows the type, `d` for directory and a dash for a regular file. You will study the remaining permission characters in a later module.

TERMINAL SESSION

```
alice@lab01:~$ pwd
/home/alice
alice@lab01:~$ ls -lah
total 28K
drwxr-xr-x 5 alice alice 4.0K Oct 5 09:12 .
drwxr-xr-x 3 root root 4.0K Oct 1 08:00 ..
-rw-r--r-- 1 alice alice 220 Oct 1 08:00 .bash_logout
drwxr-xr-x 2 alice alice 4.0K Oct 5 09:10 projects
alice@lab01:~$ cd projects && pwd
/home/alice/projects
```

WORKED EXAMPLE

You are asked to find a folder named `.hidden-notes` somewhere under your home directory. Plain `ls` shows only `projects` and `reports`, so the folder is not listed. You run `ls -a` and now see `.hidden-notes` with a leading dot. You then run `cd .hidden-notes` and confirm your new location with `pwd`, which prints `/home/alice/.hidden-notes`. Later `cd -` takes you straight back to `/home/alice`. This small routine of listing with `-a`, moving with `cd` and verifying with `pwd` is how you explore any unfamiliar system without changing anything.

KEY TERMS

pwd: Prints the full path of the current working directory.

ls: Lists the contents of a directory.

cd: Changes the current working directory.

Hidden file: A file or directory whose name starts with a dot.

Home directory: A user's personal directory, often /home/username, abbreviated ~.

REMEMBER

- pwd shows where you are, ls shows what is there, cd moves you.
- ls -a reveals hidden files; -l and -h give detail.
- cd ~, cd .. and cd - are the key shortcuts.

QUICK CHECK

1. Which command prints your current directory?

- A) pwd
- B) ls
- C) cd
- D) man

2. Which option makes ls show hidden files?

- A) -a
- B) -l
- C) -h
- D) -t

3. What does cd - do?

- A) Goes to the previous directory
- B) Goes to root
- C) Deletes a directory
- D) Lists files

4. What does .. mean in a path?

- A) Current directory
- B) Parent directory
- C) Home directory
- D) Root

Answers: 1-A 2-A 3-A 4-B

1. pwd prints the working directory.
2. -a lists all files including dotfiles.
3. It switches back to the last directory.
4. Double dot refers to the parent directory.

The Filesystem Hierarchy and Paths

Knowing where things live and how to write a path lets you find logs, configs and evidence on any Linux machine.

One tree starting at root

Linux has a single directory tree that starts at the root directory, written `/`. There are no drive letters. Other disks and devices are attached into the tree at mount points. The Filesystem Hierarchy Standard defines the common layout, so a similar structure appears on all distributions. Learning it is like learning a map of a city: once you know the districts, you can find anything. The tree uses forward slashes to separate names, and names are case sensitive. The root user's home is `/root`, which is separate from the `/home` directory used by ordinary users.

Key directories

Memorize these. `/etc` holds system configuration files, for example `/etc/passwd` and `/etc/ssh/sshd_config`. `/var` holds variable data, and `/var/log` contains log files such as `auth.log` or `syslog`. `/home` contains users' personal directories. `/tmp` is temporary space readable by all users and often cleared at reboot, a common place for attackers to drop files. `/bin` and `/usr/bin` hold commands such as `ls` and `cat`. `/sbin` holds administration programs. `/dev` holds device files, `/proc` exposes process and kernel information, and `/opt` holds optional third-party software.

Absolute and relative paths

An absolute path starts with a slash and gives the full route from the root, such as `/home/alice/projects/report.txt`. It means the same thing wherever you are. A relative path does not start with a slash and is interpreted from the current directory. If you are in `/home/alice`, `projects/report.txt` is the same file. Use `..` to go up: from `/home/alice/projects`, `../notes` refers to `/home/alice/notes`. The tilde expands to your home directory, so `~/projects` is `/home/alice/projects`. Use absolute paths in scripts and configuration for clarity, and relative paths for quick interactive work.

Using paths well

Paths can be given to any command, not just `cd`. `ls /var/log` lists logs from anywhere, and `cat /etc/hostname` prints a file regardless of location. When a path fails, check the spelling, check whether you started with a slash by mistake, and verify each directory exists with `ls`. Symbolic links act like shortcuts and may point elsewhere, which `ls -l` shows with an arrow. For investigation, remember that attackers hide in unusual places, such as hidden directories in `/tmp` or `/var/tmp`, so reading paths carefully matters. Combine knowledge of the layout with `ls -a` to review suspicious spots quickly.

DIRECTORY MAP

```
/
|-- etc/ configuration (passwd, ssh/, hostname)
|-- var/log/ logs (auth.log, syslog)
|-- home/ alice/ bob/
|-- tmp/ temporary, world-writable
|-- bin/ usr/bin/ commands
|-- dev/ proc/ devices, process info
+-- root/ root user's home
```

WORKED EXAMPLE

You are in `/home/alice/projects` and want to read `/var/log/auth.log`. As an absolute path, it works as is. A relative path would be `../../../../var/log/auth.log`, climbing three levels to the root and then down, which is clumsy, so the absolute form is better. To open a note two levels away, `../notes/todo.txt` is fine. If you type `var/log/auth.log` without a leading slash, the shell looks inside `/home/alice/projects/var/log` and reports No such file or directory. Understanding this difference resolves most beginner path errors.

KEY TERMS

Root directory: The top of the Linux file tree, written `/`.

Absolute path: A path starting from `/` that is valid from any location.

Relative path: A path interpreted from the current directory.

/etc: Directory holding system configuration files.

/var/log: Directory where system and application logs are stored.

REMEMBER

- The tree begins at /; know /etc, /var/log, /home, /tmp and /bin.
- Absolute paths start with /; relative paths start from where you are.
- Attackers often hide files in temporary and hidden locations.

QUICK CHECK

1. Which directory normally holds system configuration?

- A) /etc
- B) /tmp
- C) /dev
- D) /home

2. Which is an absolute path?

- A) notes/a.txt
- B) ../a.txt
- C) /home/alice/a.txt
- D) ./a.txt

3. Where are log files typically found?

- A) /var/log
- B) /bin
- C) /proc/log
- D) /boot/logs

4. Which directory is world-writable temporary space?

- A) /tmp
- B) /etc
- C) /bin
- D) /sbin

Answers: 1-A 2-C 3-A 4-A

1. /etc contains configuration files.
2. Absolute paths begin with a slash.
3. /var/log holds logs.
4. /tmp is shared temporary storage.

man Pages and Getting Help

Nobody memorizes every option, so the ability to look up commands quickly is a core professional skill.

Manual pages

Most Linux commands have a manual page, a built-in reference opened with the `man` command. Typing `man ls` opens the page for `ls` in a pager, a viewer that shows one screen at a time. Press the space bar to go forward, `b` to go back, `q` to quit, a slash followed by a word to search forward and `n` for the next match. The manual works offline, which helps on isolated servers. A page is organized into sections such as NAME, SYNOPSIS, DESCRIPTION, OPTIONS, EXAMPLES and SEE ALSO. Reading the first lines tells you what the command does in a sentence.

Reading the synopsis

The SYNOPSIS section shows how to write the command. Square brackets mean an item is optional, and an ellipsis means it may repeat. For `ls`, the synopsis looks like `ls [OPTION]... [FILE]...`, meaning options and file names are both optional and can be repeated. Words in capitals are placeholders for your own values. A vertical bar means choose one. The OPTIONS section lists each flag with its meaning, for example `-t` sort by time and `-S` sort by size. Search inside the page for the letter you are looking for, and read the example if one is provided.

--help, apropos and other tools

Many commands print a short summary when run with `--help`, for instance `ls --help`, which is faster than `man` for a quick reminder. The `apropos` command, equivalent to `man -k`, searches manual page descriptions by keyword, so `apropos copy` finds commands related to copying even if you do not know their names. `whatis` gives a one-line description. The type and which commands show where a program is located and whether it is an alias or a built-in. For shell built-ins like `cd`, use `help cd`. Online documentation and the `tldr` pages provide practical examples.

Learning independently and safely

Make lookup a habit: when you meet an unfamiliar flag in a command from a tutorial, a script or an attacker's shell history, read its manual entry before running it. Manual sections are numbered, and `man 5 passwd` shows the file format of `/etc/passwd` while `man 1 passwd` documents the command, so section numbers disambiguate names. Take notes of options you needed, because repetition builds recall. When a tool offers dry run or verbose flags, such as `-n` or `-v`, use them to preview effects. If the answer is not in the manual, search carefully and prefer official documentation.

GETTING HELP

```
$ ls --help | head -3
Usage: ls [OPTION]... [FILE]...
List information about the FILES (the current directory by default).
$ apropos -s 1 copy | head -2
cp (1) - copy files and directories
$ man -k passwd
passwd (1) - change user password
passwd (5) - password file
```

WORKED EXAMPLE

A tutorial tells you to run `ls -ltrh` but you do not know what it does. You open `man ls`, press slash and type `time`, and find `-t` sort by time, newest first, and `-r` reverse the order. A search for `human` shows `-h` prints sizes like `1K`, `234M`. Combined with `-l`, the command lists files in long format, with the oldest first and the newest last, using readable sizes. You then run it in your home directory to confirm. In under a minute you understood the flags and avoided guessing.

KEY TERMS

man page: The built-in manual entry that documents a command.
Synopsis: The manual section showing the command's syntax.
--help: A common flag that prints a brief usage summary.
apropos: A command that searches manual descriptions by keyword.
Pager: A program that displays long text one screen at a time.

REMEMBER

- man pages are the built-in reference; read NAME and SYNOPSIS first.
- Use --help for quick reminders and apropos to find commands.
- Look up unfamiliar flags before running them.

QUICK CHECK

1. Which command opens the manual for ls?
 - A) man ls
 - B) help ls
 - C) ls man
 - D) info ls only
2. In a synopsis, what do square brackets mean?
 - A) Optional item
 - B) Required item
 - C) Comment
 - D) Error
3. Which command searches manual pages by keyword?
 - A) apropos
 - B) pwd
 - C) cd
 - D) cat
4. How do you quit a man page?
 - A) q
 - B) x
 - C) Ctrl+Z
 - D) Enter

Answers: 1-A 2-A 3-A 4-A

1. man ls opens the manual page.
2. Brackets mark optional parts.
3. apropos, or man -k, searches descriptions.
4. The q key exits the pager.

First Terminal Session

SCENARIO

You have a terminal on a fictional Linux training machine, lab01, logged in as alice. A teammate left a note saying that project files are in a hidden folder somewhere under the home directory, but did not give the name. Use `pwd`, `ls` and `cd` to explore, reveal hidden entries, enter the folder and report its full path. Do not change or delete anything while exploring.

OBJECTIVE

Reach a hidden project folder and report its full path.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `pwd`

Simulator shows:
`/home/alice`

- 2 **Do:** Run: `ls`

Simulator shows:
`Desktop Documents notes.txt projects`

- 3 **Do:** Run: `ls -a`

Simulator shows:
`., .., .bash_logout, .bashrc, .profile, .project-x, Desktop, Documents, notes.txt, projects`

- 4 **Do:** Run: `cd .project-x`

Simulator shows:
(no output; prompt changes to `alice@lab01:~/project-x$`)

- 5 **Do:** Run: `ls -l`

Simulator shows:
`-rw-r--r-- 1 alice alice 412 Oct 1 09:00 README.md`

- 6 **Do:** Run: `pwd`

Simulator shows:
`/home/alice/.project-x`

- 7 **Do:** Run: `cd -` and then `pwd`

Simulator shows:
`/home/alice`

ANSWER

The hidden folder is `.project-x`, and its full path is `/home/alice/.project-x`. It appeared only with `ls -a` because its name starts with a dot. It contains `README.md`.

Hint shown in the app: Plain `ls` hides names that start with a dot.

Path Puzzle

SCENARIO

In a fictional training shell, you start in `/home/alice/projects/web`. Five target directories must be reached twice: first with relative paths only, then with absolute paths. The simulator prints your location after each move and tells you when a target is reached. Work out how many levels to climb with `..` and how to descend, and notice when a path with or without a leading slash behaves differently.

OBJECTIVE

Reach five directories using relative paths, then repeat with absolute paths.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `cd ..`

Simulator shows:
Now in `/home/alice/projects`.

- 2 **Do:** Run: `cd ../../Documents`

Simulator shows:
Target 1 reached: `/home/alice/Documents`.

- 3 **Do:** Run: `cd ../projects/web/css`

Simulator shows:
Target 2 reached: `/home/alice/projects/web/css`.

- 4 **Do:** Run: `cd ../../../../..`

Simulator shows:
Target 3 reached: `/ (root)`.

- 5 **Do:** Run: `cd etc/ssh`

Simulator shows:
Target 4 reached: `/etc/ssh`.

- 6 **Do:** Run: `cd ../../var/log`

Simulator shows:
Target 5 reached: `/var/log`.

- 7 **Do:** Repeat with absolute paths: `cd /home/alice/Documents`, `/var/log`, `/etc/ssh`

Simulator shows:
All accepted. Tip: an absolute path works from any directory.

ANSWER

Relative routes depend on the starting directory, while absolute paths such as `/home/alice/Documents`, `/home/alice/projects/web/css`, `/`, `/etc/ssh` and `/var/log` work from anywhere. Each `..` climbs one level toward root.

Hint shown in the app: Count the slashes from your location up to the common parent.

Find Where Logs Live

SCENARIO

A file explorer for a fictional Debian server shows the directories `/var/log` and `/etc`. Each file has a description hidden until you click it. You must match six files to their purposes: authentication events, general system messages, web server access records, user account list, SSH server settings and the hostname. Identify which are logs in `/var/log` and which are configuration in `/etc`.

OBJECTIVE

Match each file in `/var/log` and `/etc` to its purpose.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `/var/log` and select `auth.log`.

Simulator shows:

Description: login attempts and sudo use. Match: authentication events.

- 2 **Do:** Select `syslog`.

Simulator shows:

General system messages.

- 3 **Do:** Open `/var/log/apache2` and select `access.log`.

Simulator shows:

Web requests handled by the server.

- 4 **Do:** Open `/etc` and select `passwd`.

Simulator shows:

Lists user accounts, one per line.

- 5 **Do:** Select `/etc/ssh/sshd_config`.

Simulator shows:

SSH server configuration options.

- 6 **Do:** Select `/etc/hostname`.

Simulator shows:

Contains the machine name: `lab01`.

- 7 **Do:** Submit matches.

Simulator shows:

6 of 6 correct. Logs are in `/var/log`; settings are in `/etc`.

ANSWER

`auth.log` records authentication events, `syslog` general messages and `access.log` web requests, all under `/var/log`. `passwd` lists accounts, `sshd_config` configures SSH and `hostname` stores the machine name, all under `/etc`.

Hint shown in the app: Logs change over time and live in `/var`; settings live in `/etc`.

Decode the Flags

SCENARIO

A fictional training terminal asks you to produce a listing that is sorted by modification time, shows human-readable sizes and includes hidden files. You do not remember the options, so you should consult the manual with `man` and the built-in help instead of guessing. After finding the three flags, combine them in one command and run it in the current directory to confirm the result.

OBJECTIVE

Find and combine the `ls` options for time sort, readable sizes and hidden files.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `man ls`

Simulator shows:
Manual page opens. Press `/` to search.

- 2 **Do:** Search for hidden: type `/hidden` then Enter.

Simulator shows:
Match: `-a`, `--all` do not ignore entries starting with `.`

- 3 **Do:** Search for time: type `/sort by`.

Simulator shows:
Match: `-t` sort by time, newest first.

- 4 **Do:** Search for human: type `/human`.

Simulator shows:
Match: `-h`, `--human-readable` with `-l`, print sizes like `1K 234M`.

- 5 **Do:** Press `q`, then run: `ls -lath`

Simulator shows:
Lists all files with sizes like `4.0K` and `12K`, newest first, hidden files included.

- 6 **Do:** Run: `ls --help | head -2`

Simulator shows:
Usage: `ls [OPTION]... [FILE]...`

- 7 **Do:** Report the command.

Simulator shows:
Accepted: `ls -lath`.

ANSWER

The command is `ls -lath`: `-l` long format, `-a` includes hidden files, `-t` sorts by time, newest first, and `-h` gives human-readable sizes when combined with `-l`. Searching the man page with `/` found each option.

Hint shown in the app: Search the manual for the key word of each requirement.

Linux I: Meet the Terminal: 10 questions, 80% to pass

1. What is Linux strictly speaking?
 - A) A kernel
 - B) A desktop
 - C) A browser
 - D) A shell
2. What does a # prompt usually indicate?
 - A) Root user
 - B) Normal user
 - C) Error
 - D) Comment mode
3. In ls -l /etc, what is -l?
 - A) An option
 - B) An argument
 - C) The program
 - D) A path
4. Which command prints the current directory?
 - A) pwd
 - B) ls
 - C) cd
 - D) whoami
5. Which command lists hidden files too?
 - A) ls -a
 - B) ls -d
 - C) ls -x
 - D) ls -n
6. What does cd .. do?
 - A) Moves to the parent directory
 - B) Goes home
 - C) Goes to root
 - D) Lists files
7. Which is an absolute path?
 - A) /var/log/syslog
 - B) var/log/syslog
 - C) ../log
 - D) ./syslog
8. Where are system log files usually kept?
 - A) /var/log
 - B) /etc
 - C) /bin
 - D) /dev
9. Which directory holds configuration files?
 - A) /etc
 - B) /tmp
 - C) /proc
 - D) /home
10. Which command finds commands by keyword in manual descriptions?
 - A) apropos
 - B) pwd
 - C) cd
 - D) tail

Answers: 1-A 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. Linux is the kernel.
2. A hash prompt is typical for root.
3. It is a flag modifying ls.
4. pwd prints the working directory.
5. -a includes names beginning with a dot.
6. Double dot is the parent.
7. A leading slash makes it absolute.
8. /var/log holds logs.
9. /etc is for configuration.
10. apropos searches manual descriptions.

Linux II: Files and Folders

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

This module moves you from looking around the Linux file system to actively shaping it. You will create directories and files, copy, rename and delete them, read files and follow live logs, select groups of files with wildcards, and search the disk with find. These are the daily moves of every administrator and analyst, and they are also how evidence is collected, logs are read and mistakes are made. You will learn the safe habits that prevent accidental data loss.

BY THE END YOU CAN

- + Create, copy, move and delete files and directories safely
- + Read files with cat, less, head and tail
- + Locate files using wildcards and find

IN THIS MODULE

Lesson 1 mkdir and touch

Lesson 2 cp, mv and rm

Lesson 3 Reading Files: cat, less, head, tail

Lesson 4 Wildcards and Globbing

Lesson 5 Hidden Files and find

Lab 1 Build a Project Tree (Terminal)

Lab 2 Safe Cleanup (Terminal)

Lab 3 Follow the Log (Log viewer)

Lab 4 Hunt the Large File (Terminal)

Checkpoint 10 questions, 80% to pass

mkdir and touch

Every project, case folder and evidence directory starts with creating structure, and timestamps are a basic forensic clue.

Making directories with mkdir

The mkdir command creates a directory: mkdir reports creates one folder in the current location. If the parent does not exist, plain mkdir fails with 'No such file or directory'. The -p option solves this by creating the whole chain at once and by staying silent when the directory already exists, which makes it safe to put in scripts. For example, mkdir -p labs/week1/notes builds labs, then week1 inside it, then notes inside that. You can also create several siblings in one command, and brace expansion, such as mkdir -p proj/{src,docs,tests}, lets the shell expand the list for you before mkdir runs.

Creating files with touch

The touch command has two jobs. If the named file does not exist, it creates an empty file of size zero. If the file already exists, it leaves the contents alone and only updates the access and modification timestamps to the current time. That is why touch is used both to make placeholder files such as touch notes.txt and to refresh a file so that build tools or backups see it as new. You can give many names at once: touch a.txt b.txt c.txt. Redirection with > also creates an empty file, but it truncates an existing file, which touch never does.

Timestamps: atime, mtime and ctime

Linux files carry three main times. The mtime is the last time the content was modified, the atime is the last time it was read (often relaxed by the mount option relatime), and the ctime is the last time the inode metadata, such as permissions or owner, changed. You cannot set ctime directly, because the kernel updates it whenever the inode changes. The command ls -l shows mtime by default, ls -lu shows atime and ls -lc shows ctime. Investigators compare these values to build a timeline, and attackers who edit timestamps with touch -d still leave ctime behind as a clue.

Good naming habits

Names are case sensitive, so Report.txt and report.txt are different files. Avoid spaces, because the shell splits unquoted words; use hyphens or underscores, such as incident_2024-05-01.txt. If a name does contain spaces, quote it: touch 'my notes.txt'. Names starting with a dot are hidden. Use ls -l after every create operation to confirm what exists, and use pwd to confirm where you are. A habit of checking before and after catches typos like creating a directory in the wrong parent, which is among the most common beginner errors.

TERMINAL OUTPUT

```
$ mkdir -p labs/week1/notes
$ touch labs/week1/notes/todo.txt
$ ls -l labs/week1/notes
total 0
-rw-r--r-- 1 student student 0 Mar 3 09:14 todo.txt
$ touch labs/week1/notes/todo.txt
$ ls -l --time=ctime labs/week1/notes/todo.txt
```

WORKED EXAMPLE

A student needs a folder for an exercise on a host called lab01. She runs mkdir -p cases/case42/evidence and touch cases/case42/evidence/ hashes.txt. The listing shows a zero-byte file with a timestamp of the current minute. Later she runs touch again, and the size stays at zero while the time moves forward. She learns that touch never erases content, and that the directory chain was created in one step because of -p.

KEY TERMS

mkdir -p: Creates a directory and any missing parents, with no error if it exists.

touch: Creates an empty file or updates the timestamps of an existing one.

mtime: The time a file's content last changed.

ctime: The time a file's inode metadata last changed; it cannot be set directly.

Brace expansion: Shell feature that expands {a,b} into separate words before the command runs.

REMEMBER

- mkdir -p builds nested paths safely and quietly.
- touch creates empty files or refreshes timestamps without altering content.
- mtime, atime and ctime tell different stories, and ctime is hard to forge.

QUICK CHECK

1. What does mkdir -p a/b/c do when none of the folders exist?

- A) Fails with an error
- B) Creates a, b and c in a chain
- C) Creates only c
- D) Creates a file named c

2. What happens when touch is run on an existing file?

- A) Content is erased
- B) File is deleted
- C) Timestamps are updated, content kept
- D) It is renamed

3. Which timestamp changes when you chmod a file?

- A) mtime only
- B) ctime
- C) none
- D) filename

4. Why avoid spaces in file names?

- A) Linux forbids them
- B) The shell splits unquoted words
- C) They make files hidden
- D) They change permissions

Answers: 1-B 2-C 3-B 4-B

- 1. -p creates all missing parents.
- 2. touch only updates times for existing files.
- 3. ctime tracks inode metadata changes such as permissions.
- 4. Unquoted spaces break one name into several arguments.

cp, mv and rm

Copy, move and delete are the commands that can both save and destroy data, and rm has no safety net.

Copying with cp

The cp command duplicates files: `cp report.txt backup.txt` creates a new file with the same contents. To copy into a directory, name the directory last: `cp a.txt b.txt archive/`. Directories need the recursive flag, so `cp -r project project_copy` copies the whole tree. Add `-i` to be prompted before overwriting, and `-p` to preserve mode, ownership and timestamps, which matters when copying evidence or configuration files. Without `-i`, cp silently overwrites an existing destination, so a wrong target name can destroy a good file in one keystroke.

Moving and renaming with mv

The mv command both moves and renames, because in Linux a rename is just moving a file to a new name. `mv draft.txt final.txt` renames in place, while `mv final.txt archive/` moves the file into a folder. Within the same file system it is nearly instant, since only directory entries change; across file systems it copies and then deletes. Like cp, mv overwrites silently unless you add `-i`, and `-n` refuses to overwrite at all. Moving a directory needs no `-r`. Always check the destination with ls afterwards to confirm the file is where you intended.

Deleting with rm and why there is no recycle bin

The rm command unlinks files from the file system. There is no recycle bin or undo: the directory entry is removed and the space is marked free, and recovery afterwards is unreliable and often impossible. `rm file.txt` removes one file, `rm -i` asks first, `rm -r` removes a directory and everything inside it, and `-f` suppresses prompts and errors. The combination `rm -rf` is dangerous, especially with variables or wildcards: `rm -rf /home/$DIR/` with an empty DIR could wipe far more than intended. `rmdir` removes only empty directories, which is a safer choice.

Safe habits

Before deleting, run ls with the same pattern to see exactly what will match. Use `-i` for interactive work, and consider aliasing rm to `rm -i` in your own shell. Avoid running as root when practicing. Back up before bulk operations, and test destructive scripts on a copy. Use absolute paths in scripts and quote variables so that spaces or empty values do not alter the command. Remember that on shared servers, an attacker or a clumsy script that deletes logs is destroying evidence, so file deletion events are worth monitoring and protecting with backups and append-only logging.

TERMINAL OUTPUT

```
$ cp -i report.txt backup.txt
$ cp -r project project_copy
$ mv draft.txt final.txt
$ rm -i old.tmp
rm: remove regular file 'old.tmp'? y
$ rm -r project_copy
$ rmdir empty_dir
```

WORKED EXAMPLE

An analyst wants to rename notes.txt to notes_final.txt and keep an archive copy. She runs `cp -p notes.txt archive/notes_2024.txt`, then `mv notes.txt notes_final.txt`. Next she wants to delete the folder tmp_work, so first she types `ls tmp_work` to see what is inside, then `rm -r tmp_work`. Because she checked first and copied before moving, a typing mistake could be undone, whereas a bare `rm -rf` could not.

KEY TERMS

cp -r: Copies directories recursively.

mv: Moves or renames files and directories.

rm -i: Prompts before each deletion.

rm -rf: Recursive forced deletion with no prompts; very dangerous.

rmdir: Removes only empty directories.

REMEMBER

- cp copies, mv moves or renames, rm deletes permanently.

- Use -i and preview with ls before destructive commands.
- rm -rf with variables or wildcards is a classic way to lose data.

QUICK CHECK

1. Which flag makes cp ask before overwriting?

- A) -r
- B) -i
- C) -p
- D) -v

2. How do you rename a file in Linux?

- A) rename only
- B) mv old new
- C) cp old new
- D) touch old new

3. Why is rm risky?

- A) It is slow
- B) It has no recycle bin
- C) It needs root
- D) It changes ownership

4. Which command removes only an empty directory?

- A) rm -rf
- B) rmdir
- C) mv
- D) cp -r

Answers: 1-B 2-B 3-B 4-B

- 1. -i is interactive mode.
- 2. mv with a new name in the same folder renames.
- 3. Deleted files are unlinked and not recoverable in general.
- 4. rmdir refuses to remove a non-empty directory.

Reading Files: cat, less, head, tail

Logs and configuration files are plain text, and choosing the right viewer saves time and avoids flooding your screen.

cat for short files

The cat command prints a whole file to the terminal, and its name comes from concatenate, because cat a.txt b.txt joins files in order. It suits short files such as a one-line config or a hash value. Add -n to number lines. For large files it is a poor choice, because thousands of lines scroll past and you only see the end. cat is also commonly used to feed files into pipes, but many commands can read files directly. Do not cat binary files; the raw bytes can garble your terminal, and you can recover with the reset command.

less for paging

The less pager opens a file and lets you move through it without loading everything: space moves down a page, b moves back, g and G jump to start and end, /word searches forward and n repeats the search, and q quits. less -N shows line numbers and less +F follows a file. Unlike the older more, less can scroll backward and does not read the entire file first, so it opens huge logs quickly. For investigation, less is the default choice, because you can search for an IP address or error code and jump between hits.

head and tail for the ends

head prints the first lines of a file, ten by default, and tail prints the last ten. Use -n to choose: head -n 3 file or tail -n 20 /var/log/auth.log. Recent events sit at the bottom of most logs, so tail is the go-to command for what just happened. The option tail -f follows the file, printing new lines as they are appended, which lets you watch a login attempt arrive in real time; stop it with Ctrl+C. tail -F also copes with log rotation by reopening the file when it is replaced.

Choosing the right tool

Match the viewer to the job. Tiny file: cat. Big file you must explore: less. Header or format check of a CSV: head. Most recent events: tail. Live monitoring: tail -f. Combining them is common: head -n 100 file | tail -n 20 prints lines 81 to 100. Remember permissions apply, so many logs under /var/log need sudo to read. When reading logs as part of an investigation, never edit the original; view it read-only and copy it if you need to experiment, to keep evidence intact.

TERMINAL OUTPUT

```
$ head -n 2 /var/log/auth.log
Mar 3 08:01:12 web01 sshd[811]: Server listening on 0.0.0.0 port 22.
Mar 3 08:01:12 web01 sshd[811]: Server listening on :: port 22.
$ tail -n 1 /var/log/auth.log
Mar 3 09:20:41 web01 sshd[2210]: Failed password for invalid user admin from 203.0.113.9 port 51522 ssh2
$ tail -f /var/log/auth.log
```

WORKED EXAMPLE

An administrator on web01 hears that someone is guessing passwords. She runs tail -n 20 /var/log/auth.log with sudo and sees repeated Failed password lines from 203.0.113.9. To watch whether it continues, she opens sudo tail -f /var/log/auth.log and sees a new line every second. She then opens the whole file in less and searches /203.0.113.9 to count the first occurrence. Each tool fitted a different step of the same investigation.

KEY TERMS

- cat:** Prints a file, or concatenates several, to standard output.
- less:** A pager for viewing large files with scrolling and search.
- head:** Shows the first lines of a file.
- tail -f:** Shows the end of a file and follows new lines as they are added.
- Log rotation:** Periodic renaming and compression of logs so files do not grow forever.

REMEMBER

- Use cat for small files, less for exploring, head/tail for the ends.
- tail -f is the live-monitoring tool for logs.
- Read evidence files without modifying them.

QUICK CHECK

1. Which command shows the last 20 lines of a file?

- A) head -n 20
- B) tail -n 20
- C) cat -20
- D) less -20

2. Which exits less?

- A) x
- B) q
- C) Ctrl+S
- D) Esc Esc

3. What does tail -f do?

- A) Finds files
- B) Follows new lines as they are written
- C) Formats output
- D) Fixes permissions

4. Best viewer for a 2 GB log you want to search?

- A) cat
- B) less
- C) echo
- D) touch

Answers: 1-B 2-B 3-B 4-B

1. tail prints from the end.

2. q quits less.

3. -f keeps the file open and prints appended lines.

4. less opens quickly and supports /search.

Wildcards and Globbing

Globs let one command act on many files, but a wrong pattern can match more than you expect, so predicting matches is a core skill.

What globbing is

Globbing is the shell expanding a pattern into a list of matching file names before the command runs. The command never sees the star; it receives the expanded names. If you type `ls *.txt` in a folder with `a.txt` and `b.txt`, `ls` actually runs as `ls a.txt b.txt`. This means that `echo *.txt` is a safe way to preview what a pattern will match. If nothing matches, Bash by default passes the pattern through unchanged, which can cause confusing errors. Quoting the pattern, as in `'*.txt'`, stops the shell from expanding it.

The main wildcards

The asterisk `*` matches any number of characters, including none, so `log*` matches `log`, `log1` and `logfile.txt`. The question mark `?` matches exactly one character, so `file?.txt` matches `file1.txt` but not `file10.txt`. Square brackets match one character from a set or range: `[abc]` or `[a-z]` or `[0-9]`. A leading `!` or `^` inside brackets negates the set. Braces such as `{jpg,png}` are different: brace expansion generates words and does not depend on files existing. Globs do not match a leading dot by default, so `*` ignores hidden files.

Predicting matches

Practice by reading a directory and deciding the result first. Given `report1.txt`, `report2.txt`, `report10.txt`, `report_final.txt` and `notes.txt`, `report?.txt` matches `report1.txt` and `report2.txt` only. `report*.txt` matches all four report files. `report[0-9].txt` matches the two single digit files. `*.txt` matches everything, and `[a-n]*.txt` matches only `notes.txt` because `report` starts with `r`. Always preview with `ls` or `echo` before attaching `rm` to a pattern, as `rm *.tmp` is fine but `rm * .tmp`, with a stray space, deletes everything.

Globs versus regular expressions

Beginners confuse shell globs with regular expressions. In a glob, `*` means any characters; in a regex, `*` means repeat the previous item, and `.` is needed for any characters. Globs are handled by the shell and apply to file names, while regex is used by tools such as `grep` and `sed` on text. The same symbols mean different things in each, so check which tool is interpreting the pattern. Quoting matters: `grep 'a.*b' file` needs quotes so the shell leaves the pattern for `grep` to interpret.

TERMINAL OUTPUT

```
$ ls
notes.txt report1.txt report2.txt report10.txt old.tmp cache.tmp
$ echo report?.txt
report1.txt report2.txt
$ echo *.tmp
old.tmp cache.tmp
$ rm -i *.tmp
```

WORKED EXAMPLE

A folder holds `backup_01.tar` through `backup_12.tar` plus `backup_old.tar`. To list only the first nine numbered backups, the learner types `ls backup_0?.tar` and gets `backup_01` to `backup_09`. For all numbered ones she uses `backup_[0-9][0-9].tar`, which excludes `backup_old.tar`. Before removing anything, she runs `echo backup_1?.tar` to see that it matches 10, 11 and 12 only, then confirms the result and deletes with `rm -i`.

KEY TERMS

Glob: A shell pattern that expands to matching file names.

*****: Matches zero or more characters.

?: Matches exactly one character.

[a-z]: Matches one character in the range.

Brace expansion: Generates a list of words from `{a,b}` regardless of what files exist.

REMEMBER

- The shell, not the command, expands globs.

- * any run, ? one character, [] a set or range.
- Preview patterns with echo or ls before deleting.

QUICK CHECK

1. Which pattern matches file1.txt but not file10.txt?

- A) file*.txt
- B) file?.txt
- C) file[0-9]*
- D) *.txt

2. Who expands a glob?

- A) The command
- B) The shell
- C) The kernel
- D) The file system

3. Does * match hidden files by default?

- A) Yes
- B) No
- C) Only in /tmp
- D) Only as root

4. Safest way to preview rm *.tmp?

- A) Run it and see
- B) echo *.tmp
- C) rm -f *.tmp
- D) mv *.tmp /

Answers: 1-B 2-B 3-B 4-B

- 1. ? matches exactly one character.
- 2. The shell expands it before running the command.
- 3. Leading dots must be matched explicitly.
- 4. echo shows the expansion without deleting.

Hidden Files and find

Configuration and persistence often hide in dotfiles, and find lets you locate files by properties when you do not know the path.

Hidden files

In Linux a file or folder whose name starts with a dot is hidden from the default ls output. This is a convention, not security: ls -a shows them all, and ls -A omits the . and .. entries. User settings live in dotfiles in the home directory, such as .bashrc for shell setup, .profile, .ssh/ for keys and authorized_keys, and .bash_history for command history. Because they are inconspicuous, malware and attackers sometimes hide files this way, for example a folder named '..' with trailing spaces, so reviewers always look with ls -la.

Finding by name and type

The find command walks a directory tree and tests each entry. The syntax is find START TESTS ACTIONS. find /home -name '*.conf' finds by name, and -iname ignores case. Quote the pattern so find, not the shell, handles it. Use -type f for regular files and -type d for directories. If you omit the start path, current directory is used on most systems, but giving it explicitly is clearer. Searching / without privileges prints 'Permission denied' for protected areas; redirect with 2>/dev/null to hide those messages, or use sudo.

Finding by size and time

find can filter by size and age. -size +50M matches files larger than 50 megabytes, and -size -1k matches under one kilobyte. For time, -mtime -1 means modified less than one day ago, -mtime +30 means more than thirty days ago, and -mmin -10 uses minutes. Tests combine as AND by default, so find /var -type f -size +50M -mtime -1 finds large files changed in the last day. This is how you track down a log that suddenly filled the disk, or spot files an intruder dropped recently.

Actions and care

By default find prints matches, but -exec runs a command for each one: find . -name '*.tmp' -exec rm -i {} \; where {} is the matched name. -delete removes matches directly and must come last in the expression, so test the same command without it first. Another tool, locate, searches a prebuilt database and is faster, but may be stale until updatedb runs. For security work, useful searches include files with world-write permission, recently modified files in /etc, and unexpected files in /tmp.

TERMINAL OUTPUT

```
$ ls -a ~
. .. .bash_history .bashrc .ssh notes.txt
$ find /var/log -type f -size +50M -mtime -1 2>/dev/null
/var/log/app/debug.log
$ find ~ -iname '*.conf' -type f
```

WORKED EXAMPLE

Disk usage on app01 jumped overnight. The administrator runs find /var -type f -size +50M -mtime -1 2>/dev/null and gets one hit: /var/log/app/debug.log. She checks with ls -lh, finds it is 2 GB because debug logging was left on, and compresses it. The same combination of size and mtime tests found the cause in seconds, whereas browsing folders by hand would have taken far longer.

KEY TERMS

Dotfile: A file whose name begins with a dot and is hidden from default listings.

find: Command that searches a directory tree using tests on name, type, size and time.

-mtime -1: Matches files modified within the last day.

-exec: Runs a command on each file that find matches.

locate: Fast search over a prebuilt file name database.

REMEMBER

- Dotfiles are hidden by convention only; use ls -la.

- find filters by name, type, size and time and can act on results.
- Test with print first, then add -exec or -delete.

QUICK CHECK

1. Which command lists hidden files too?
 - A) ls -l
 - B) ls -a
 - C) ls -h
 - D) ls -r
2. What does find / -size +50M match?
 - A) Files under 50 MB
 - B) Files over 50 MB
 - C) Empty files
 - D) Directories
3. What does -mtime -1 mean?
 - A) Modified over a day ago
 - B) Modified within the last day
 - C) Created yesterday only
 - D) Accessed once
4. Why quote '*.conf' in find -name?
 - A) It is required for speed
 - B) So find rather than the shell handles the pattern
 - C) To make it hidden
 - D) To ignore case

Answers: 1-B 2-B 3-B 4-B

1. -a includes names that begin with a dot.
2. + means greater than.
3. Negative values mean less than that many days ago.
4. Otherwise the shell may expand it first.

Build a Project Tree

SCENARIO

You are setting up a workspace for a fictional team project named webapp on host lab01. The layout must hold source code, documentation and test data, with a few starter files, and you must verify the result recursively before handing it over.

OBJECTIVE

Create a nested directory layout with `mkdir -p` and `touch` and verify it with `ls -R`.

STEPS IN THE SIMULATOR

- 1 **Do:** Type `pwd` to confirm location.

Simulator shows:
/home/student

- 2 **Do:** Type `mkdir -p webapp/{src,docs,tests/data}`

Simulator shows:
(no output; directories created)

- 3 **Do:** Type `touch webapp/src/main.py webapp/docs/README.txt webapp/tests/data/sample.csv`

Simulator shows:
(no output)

- 4 **Do:** Type `ls -R webapp`

Simulator shows:
webapp:
docs src tests

webapp/docs:
README.txt

webapp/src:
main.py

webapp/tests:
data

webapp/tests/data:
sample.csv

- 5 **Do:** Type `ls -l webapp/src`

Simulator shows:
-rw-r--r-- 1 student student 0 Mar 3 10:02 main.py

- 6 **Do:** Type `mkdir webapp/tests/data/extra`

Simulator shows:
(no output, but extra is created since parent exists)

ANSWER

The tree webapp with `src/main.py`, `docs/README.txt` and `tests/data/sample.csv` exists, confirmed by `ls -R`. `mkdir -p` created nested folders in one command, and `touch` created zero-byte files.

Hint shown in the app: Use brace expansion inside `mkdir -p` to make several folders at once.

Safe Cleanup

SCENARIO

In a fictional reports folder on host lab01 you find q1.txt, q2.txt, q3.txt and three leftovers named a.tmp, b.tmp and c.tmp. Your manager wants the reports archived and only the temporary files removed, without touching anything else.

OBJECTIVE

Copy and move report files, then delete only the .tmp files using wildcards and rm -i.

STEPS IN THE SIMULATOR

1 Do: Type ls

Simulator shows:

a.tmp b.tmp c.tmp q1.txt q2.txt q3.txt

2 Do: Type mkdir archive

Simulator shows:

(no output)

3 Do: Type cp q*.txt archive/

Simulator shows:

(no output; three files copied)

4 Do: Type mv q3.txt q3_final.txt

Simulator shows:

(no output)

5 Do: Type echo *.tmp

Simulator shows:

a.tmp b.tmp c.tmp

6 Do: Type rm -i *.tmp

Simulator shows:

rm: remove regular file 'a.tmp'? (answer y for each)

7 Do: Type ls

Simulator shows:

archive q1.txt q2.txt q3_final.txt

ANSWER

Only the three .tmp files were removed, the reports remain, and archive holds copies of q1 to q3. Previewing with echo and using rm -i prevented deleting the wrong files.

Hint shown in the app: Preview the glob with echo before attaching rm to it.

Follow the Log

SCENARIO

A fictional web server web01 writes `/var/log/auth.log`. Someone reports possible password guessing against SSH in the last few minutes. You must find the newest failed login lines and the first lines of the file to confirm the log start time.

OBJECTIVE

Use `tail` and `head` on a simulated auth log to find the most recent failed login lines.

STEPS IN THE SIMULATOR

- 1 **Do:** Run `head -n 3 auth.log`

Simulator shows:

```
Mar 3 08:00:01 web01 CRON[501]: session opened for user root
Mar 3 08:00:02 web01 CRON[501]: session closed
Mar 3 08:01:12 web01 sshd[811]: Server listening on 0.0.0.0 port 22.
```

- 2 **Do:** Run `tail -n 5 auth.log`

Simulator shows:

```
Mar 3 09:20:37 web01 sshd[2210]: Failed password for invalid user admin from 203.0.113.9 port 51520 ssh2
(4 more similar lines for users root, test, oracle, guest)
```

- 3 **Do:** Run `tail -f auth.log` and wait

Simulator shows:

```
New line appears: 09:20:44 ... Failed password for invalid user ubuntu from 203.0.113.9 port 51531 ssh2
```

- 4 **Do:** Stop with `Ctrl+C`

Simulator shows:

```
Prompt returns.
```

- 5 **Do:** Run `tail -n 50 auth.log`

Simulator shows:

```
All failed attempts share one source address
```

ANSWER

The most recent failed logins all come from 203.0.113.9 against many usernames, with new attempts every few seconds, which indicates automated password guessing against SSH.

Hint shown in the app: Recent events are at the end of the file, so `tail` beats `head`.

Hunt the Large File

SCENARIO

A fictional server app01 has a nearly full disk after a night job. You suspect one file of about 50 MB or more under /var/tmp was written in the last day. Search without scanning by hand and ignore permission noise.

OBJECTIVE

Use find with -name, -size and -mtime to locate a 50 MB file modified in the last day.

STEPS IN THE SIMULATOR

- 1 **Do:** Type `find /var/tmp -type f`

Simulator shows:
(long list of 40 files)

- 2 **Do:** Type `find /var/tmp -type f -size +50M`

Simulator shows:
/var/tmp/export_old.dat
/var/tmp/export_new.dat

- 3 **Do:** Add the time test: `find /var/tmp -type f -size +50M -mtime -1`

Simulator shows:
/var/tmp/export_new.dat

- 4 **Do:** Narrow by name: `find /var/tmp -name 'export*.dat' -size +50M -mtime -1`

Simulator shows:
/var/tmp/export_new.dat

- 5 **Do:** Type `ls -lh /var/tmp/export_new.dat`

Simulator shows:
-rw-r--r-- 1 svc svc 52M Mar 3 02:14 /var/tmp/export_new.dat

ANSWER

The file is /var/tmp/export_new.dat, 52 MB, modified at 02:14 today. The size test alone found two files, and the -mtime -1 test excluded the older export_old.dat.

Hint shown in the app: Combine -size +50M with -mtime -1; tests are ANDed.

Linux II: Files and Folders: 10 questions, 80% to pass

1. Which command creates nested directories in one step?

- A) mkdir
- B) mkdir -p
- C) touch -p
- D) mv -p

2. touch on an existing file does what?

- A) Truncates it
- B) Updates timestamps
- C) Deletes it
- D) Hides it

3. Why is rm dangerous?

- A) It needs a license
- B) Files are not moved to a recycle bin
- C) It always asks twice
- D) It changes ownership

4. Which flag makes cp or rm prompt first?

- A) -r
- B) -i
- C) -v
- D) -a

5. Best way to watch new lines in a log as they arrive?

- A) cat log
- B) tail -f log
- C) head log
- D) less log

6. report?.txt matches which file?

- A) report10.txt
- B) report1.txt
- C) report.txt
- D) reports.txt

7. Who expands wildcards such as *.txt?

- A) The command
- B) The shell
- C) The kernel
- D) The disk

8. Which command lists dotfiles in the current folder?

- A) ls
- B) ls -a
- C) ls -S
- D) ls -d

9. find / -type f -size +50M finds...

- A) Directories over 50 MB
- B) Regular files over 50 MB
- C) Files younger than 50 days
- D) Empty files

10. Which timestamp is updated when permissions change?

- A) mtime
- B) ctime
- C) btime only
- D) none

Answers: 1-B 2-B 3-B 4-B 5-B 6-B 7-B 8-B 9-B 10-B

1. -p creates missing parents.
2. touch refreshes times only.
3. Deletion is permanent in practice.
4. -i is interactive.
5. tail -f follows appended lines.
6. ? is exactly one character.
7. The shell expands globs.
8. -a shows hidden entries.
9. -type f plus +50M.
10. ctime records inode changes.

Linux III: Users and Permissions

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

Linux decides who may read, change or run every file using users, groups and permission bits. This module teaches you to read account files, decode `ls -l` output, set modes with `chmod` in symbolic and octal form, change owners with `chown` and `chgrp`, and understand `setuid`, `setgid` and the sticky bit. You will also learn `sudo` and least privilege. Misconfigured permissions are one of the most common causes of privilege escalation and data exposure, so this is core defensive knowledge.

BY THE END YOU CAN

- + Interpret `ls -l` output and numeric permissions
- + Change ownership and modes with `chown` and `chmod`
- + Apply `sudo` and least privilege correctly

IN THIS MODULE

Lesson 1 Users, Groups and `/etc/passwd`

Lesson 2 Reading `ls -l` Output

Lesson 3 `chmod`: Symbolic and Octal Modes

Lesson 4 `chown`, `chgrp` and Special Bits

Lesson 5 `sudo` and Least Privilege

Lab 1 Decode the Permissions (Scenario)

Lab 2 Fix the Broken Script (Terminal)

Lab 3 Lock Down a Secrets File (Terminal)

Lab 4 Audit `sudoers` and Accounts (Config editor)

Checkpoint 10 questions, 80% to pass

Users, Groups and /etc/passwd

Permissions only make sense once you know who the users and groups are, and account files are a standard audit target.

Accounts and UIDs

Every process and file in Linux is tied to numbers, not names. A user has a user ID (UID), and the kernel uses the number for access checks. UID 0 is root, the superuser with unrestricted power. System and service accounts such as www-data or daemon usually have low UIDs, commonly below 1000, while regular people typically start at 1000. A user also has a primary group ID (GID) and may belong to supplementary groups. Because the kernel only cares about numbers, a second account with UID 0 is just as powerful as root, even if it is named something innocent.

The /etc/passwd file

The file /etc/passwd lists accounts, one per line, with seven colon-separated fields: username, password placeholder, UID, GID, comment (GECOS), home directory and login shell. For example, `alice:x:1001:1001:Alice`
Example: `/home/alice:/bin/bash`. The x means the password hash lives elsewhere. A shell of `/usr/sbin/nologin` or `/bin/false` means the account cannot log in interactively, which is the right setting for service accounts. The file must be world-readable so programs can map UIDs to names, so it holds no secrets, but changes to it are security relevant.

/etc/shadow and /etc/group

The file /etc/shadow stores password hashes and aging information, and is readable only by root and the shadow group, typically mode 640 or 600. A hash field that starts with `$y$` or `$6$` indicates a modern hashing scheme, while `!` or `*` means the password is locked. The file /etc/group lists groups in the form `name:x:GID:members`. Splitting hashes into shadow was done so ordinary users cannot copy them and attempt offline cracking. If shadow is readable by everyone, that is a serious finding because it exposes every account to password guessing.

id, whoami and groups

Three commands answer who am I. `whoami` prints the effective username. `id` prints the UID, GID and every group, for example `uid=1001(alice) gid=1001(alice) groups=1001(alice),27(sudo)`. The `groups` command lists group names alone, and `id bob` shows another user. Membership in groups such as `sudo` or `adm` grants extra rights, so reviewing it is a routine audit. Use `getent passwd` to include accounts from directory services, and `awk -F: '$3==0' /etc/passwd` to list every UID 0 account, a quick check for a hidden root-equivalent user.

TERMINAL OUTPUT

```
$ whoami
alice
$ id
uid=1001(alice) gid=1001(alice) groups=1001(alice),27(sudo)
$ grep alice /etc/passwd
alice:x:1001:1001:Alice Example:/home/alice:/bin/bash
$ awk -F: '$3==0 {print $1}' /etc/passwd
root
```

WORKED EXAMPLE

During an audit of host `srv01`, an analyst runs `awk -F: '$3==0 {print $1}' /etc/passwd` and gets two names: `root` and `support`. The second account has UID 0, so it is a second superuser even though its name looks harmless. She also notes its shell is `/bin/bash` while genuine service accounts use `nologin`. She records it as a high-severity finding and recommends checking when it was added and who created it.

KEY TERMS

UID: The numeric identifier the kernel uses for a user; 0 is root.

GID: The numeric identifier of a group.

/etc/shadow: Root-only file holding password hashes and aging data.

Shell field: The last passwd field; `nologin` blocks interactive logins.

Supplementary group: An extra group a user belongs to beyond the primary group.

REMEMBER

- Access is decided by UID and GID numbers.
- /etc/passwd is readable and has seven fields; hashes are in /etc/shadow.
- Any extra UID 0 account is a root-equivalent backdoor.

QUICK CHECK

1. Which UID is the superuser?
 - A) 1
 - B) 0
 - C) 1000
 - D) 65534
2. How many fields does a passwd line have?
 - A) 5
 - B) 6
 - C) 7
 - D) 9
3. Where are password hashes kept?
 - A) /etc/passwd
 - B) /etc/shadow
 - C) /etc/hosts
 - D) /var/log/auth.log
4. Which command lists your groups and UID?
 - A) whoami
 - B) id
 - C) pwd
 - D) uname

Answers: 1-B 2-C 3-B 4-B

1. UID 0 is root.
2. Seven colon-separated fields.
3. Hashes are in the restricted shadow file.
4. id shows uid, gid and groups.

Reading ls -l Output

Every permission audit starts with decoding the ten-character mode string and owner fields.

The anatomy of a line

A long listing line looks like `-rw-r--r-- 1 alice staff 2048 Mar 3 09:10 notes.txt`. Reading left to right: the first character is the file type, then nine permission characters, then the link count, the owner, the group, the size in bytes, the modification date and time, and the name. Use `ls -lh` for human-readable sizes and `ls -la` to include hidden files. Directories show a total line first. Learning to scan this line quickly lets you spot a risky file in seconds.

The type character

The first character tells you what kind of object it is. A dash means a regular file, `d` is a directory, `l` is a symbolic link (with an arrow to its target), `c` is a character device, `b` is a block device, `s` is a socket and `p` is a named pipe. Links matter for security because a symlink can point somewhere unexpected, so a world-writable directory containing links deserves attention. Devices live under `/dev`. When you see a `d`, remember the permission bits mean something slightly different, as the next section explains.

Owner, group and others triplets

The nine permission characters form three triplets: owner (user), group, and others. Each triplet is `r` for read, `w` for write and `x` for execute, with a dash when absent. In `-rwxr-x---` the owner has `rwx`, the group has `r-x`, and others have nothing. The kernel checks in order: if you are the owner, owner bits apply; otherwise if you are in the file's group, group bits apply; otherwise others apply. For directories, `r` lets you list names, `w` lets you create or delete entries, and `x` lets you enter and access items inside.

What to look for

Red flags include world-writable files (others have `w`), such as `-rw-rw-rw-`, scripts run by root that others can edit, private keys with group or other access, and an `s` in place of `x`, which means `setuid` or `setgid`. A trailing `+` after the mode means an extra ACL exists. Compare against expectations: `/etc/shadow` should not be world-readable, SSH private keys should be 600, and home directories are commonly 750 or 700. Use `stat file` to see the same information in octal along with all timestamps.

TERMINAL OUTPUT

```
$ ls -l
-rw-r--r-- 1 alice staff 2048 Mar 3 09:10 notes.txt
drwxr-x--- 2 alice staff 4096 Mar 3 09:11 private
-rwxrwxrwx 1 root root 312 Mar 2 17:40 backup.sh
lrwxrwxrwx 1 root root 9 Mar 1 08:00 log -> /var/log
$ stat -c '%a %U %n' backup.sh
777 root backup.sh
```

WORKED EXAMPLE

The line `-rwxrwxrwx 1 root root 312 Mar 2 17:40 backup.sh` shows a regular file owned by root, executable and writable by everyone. If cron runs it as root, any local user could append a command and gain root. The analyst reads the owner triplet, group triplet and others triplet separately, notes all three are `rwx`, and recommends mode 750 or 700 plus a review of what the script contains.

KEY TERMS

Mode string: The ten-character field with file type and permission bits.

Triplet: One group of three `r`, `w`, `x` flags for owner, group or others.

Link count: Number of hard links to the inode; for directories it reflects subdirectories.

Symlink: A file of type `l` that points to another path.

ACL: An extended access control list shown by a `+` after the mode.

REMEMBER

- Read type, then owner, group and others triplets in order.
- Directory `r`, `w`, `x` mean list, modify entries and enter.

- World-writable and setuid files deserve scrutiny.

QUICK CHECK

1. What does the first character d mean?

- A) Deleted
- B) Directory
- C) Device only
- D) Data file

2. In -rwxr-x--- who can write?

- A) Everyone
- B) Owner only
- C) Owner and group
- D) Group only

3. What does w mean on a directory?

- A) Edit file contents
- B) Create or delete entries in it
- C) Search inside
- D) Rename the disk

4. Which is a red flag?

- A) -rw-r--r-- on notes.txt
- B) -rw-rw-rw- on a root cron script
- C) drwx----- on ~/.ssh
- D) -rw----- on a key

Answers: 1-B 2-B 3-B 4-B

- 1. d marks a directory.
- 2. Only the owner triplet has w.
- 3. Directory write governs entries.
- 4. World-writable script run by root allows takeover.

chmod: Symbolic and Octal Modes

chmod is how you fix or break access, and octal math is a common exam and audit skill.

Symbolic mode

Symbolic mode names who, an operator and the permissions. Who is u for user (owner), g for group, o for others and a for all. The operators are + to add, - to remove and = to set exactly. So `chmod u+x script.sh` adds execute for the owner, `chmod go-w file` removes write from group and others, and `chmod a=r file` makes it read-only for everyone. You can combine clauses with commas: `chmod u=rwx,g=rx,o= tool`. Symbolic mode changes only what you name, which makes it safer for small adjustments than retyping a full number.

Octal mode

Each permission has a value: read is 4, write is 2 and execute is 1. Add them per triplet to build one digit, then write three digits for owner, group and others. So `rwx` is 7, `rw-` is 6, `r-x` is 5, `r--` is 4 and `---` is 0. Thus 755 is `rw-r-x-r-x`, 644 is `rw-r--r--`, 640 is `rw-r-----`, 600 is `rw-----` and 777 is everyone full control. `chmod 640 report.txt` gives the owner read and write, the group read, and others nothing. Memorizing these few common values is worth the effort.

Directories, umask and recursion

Directories need `x` to be entered, so typical modes are 755 or 750, not 644. New files start from a base of 666 and directories from 777, then the `umask` removes bits; with `umask 022` files become 644 and directories 755, and with `umask 077` they become 600 and 700. The `-R` flag applies a change recursively, and beware that `chmod -R 644` on a tree removes execute from directories, breaking access. Use `find` with `-type f` and `-type d` to set files and directories separately, such as `find . -type d -exec chmod 750 {} +`.

Choosing safe modes

Start from least access and add only what is required. Scripts that someone runs need `x` for that person; data files rarely need `x` at all. Secrets such as API keys, private keys and password files should be 600 or 640 with a restricted group. Web content typically is readable by the web server but not writable by it. Never solve a permission denied message with `chmod 777`, because it opens the file to every local user and any compromised service account. Always confirm with `ls -l` after the change, and remember you can only `chmod` files you own, unless you are root.

TERMINAL OUTPUT

```
$ ls -l deploy.sh
-rw-r--r-- 1 alice alice 120 Mar 3 10:00 deploy.sh
$ chmod u+x deploy.sh
$ chmod 750 deploy.sh
$ ls -l deploy.sh
-rwxr-x--- 1 alice alice 120 Mar 3 10:00 deploy.sh
```

WORKED EXAMPLE

A script `deploy.sh` shows `-rw-r--r--` and fails with Permission denied. The owner needs to run it, the team group needs to run but not edit it, and strangers must not see it. Using octal: owner `rwx` is 7, group `r-x` is 5, others none is 0, so `chmod 750 deploy.sh`. The learner verifies with `ls -l` and sees `-rwxr-x---`. A symbolic equivalent would be `chmod u=rwx,g=rx,o= deploy.sh`.

KEY TERMS

chmod: Command that changes the permission bits of files.

Octal mode: Numeric notation where `r=4`, `w=2`, `x=1` summed per triplet.

Symbolic mode: Notation such as `u+x` or `go-w` that adds, removes or sets bits.

umask: Mask that removes permission bits from newly created files and directories.

-R: Option that applies a change recursively through a directory tree.

REMEMBER

- Octal digits add `r=4`, `w=2`, `x=1` for each of owner, group, others.
- Symbolic mode changes only the bits you name.
- Use least permissions, and never 777 to fix errors.

QUICK CHECK

1. What does `chmod 640` give the group?
 - A) Nothing
 - B) Read
 - C) Read and write
 - D) Execute
2. Octal for `rxr-xr-x`?
 - A) 644
 - B) 755
 - C) 775
 - D) 700
3. What does `chmod go-w` file do?
 - A) Adds write for all
 - B) Removes write from group and others
 - C) Sets mode 000
 - D) Makes owner read only
4. With `umask 022` a new file is?
 - A) 666
 - B) 644
 - C) 600
 - D) 755

Answers: 1-B 2-B 3-B 4-B

1. 6 is for owner, 4 for group is read only.
2. 7,5,5.
3. The `-` operator removes bits for `g` and `o`.
4. 666 minus 022 gives 644.

chown, chgrp and Special Bits

Ownership decides who controls a file, and special bits can hand out elevated rights, so they are frequent privilege-escalation targets.

chown and chgrp

The `chown` command changes a file's owner and, optionally, group: `chown alice file` sets the owner, `chown alice:developers file` sets both, and `chown :developers file` sets only the group, equivalent to `chgrp developers file`. The `-R` option recurses through directories. On Linux only root can give a file away to another owner, so ordinary users run these commands through `sudo`. A typical use is giving a service account ownership of its data, such as `sudo chown -R svc-app:svc-app /var/lib/app`, so the service can run without root and other accounts cannot touch the data.

Setuid and setgid

A `setuid` program runs with the privileges of its owner instead of the caller. It shows as `s` in the owner execute position, such as `-rwsr-xr-x`, and has octal value 4000. The classic example is `/usr/bin/passwd`, which is owned by root so ordinary users can update the protected shadow file. `Setgid`, value 2000, runs with the file's group, and on a directory it makes new files inherit the directory's group, which is useful for shared project folders. Because a bug in a root-owned `setuid` program can give an attacker root, the list should be short and reviewed.

The sticky bit

The sticky bit, value 1000, appears as `t` in the others execute position. On a directory it means that files inside can be deleted or renamed only by the file's owner, the directory's owner or root, even if the directory is world-writable. This is why `/tmp` is mode 1777, shown as `drwxrwxrwt`: everyone can create files there, but users cannot delete each other's files. Without it, any user could remove another user's temporary files or swap them for malicious ones. Set it with `chmod +t dir` or `chmod 1777 dir`.

Auditing special bits

Defenders enumerate `setuid` and `setgid` files regularly with `find / -perm -4000 -type f 2>/dev/null` and `find / -perm -2000 -type f`. Compare the list with a known baseline: new entries, or `setuid` on a shell, an editor or a copy of an interpreter, indicate possible compromise or misconfiguration. Capital `S` or `T` in a listing means the bit is set but execute is not, which is usually a mistake. Remove unneeded bits with `chmod u-s file`. Mounting user-writable file systems with the `nosuid` option also prevents `setuid` from taking effect there.

TERMINAL OUTPUT

```
$ sudo chown svc-app:svc-app /var/lib/app/data.db
$ ls -ld /tmp
drwxrwxrwt 18 root root 4096 Mar 3 10:30 /tmp
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 59976 Feb 6 2024 /usr/bin/passwd
$ find / -perm -4000 -type f 2>/dev/null | head -n 3
```

WORKED EXAMPLE

A weekly audit of host `srv01` lists `setuid` files and compares them to last week's baseline. One new entry appears: `/usr/local/bin/vim.tiny` with `-rwsr-xr-x` owned by root. A `setuid` editor lets any user open and overwrite protected files such as `/etc/shadow`. The analyst removes the bit with `chmod u-s`, notes the time from `stat` for the incident log, and checks who installed the file.

KEY TERMS

chown: Changes the owner and optionally the group of a file.

Setuid: Bit (4000) that runs a program as its file owner.

Setgid: Bit (2000) that runs as the file group or makes a directory inherit its group.

Sticky bit: Bit (1000) that restricts deletion in a shared directory to the owner.

nosuid: Mount option that ignores `setuid` and `setgid` bits.

REMEMBER

- chown and chgrp set ownership; recursion needs -R.
- Setuid and setgid run with the file's owner or group privileges.
- Audit setuid files and use nosuid where users can write.

QUICK CHECK

1. Why does /tmp have the sticky bit?
 - A) To make it faster
 - B) So users cannot delete others' files
 - C) To hide it
 - D) To allow setuid
2. What does an s in -rwsr-xr-x mean?
 - A) Setuid is set
 - B) Sticky is set
 - C) Symlink
 - D) Secure
3. Who can normally change a file's owner on Linux?
 - A) Any user
 - B) Only root
 - C) The group
 - D) Nobody
4. Which command finds setuid files?
 - A) find / -perm -4000 -type f
 - B) ls -s
 - C) chmod 4000
 - D) stat /

Answers: 1-B 2-A 3-B 4-A

1. Sticky limits deletion to owners.
2. s in the owner execute slot is setuid.
3. Giving files away requires root.
4. -perm -4000 matches the setuid bit.

sudo and Least Privilege

Working as root all the time turns small mistakes and malware into total compromise; sudo gives controlled, logged elevation.

What sudo does

The sudo command runs a single command as another user, by default root, after checking policy and typically asking for the caller's own password. Unlike su, it does not share the root password, and each use is recorded in the log, usually /var/log/auth.log or the journal, with the user, working directory and command. This gives accountability. sudo -l lists what you are allowed to run, and sudo -u svc-app command runs as a particular user. Members of the sudo group on Debian-style systems or the wheel group on Red Hat-style systems are normally granted full rights.

Editing sudoers safely

Policy lives in /etc/sudoers and drop-in files under /etc/sudoers.d. Always edit with visudo, which locks the file and checks syntax, because a broken sudoers file can lock everyone out of elevation. A rule has the form user host=(runas) commands, for example alice ALL=(ALL) ALL, which gives full rights, versus bob ALL=(root) /usr/bin/systemctl restart nginx, which allows exactly one command. NOPASSWD removes the password prompt and should be limited to specific tasks. Wildcards and commands that open shells or editors, like vi or less, can be abused to escape the restriction.

Why root is risky

Root bypasses normal permission checks, so any program running as root can read or change anything. If a web server runs as root and is compromised, the attacker is root. Everyday work such as browsing, editing documents or reading mail should happen as an unprivileged user. Many distributions disable direct root login over SSH with PermitRootLogin no, forcing attackers to compromise a user first and then escalate, which creates extra events to detect. Services should run under dedicated accounts with only the files they need.

Least privilege and 777

The principle of least privilege says each user, process and file should have only the access needed for its job. Applied to files, that means narrow modes and owners; applied to sudo, narrow command lists. A mode of 777 is a vulnerability because any local user, and any compromised service, can read, change or replace the file, and if it is a script or directory used by a privileged process, can escalate to root. Fix permission errors by correcting the owner or group instead. Review sudoers and group membership regularly, remove unused accounts, and log elevation.

CONFIG EXCERPT

```
# /etc/sudoers.d/ops (edit with visudo)
alice ALL=(ALL) ALL
bob ALL=(root) /usr/bin/systemctl restart nginx
# risky rule:
%contractors ALL=(ALL) NOPASSWD: ALL
```

WORKED EXAMPLE

A contractor group has the rule %contractors ALL=(ALL) NOPASSWD: ALL, so any member can become root without a password. During review, the administrator replaces it with a rule that allows only /usr/bin/systemctl restart nginx for the group, edits through visudo, and tests with sudo -l -U carol. The change keeps the job doable while limiting damage if a contractor password is stolen.

KEY TERMS

sudo: Runs a command with elevated privileges according to policy and logs it.

visudo: Safe editor for sudoers that validates syntax.

NOPASSWD: Sudoers tag that skips the password prompt.

Least privilege: Granting only the access needed to do a task.

PermitRootLogin: SSH setting; no blocks direct root logins.

REMEMBER

- sudo gives logged, policy-controlled elevation; edit policy with visudo.

- Avoid working or running services as root.
- Fix access with owners and groups, not 777.

QUICK CHECK

1. Why use visudo?
 - A) It is faster
 - B) It validates syntax and locks the file
 - C) It hides rules
 - D) It grants root
2. What does sudo -l show?
 - A) Log entries
 - B) Commands you may run
 - C) Logged in users
 - D) Locked accounts
3. Why is chmod 777 harmful?
 - A) Slows disk
 - B) Lets any user modify the file
 - C) Hides the file
 - D) Removes owner
4. Which rule is broadest?
 - A) bob ALL=(root) /usr/bin/systemctl restart nginx
 - B) alice ALL=(ALL) NOPASSWD: ALL
 - C) carol ALL=(www-data) /usr/bin/id
 - D) dave ALL=(root) /bin/lis /var/log

Answers: 1-B 2-B 3-B 4-B

1. A syntax error could break sudo entirely.
2. -l lists allowed commands.
3. All users get write access.
4. NOPASSWD: ALL gives unrestricted root without a password.

Decode the Permissions

SCENARIO

You review ten lines of `ls -l` output from a fictional file server fs01. For each file you must state who can read, write and execute, and flag unsafe ones. The files include notes, scripts, a key, a shared folder and a config.

OBJECTIVE

Read ten `ls -l` lines and state who can read, write and execute each file.

STEPS IN THE SIMULATOR

- 1 **Do:** Open line 1: `-rw-r--r-- 1 alice staff 2048 notes.txt`

Simulator shows:

Prompt: who can write? Choose owner, group, others.

- 2 **Do:** Answer: owner only

Simulator shows:

Correct. Group and others have read only.

- 3 **Do:** Open line 2: `-rwxrwxrwx 1 root root 312 backup.sh`

Simulator shows:

Prompt: risk level?

- 4 **Do:** Answer: high

Simulator shows:

Correct. All users can edit a root-owned script.

- 5 **Do:** Open line 3: `-rw-r----- 1 root shadow 1180 /etc/shadow`

Simulator shows:

Prompt: is this expected?

- 6 **Do:** Answer: yes

Simulator shows:

Correct. Only root and the shadow group can read.

- 7 **Do:** Open line 4: `-rw-rw-r-- 1 bob bob 1679 id_rsa`

Simulator shows:

Prompt: private key mode?

- 8 **Do:** Answer: too open

Simulator shows:

Correct. Should be 600.

ANSWER

Owner-only write on notes.txt is normal. backup.sh at 777 and an id_rsa at 664 are findings. shadow at 640 root:shadow is correct. The rest should be classified by reading owner, group and others triplets in order.

Hint shown in the app: Read the three triplets separately, left to right.

Fix the Broken Script

SCENARIO

A fictional script `run_report.sh` in `/home/student/tools` fails with Permission denied when its owner runs `./run_report.sh`. Other users must not be able to run or edit it, and group members should have no access.

OBJECTIVE

Apply `chmod` to make a script executable by its owner only and verify with `ls -l`.

STEPS IN THE SIMULATOR

- 1 **Do:** Type `ls -l run_report.sh`

Simulator shows:

```
-rw-r--r-- 1 student student 85 Mar 3 11:00 run_report.sh
```

- 2 **Do:** Type `./run_report.sh`

Simulator shows:

```
bash: ./run_report.sh: Permission denied
```

- 3 **Do:** Type `chmod 700 run_report.sh`

Simulator shows:

```
(no output)
```

- 4 **Do:** Type `ls -l run_report.sh`

Simulator shows:

```
-rwx----- 1 student student 85 Mar 3 11:00 run_report.sh
```

- 5 **Do:** Type `./run_report.sh`

Simulator shows:

```
Report generated: report_2024-03-03.txt
```

- 6 **Do:** Type `chmod 777 run_report.sh (trap)`

Simulator shows:

```
Simulator warns: world-writable executable, rejected by checker.
```

ANSWER

`chmod 700` (or `u+x` with `go-rwx`) leaves `rwx-----`: only the owner can read, write and execute. Verification with `ls -l` shows the mode and the script then runs.

Hint shown in the app: Octal 7 for the owner and 0 for the others.

Lock Down a Secrets File

SCENARIO

A fictional API key file `/etc/app/api.key` is owned by `root` and readable by everyone. The service account `svc-app` must read it, and nobody else should. You have `sudo` on host `app01`.

OBJECTIVE

Use `chown` and `chmod 600` so only the service account can read an API key file.

STEPS IN THE SIMULATOR

- Do:** Type `ls -l /etc/app/api.key`
Simulator shows:
`-rw-r--r-- 1 root root 41 Mar 3 11:20 /etc/app/api.key`
- Do:** Type `sudo chown svc-app:svc-app /etc/app/api.key`
Simulator shows:
`(no output)`
- Do:** Type `sudo chmod 600 /etc/app/api.key`
Simulator shows:
`(no output)`
- Do:** Type `ls -l /etc/app/api.key`
Simulator shows:
`-rw----- 1 svc-app svc-app 41 Mar 3 11:20 /etc/app/api.key`
- Do:** Type `sudo -u svc-app cat /etc/app/api.key`
Simulator shows:
`FAKE-API-KEY-0000`
- Do:** Type `cat /etc/app/api.key` as `student`
Simulator shows:
`cat: /etc/app/api.key: Permission denied`

ANSWER

Owner `svc-app`, group `svc-app`, mode `600`. The service account can read the key and any other user is denied, which meets least privilege.

Hint shown in the app: Change ownership first, then restrict the mode.

Audit sudoers and Accounts

SCENARIO

You are given excerpts of `/etc/passwd` and `/etc/sudoers.d/` files from a fictional server `srv01`. One account has UID 0 besides root, and one sudo rule gives a group unrestricted rights without a password. Find both and propose a fix.

OBJECTIVE

Review `/etc/passwd` and `sudoers` entries to find an extra UID 0 user and an unrestricted sudo rule.

STEPS IN THE SIMULATOR

1 Do: Open `/etc/passwd`

Simulator shows:

```
root:x:0:0:root:/root:/bin/bash
support:x:0:0::/home/support:/bin/bash
alice:x:1001:1001:/home/alice:/bin/bash
www-data:x:33:33:/var/www:/usr/sbin/nologin
```

2 Do: Select the suspicious line

Simulator shows:

Highlight support: UID 0 and a login shell.

3 Do: Open `/etc/sudoers.d/contractors`

Simulator shows:

```
%contractors ALL=(ALL) NOPASSWD: ALL
```

4 Do: Mark the risky rule

Simulator shows:

Rule flagged: unrestricted, no password.

5 Do: Edit the rule to: `%contractors ALL=(root) /usr/bin/systemctl restart nginx`

Simulator shows:

Checker: syntax valid, privilege reduced.

6 Do: Change support UID to 1002 or remove account

Simulator shows:

Checker: no extra UID 0 accounts remain.

ANSWER

The account `support` has UID 0 and is a hidden root-equivalent user; the rule `%contractors ALL=(ALL) NOPASSWD: ALL` gives unrestricted root. Remove or renumber `support` after investigation, and limit the sudo rule to specific commands.

Hint shown in the app: Check the third `passwd` field for 0.

Linux III: Users and Permissions: 10 questions, 80% to pass

1. What does UID 0 represent?
 - A) A guest
 - B) The superuser root
 - C) A service
 - D) A group
2. How many characters make up the permission section of `ls -l` (excluding type)?
 - A) 3
 - B) 6
 - C) 9
 - D) 12
3. What octal is `rw-r-----`?
 - A) 640
 - B) 604
 - C) 750
 - D) 600
4. What does `chmod u+x` do?
 - A) Adds execute for the owner
 - B) Removes execute for all
 - C) Sets 777
 - D) Changes owner
5. Which bit makes a program run as its owner?
 - A) Sticky
 - B) Setuid
 - C) Setgid only
 - D) noexec
6. Why does `/tmp` use the sticky bit?
 - A) Prevent users deleting others' files
 - B) Make it executable
 - C) Hide files
 - D) Enable setgid
7. Safest tool for editing sudo policy?
 - A) `nano /etc/sudoers`
 - B) `visudo`
 - C) `echo >>`
 - D) `chmod`
8. Why is 777 a vulnerability?
 - A) Too slow
 - B) Any user can modify the file
 - C) Needs a reboot
 - D) It removes the owner
9. Command to find setuid files?
 - A) `find / -perm -4000`
 - B) `ls -u`
 - C) `chmod -4000`
 - D) `id -s`
10. A service needs to read one key file. Best approach?
 - A) `chmod 777`
 - B) Own it by the service account with mode 600
 - C) Run service as root
 - D) Put it in `/tmp`

Answers: 1-B 2-C 3-A 4-A 5-B 6-A 7-B 8-B 9-A 10-B

1. UID 0 has unrestricted power.
2. Three triplets of three.
3. 6,4,0.
4. u is the owner, + adds.
5. Setuid 4000.
6. Sticky restricts deletion.
7. visudo validates syntax.
8. Everyone gets write access.
9. -perm -4000 matches setuid.
10. Narrow ownership and mode follow least privilege.

Linux IV: Pipes, Processes and Network

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

This module turns the Linux shell into an analysis and administration toolkit. You will search text with grep, chain commands with pipes and redirects, inspect and stop processes, keep software patched with apt, and check network state with ip, ping and ss. Together these let you rank attackers in a log, find a runaway or suspicious process, close known vulnerabilities and see which services are exposed. They are the everyday commands of system administrators and security analysts.

BY THE END YOU CAN

- + Search and transform text with grep, pipes and redirects
- + Inspect and stop processes with ps, top and kill
- + Check updates and network state with apt, ip, ping and ss

IN THIS MODULE

Lesson 1 grep and Regular Patterns

Lesson 2 Pipes and Redirects

Lesson 3 Processes: ps, top and kill

Lesson 4 Package Management with apt

Lesson 5 Network Checks: ip, ping and ss

Lab 1 Failed Logins by IP (Terminal)

Lab 2 Runaway Process (Terminal)

Lab 3 Patch the Box (Terminal)

Lab 4 Who Is Listening? (Terminal)

Checkpoint 10 questions, 80% to pass

grep and Regular Patterns

Logs are huge, and grep is the fastest way to pull out the few lines that matter.

Basic grep

The grep command prints lines that contain a pattern: `grep 'Failed password' /var/log/auth.log` shows every failed SSH login line. Quote patterns so the shell leaves them alone. By default matching is case sensitive and finds the pattern anywhere in a line. grep reads one or more files, or standard input, which is what allows it to sit in the middle of a pipeline. Its exit status is 0 when something matched and 1 when nothing did, which scripts use for decisions. Reading logs requires permission, so on many systems you need `sudo` for files under `/var/log`.

Useful options

Four options cover most daily use. `-i` ignores case, so `grep -i error` matches `Error` and `ERROR`. `-n` adds line numbers, helping you jump to the location in an editor. `-r` (or `-R`) searches recursively through a directory, as in `grep -rn 'password' /etc/app`, handy for hunting hardcoded secrets. `-v` inverts the match and prints lines that do not contain the pattern, so `grep -v CRON` removes noisy cron entries. Also useful are `-c` to count matching lines, `-l` to list only file names, `-w` for whole words and `-A` or `-B` to show lines after or before a match.

Simple patterns

Plain text is the simplest pattern, but grep also supports regular expressions. The caret `^` anchors to the start of a line and `$` to the end, so `grep '^Mar 3'` selects lines starting with that date. A dot matches any one character, and `.*` matches any run. Brackets match one character from a set, like `[0-9]`. With `-E` you get extended expressions: `grep -E 'Failed|Invalid'` matches either word, and `grep -E '[0-9]{1,3}(\.[0-9]{1,3}){3}'` matches the shape of an IPv4 address. Remember that regex symbols differ from shell wildcards, so always quote.

Filtering logs

Combine options to answer questions. `grep -i 'invalid user' /var/log/auth.log` finds attempts against nonexistent accounts. `grep -c 'Failed password'` counts failures. `grep 'Accepted'` shows successful logins, which you compare with expected users and times. To exclude your own address from results, use `grep -v 192.0.2.10`. A caution: grep matches text, not meaning, so a pattern such as `'fail'` can also match a username containing those letters. Check a sample of matches before drawing conclusions, and widen with `-A 2` to see context around an event.

TERMINAL OUTPUT

```
$ grep -n 'Failed password' /var/log/auth.log | head -n 2
112:Mar 3 09:20:37 web01 sshd[2210]: Failed password for root from 203.0.113.9 port 51520 ssh2
113:Mar 3 09:20:39 web01 sshd[2210]: Failed password for invalid user admin from 203.0.113.9 port 51522
ssh2
$ grep -c 'Accepted' /var/log/auth.log
3
$ grep -rn 'password=' /etc/app
```

WORKED EXAMPLE

On web01 an analyst wants to know whether anyone logged in successfully after the failed attempts. She runs `grep 'Accepted' /var/log/auth.log` and sees three lines, one from 198.51.100.7 at 09:25 using the user `deploy`. She then runs `grep -n '198.51.100.7'` to see everything that address did, and uses `-A 3` to read context. grep reduced thousands of lines to a handful that told the story.

KEY TERMS

- grep:** Prints lines matching a pattern from files or standard input.
- v:** Inverts the match, printing lines without the pattern.
- r:** Searches directories recursively.
- Anchor:** Regex symbols `^` and `$` that match line start and end.
- E:** Enables extended regular expressions such as alternation with `|`.

REMEMBER

- grep filters lines by pattern; quote your patterns.

- i, -n, -r, -v and -c cover most use.
- Verify matches and use -A and -B for context.

QUICK CHECK

1. Which grep option ignores case?

- A) -i
- B) -v
- C) -n
- D) -r

2. What does grep -v CRON do?

- A) Counts CRON
- B) Prints lines without CRON
- C) Searches recursively
- D) Colors CRON

3. Which option adds line numbers?

- A) -n
- B) -l
- C) -c
- D) -w

4. What does ^Mar match?

- A) Mar anywhere
- B) Lines starting with Mar
- C) Lines ending with Mar
- D) Mar in file names

Answers: 1-A 2-B 3-A 4-B

- 1. -i is case insensitive.
- 2. -v inverts the match.
- 3. -n shows line numbers.
- 4. The caret anchors to line start.

Pipes and Redirects

Chaining small tools lets you answer questions like which IP failed most, without writing a program.

Standard streams

Every Linux command has three streams: standard input (stream 0), standard output (stream 1) and standard error (stream 2). Normally input comes from the keyboard and both outputs go to the terminal. Redirection and pipes change where they go. Understanding that errors are a separate stream explains why a command can print Permission denied even when you redirected output to a file. The special file /dev/null discards anything written to it, so 2>/dev/null hides error messages. This design is why the same small tools can be recombined in many ways.

Redirects

The operator > sends standard output to a file, creating it or overwriting it. The operator >> appends instead, which is how you keep adding to a log or notes file. The operator 2> redirects standard error, and 2>&1 sends errors to the same place as output. The operator < feeds a file into standard input. For example, grep 'Failed' auth.log > failed.txt saves matches, and find / -name '*.key' 2> errors.txt keeps the noise separate. Be careful: > on an existing file destroys its previous content immediately, even before the command finishes.

Pipes

A pipe, written |, connects the output of the left command to the input of the right command with no temporary file. Pipelines may be long: cat file | grep x | sort | uniq. Each stage does one job. Useful partners are sort to order lines, uniq -c to collapse adjacent duplicates and count them (so data must be sorted first), wc -l to count lines, cut -d' ' -f1 to pick a field, and head to keep the top rows. Because uniq only merges adjacent lines, the common pattern is sort | uniq -c | sort -nr to rank by count.

A ranking example

To find the noisiest attacker in an SSH log, extract the failed lines, isolate the address, then count: grep 'Failed password' auth.log | awk '{print \$(NF-3)}' | sort | uniq -c | sort -nr | head. The awk step picks the IP field, which sits four from the end of the line in the standard format; check that on your own sample, because formats vary. The first sort groups identical addresses, uniq -c counts them, the second sort -nr orders numerically from high to low, and head shows the top results. Build such pipelines one stage at a time, inspecting output as you go.

TERMINAL OUTPUT

```
$ grep 'Failed password' auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head -n 3
  212 203.0.113.9
   14 198.51.100.7
    3 192.0.2.44
$ wc -l auth.log
1840 auth.log
```

WORKED EXAMPLE

A small web host sees slow SSH. The administrator runs the ranking pipeline on auth.log and gets 212 failed attempts from 203.0.113.9, 14 from 198.51.100.7 and 3 from 192.0.2.44. She saves the result with > top_attackers.txt, then appends a timestamp with date >> top_attackers.txt. The top address is a candidate to block at the firewall, while the three-attempt address is probably a user mistyping a password.

KEY TERMS

Pipe: Connects one command's output to the next command's input.

Redirect >: Writes output to a file, overwriting it.

Append >>: Adds output to the end of a file.

stderr: Standard error, stream 2, used for error messages.

uniq -c: Counts adjacent duplicate lines.

REMEMBER

- Pipes pass output to input; redirects send streams to files.

- > overwrites, >> appends, 2> handles errors.
- sort | uniq -c | sort -nr ranks frequency.

QUICK CHECK

1. Which operator appends to a file?
 - A) >
 - B) >>
 - C) 2>
 - D) |
2. Why sort before uniq -c?
 - A) For color
 - B) uniq only collapses adjacent duplicates
 - C) sort is required by the shell
 - D) to hide errors
3. What does 2>/dev/null do?
 - A) Saves errors
 - B) Discards error messages
 - C) Appends output
 - D) Reads input
4. What does wc -l count?
 - A) Words
 - B) Lines
 - C) Bytes
 - D) Files

Answers: 1-B 2-B 3-B 4-B

1. >> appends.
2. uniq counts only neighboring identical lines.
3. Stream 2 is sent to the null device.
4. -l is lines.

Processes: ps, top and kill

Malware, cryptominers and runaway jobs all show up as processes, and you must know how to see and stop them safely.

What a process is

A process is a running program with a process ID (PID), a parent PID (PPID), an owner, a state and resource use. The first process, PID 1 (systemd on most modern systems), starts all others. Processes can be in states such as running (R), sleeping (S) or zombie (Z), a finished process waiting for its parent to collect it. Each runs as a user, and its privileges equal that user's, so a process running as root deserves more scrutiny. Seeing which processes exist, who owns them and where they came from is a basic detection skill.

Listing with ps

The `ps` command prints a snapshot. `ps aux` shows all processes with columns USER, PID, %CPU, %MEM, START, TIME and COMMAND. `ps -ef` shows the parent PID too, which helps trace what launched what. Combine with `grep` to find one program: `ps aux | grep nginx`. Be aware that the `grep` command itself appears in the results; the pattern `[n]ginx` avoids that. `pgrep -a` name prints PIDs and command lines directly. Suspicious signs include unfamiliar names, processes running from `/tmp` or a hidden directory, odd parents, and a name that mimics a system process with a small spelling change.

Live view with top

The `top` command refreshes continuously, showing load average, CPU and memory totals, and processes sorted by CPU by default. Inside `top`, press `M` to sort by memory, `P` to sort by CPU, `k` to kill a PID and `q` to quit. `htop` is a friendlier alternative if installed. A single process at 95 percent CPU for a long time can mean an infinite loop, a backup or a cryptominer. Check its owner, its command line and its start time, then use `ls -l /proc/PID/exe` to see which binary on disk it truly is.

Signals and kill

The `kill` command sends a signal to a process; despite its name it does not always kill. `kill PID` sends `SIGTERM` (15), a polite request that lets the program clean up. If it ignores the request, `kill -9 PID` sends `SIGKILL`, which the kernel enforces immediately with no cleanup, risking corrupted data. `kill -1` (`SIGHUP`) often asks a daemon to reload its configuration, and `Ctrl+C` sends `SIGINT`. `killall` name and `pkill` name match by name, so use them carefully. You can only signal your own processes unless using `sudo`. Try `SIGTERM` first, escalate to `SIGKILL` only if needed, and preserve evidence before killing a suspected malicious process.

TERMINAL OUTPUT

```
$ ps aux --sort=-%cpu | head -n 3
USER PID %CPU %MEM START TIME COMMAND
svc 4021 95.2 1.1 02:14 410:12 /tmp/.cache/miner --pool 198.51.100.50
root 1 0.0 0.1 Mar01 0:09 /sbin/init
$ kill 4021
$ kill -9 4021
```

WORKED EXAMPLE

Monitoring on host `app01` alerts on sustained CPU. The administrator runs `top`, sees PID 4021 at 95 percent, and `ps` shows it runs from `/tmp/.cache` under the `svc` account. She first records the command line and the binary hash for evidence, then sends `kill 4021`. The process survives, so she escalates to `kill -9 4021`. She then investigates how it arrived, since killing it does not remove the root cause.

KEY TERMS

PID: Numeric identifier of a process.

SIGTERM: Signal 15, a graceful termination request; the default for `kill`.

SIGKILL: Signal 9, forced termination that cannot be caught.

ps aux: Snapshot listing of all processes with user and resource data.

Zombie: A finished process whose exit status has not yet been collected.

REMEMBER

- ps gives snapshots; top gives a live view.
- SIGTERM first, SIGKILL last.
- Investigate location, owner and parent of odd processes.

QUICK CHECK

1. What signal does plain kill send?
 - A) SIGKILL
 - B) SIGTERM
 - C) SIGHUP
 - D) SIGSTOP
2. Which command shows live CPU use?
 - A) ps
 - B) top
 - C) ls
 - D) find
3. Why try SIGTERM before SIGKILL?
 - A) It is faster
 - B) It lets the program clean up
 - C) It needs no permission
 - D) It hides evidence
4. Which clue suggests malware?
 - A) Process running from /tmp with high CPU
 - B) systemd as PID 1
 - C) sshd run by root
 - D) cron

Answers: 1-B 2-B 3-B 4-A

1. Default is 15, SIGTERM.
2. top refreshes continuously.
3. Graceful stop avoids corruption.
4. Unusual location and heavy CPU are red flags.

Package Management with apt

Unpatched software is a leading cause of breaches, and apt is how Debian-family systems stay current.

How apt works

On Debian, Ubuntu and Kali, apt installs software from repositories listed in `/etc/apt/sources.list` and files in `sources.list.d`. Packages are signed, and apt verifies the signatures with trusted keys, which protects against tampered downloads. Package management also resolves dependencies, installing the libraries a program needs. The underlying tool is `dpkg`, which handles individual `.deb` files, while apt handles repositories and dependencies. Most commands need root, so they are prefixed with `sudo`. Red Hat-family systems use `dnf` or `yum`, and the ideas are the same.

The daily commands

The sequence is: `sudo apt update` refreshes the package index from repositories (it does not install anything), `apt list --upgradable` shows which installed packages have newer versions, and `sudo apt upgrade` installs those updates. `sudo apt full-upgrade` also allows removing or adding packages to satisfy changes. `apt search keyword` and `apt show package` find and describe software. `sudo apt install nginx` installs, `sudo apt remove nginx` removes it but keeps configuration, and `sudo apt purge` removes the configuration too. `sudo apt autoremove` clears orphaned dependencies. Always run update before install, or you may fetch outdated versions.

Why patching reduces risk

Attackers scan the internet for software with known vulnerabilities, identified by CVE numbers, and exploit them within days of publication. A vendor fix closes the hole; until you apply it, you are exposed. Security updates are often published in a dedicated security repository, and the Ubuntu `unattended-upgrades` package can install them automatically. Prioritize internet-facing services and anything with a critical or actively exploited CVE. Patching also applies to the kernel and libraries; services using an updated library may need a restart, and a kernel update needs a reboot to take effect.

Safe habits

Install only what you need, because every package is attack surface and future patch work. Use official repositories, and be cautious with third-party repositories and with curl piped to a shell, which bypass signature checks and review. Test updates on a staging system where possible, and keep a rollback plan or snapshot for critical servers. Read the apt output: held-back packages and errors matter. Check history in `/var/log/apt/history.log` to see what was installed and when, which is useful during incident response. Remove software you no longer use.

TERMINAL OUTPUT

```
$ sudo apt update
Get:1 http://security.example.com/debian-security bookworm-security InRelease
Reading package lists... Done
12 packages can be upgraded.
$ apt list --upgradable
openssl/stable-security 3.0.15-1 amd64 [upgradable from: 3.0.11-1]
$ sudo apt install --only-upgrade openssl
```

WORKED EXAMPLE

The security team learns of a critical flaw in a library on `srv01`. The administrator runs `sudo apt update`, then `apt list --upgradable` and sees the library has a fixed version in the security repository. She installs it with `sudo apt install --only-upgrade`, restarts the dependent service, and confirms the new version with `apt show`. The window of exposure ends as soon as the fix is installed, so speed matters.

KEY TERMS

apt update: Refreshes the package index; installs nothing.

apt upgrade: Installs available updates for installed packages.

Repository: A signed source of packages that apt downloads from.

CVE: A public identifier for a known vulnerability.

apt purge: Removes a package and its configuration files.

REMEMBER

- update refreshes, upgrade installs, install adds packages.
- Patching closes known CVEs and shrinks the exposure window.
- Install less software, use trusted repositories.

QUICK CHECK

1. What does apt update do?
 - A) Installs updates
 - B) Refreshes the package list
 - C) Removes packages
 - D) Reboots
2. Which command installs available updates?
 - A) apt upgrade
 - B) apt search
 - C) apt show
 - D) apt list
3. Why patch quickly?
 - A) Attackers exploit known flaws soon after disclosure
 - B) Patches slow the system
 - C) It resets passwords
 - D) It is required by sudo
4. Why verify package signatures?
 - A) Speed
 - B) To detect tampered packages
 - C) To save space
 - D) To hide history

Answers: 1-B 2-A 3-A 4-B

1. It only refreshes the index.
2. apt upgrade applies them.
3. Known CVEs are exploited quickly.
4. Signatures prove packages are from the repository.

Network Checks: ip, ping and ss

Before you can defend a host you must know its addresses, its route out and what is listening for connections.

Addresses with ip a

The command `ip a` (short for `ip address show`) lists each network interface, its state and its addresses. A line like `inet 192.0.2.15/24 brd 192.0.2.255 scope global eth0` means the interface `eth0` has IPv4 address `192.0.2.15` with a 24-bit prefix, so the network is `192.0.2.0/24`. `lo` is the loopback interface with `127.0.0.1`. `inet6` lines show IPv6 addresses. The `link/ether` line shows the MAC address. Look for state `UP`, and for unexpected extra interfaces, such as an unknown tunnel or VPN, which can indicate a pivot or misconfiguration.

Routes with ip route

The command `ip route` shows the routing table. The line `default via 192.0.2.1 dev eth0` means traffic to unknown destinations goes to the gateway at `192.0.2.1` through `eth0`, and lines like `192.0.2.0/24 dev eth0` mean that network is directly connected. If the default route is missing, the host cannot reach the internet. Changes to routes that you did not make are suspicious, as an attacker may redirect traffic. The older `ifconfig` and `netstat` tools still appear in tutorials, but `ip` and `ss` are their modern replacements.

Reachability with ping

The `ping` command sends ICMP echo requests and reports replies and round-trip times. Use `ping -c 4 192.0.2.1` to send four packets and stop; without `-c`, Linux pings forever until `Ctrl+C`. Test in layers: `ping 127.0.0.1` checks the local stack, then the gateway, then an outside address such as `203.0.113.10`, then a name like `example.com`, which also tests DNS. A reply proves reachability, but silence does not prove a host is down, since firewalls often drop ICMP. Output also gives packet loss percentage and TTL, a hint about the operating system and hop count.

Listening ports with ss

The command `ss -tulpn` lists sockets: `t` for TCP, `u` for UDP, `l` for listening, `p` to show the process and `n` for numeric ports. Run it with `sudo` to see process names for all users. In the Local Address column, `127.0.0.1:5432` means only the local host can connect, while `0.0.0.0:22` or `:::22` means every interface. Every listening service is part of your attack surface, so compare the list with what should be running. An unexpected service on `0.0.0.0`, such as a database on `3306`, is a finding. The PID in `users:("name",pid=N,fd=M)` lets you inspect the process with `ps`.

TERMINAL OUTPUT

```
$ ip -4 a show eth0
2: eth0: <BROADCAST,MULTICAST,UP> state UP
    inet 192.0.2.15/24 brd 192.0.2.255 scope global eth0
$ ip route
default via 192.0.2.1 dev eth0
$ ping -c 2 192.0.2.1
2 packets transmitted, 2 received, 0% packet loss
$ sudo ss -tulpn
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=811,fd=3))
tcp LISTEN 0 80 0.0.0.0:3306 0.0.0.0:* users:(("mysqld",pid=1450,fd=21))
```

WORKED EXAMPLE

A new web server was supposed to expose only ports 22 and 443. The administrator runs `sudo ss -tulpn` and sees `mysqld` listening on `0.0.0.0:3306`, reachable from every network. She notes the PID 1450, confirms with `ps`, and changes the database bind-address to `127.0.0.1` and restarts it. A second `ss` run shows `127.0.0.1:3306`, so the database is now reachable only from the host itself.

KEY TERMS

ip a: Shows interfaces, addresses and link state.

ip route: Shows the routing table including the default gateway.

ICMP echo: The request type used by `ping` to test reachability.

ss -tulpn: Lists listening TCP and UDP sockets with owning processes.

0.0.0.0: A listening address meaning all IPv4 interfaces.

REMEMBER

- ip a and ip route show addresses and paths.
- ping tests reachability but silence is not proof of down.
- ss -tulpn reveals exposed services and their PIDs.

QUICK CHECK

1. Which command shows the default gateway?

- A) ip a
- B) ip route
- C) ping
- D) ss

2. What does ss -tulpn show?

- A) Disk usage
- B) Listening ports and their processes
- C) DNS records
- D) Routing only

3. What does a listener on 127.0.0.1 mean?

- A) Open to the internet
- B) Local connections only
- C) Broken
- D) IPv6 only

4. Why might ping get no reply from a live host?

- A) Firewall blocks ICMP
- B) Host has no IP
- C) ping is slow
- D) TTL is high

Answers: 1-B 2-B 3-B 4-A

1. ip route lists the default route.
2. It lists sockets with PIDs.
3. Loopback is reachable only from the same host.
4. Many firewalls drop ICMP echo.

Failed Logins by IP

SCENARIO

A fictional web server web01 has an auth.log with many SSH failures. Management wants to know which source addresses are the worst offenders so they can be blocked. Use a pipeline, not manual counting.

OBJECTIVE

Pipe grep, sort and uniq -c over an auth log to rank source IPs by failed attempts.

STEPS IN THE SIMULATOR

- 1 **Do:** Type `grep 'Failed password' auth.log | head -n 2`

Simulator shows:

```
Mar 3 09:20:37 web01 sshd[2210]: Failed password for root from 203.0.113.9 port 51520 ssh2
Mar 3 09:20:39 web01 sshd[2210]: Failed password for invalid user admin from 203.0.113.9 port 51522 ssh2
```

- 2 **Do:** Add `awk: ... | awk '{print $(NF-3)}'`

Simulator shows:

```
203.0.113.9
203.0.113.9
198.51.100.7
...
```

- 3 **Do:** Add `sort`

Simulator shows:

Identical addresses are grouped together.

- 4 **Do:** Add `uniq -c`

Simulator shows:

```
212 203.0.113.9
14 198.51.100.7
3 192.0.2.44
```

- 5 **Do:** Add `sort -nr | head -n 3`

Simulator shows:

```
212 203.0.113.9
14 198.51.100.7
3 192.0.2.44
```

- 6 **Do:** Save with `> top_ips.txt`

Simulator shows:

(no output; file written)

ANSWER

The top offender is 203.0.113.9 with 212 failed attempts, followed by 198.51.100.7 with 14 and 192.0.2.44 with 3. The full pipeline is `grep, awk` to extract the IP, `sort, uniq -c` and `sort -nr`.

Hint shown in the app: `uniq -c` needs sorted input.

Runaway Process

SCENARIO

A fictional host app01 is sluggish. Monitoring shows one process using almost all CPU. You must identify it and stop it with the least forceful signal that works, and record what you did.

OBJECTIVE

Use `top` and `ps` to find a process using 95 percent CPU and terminate it with the right signal.

STEPS IN THE SIMULATOR

1 Do: Type `top`

Simulator shows:

```
PID 4021 svc 95.2 %CPU /tmp/.cache/miner
```

2 Do: Press `q`, then type `ps -fp 4021`

Simulator shows:

```
svc 4021 1 95 02:14 ? 07:12:03 /tmp/.cache/miner --pool 198.51.100.50
```

3 Do: Type `ls -l /proc/4021/exe`

Simulator shows:

```
/proc/4021/exe -> /tmp/.cache/miner
```

4 Do: Type `kill 4021`

Simulator shows:

(no output)

5 Do: Type `ps -p 4021`

Simulator shows:

```
4021 ... still running
```

6 Do: Type `kill -9 4021`

Simulator shows:

(no output)

7 Do: Type `ps -p 4021`

Simulator shows:

```
PID not found: process ended
```

ANSWER

PID 4021, running from `/tmp/.cache` as account `svc`, used 95 percent CPU. `SIGTERM` (`kill 4021`) was ignored, so `SIGKILL` (`kill -9 4021`) ended it. The suspicious path suggests a cryptominer and a compromise to investigate.

Hint shown in the app: Try plain kill first, then -9.

Patch the Box

SCENARIO

A fictional server `srv01` runs an outdated `openssl` package and a security advisory says it is fixed in a newer version. You have `sudo`. Refresh, review and install just the needed update, then verify it.

OBJECTIVE

Run `apt update`, list upgradable packages and install a security update for a vulnerable package.

STEPS IN THE SIMULATOR

- 1 **Do:** Type `sudo apt update`

Simulator shows:

Hit/Get lines... 12 packages can be upgraded.

- 2 **Do:** Type `apt list --upgradable`

Simulator shows:

`openssl/stable-security 3.0.15-1 [upgradable from: 3.0.11-1]`
(+11 more)

- 3 **Do:** Type `apt show openssl | head -n 5`

Simulator shows:

Package: `openssl`
Version: `3.0.15-1`
Priority: `optional`

- 4 **Do:** Type `sudo apt install --only-upgrade openssl`

Simulator shows:

Setting up `openssl (3.0.15-1) ...`

- 5 **Do:** Type `apt list --installed | grep openssl`

Simulator shows:

`openssl/stable-security,now 3.0.15-1 amd64 [installed]`

- 6 **Do:** Type `sudo systemctl restart nginx`

Simulator shows:

(no output; service uses new library)

ANSWER

`openssl` was upgraded from `3.0.11-1` to `3.0.15-1` from the security repository after `apt update` refreshed the index. Verification shows the new installed version, and the dependent service was restarted.

Hint shown in the app: Run `apt update` before listing upgradable packages.

Who Is Listening?

SCENARIO

A fictional web server web02 should expose only SSH on 22 and HTTPS on 443. A scan shows an extra open port. Find the service, identify the process and decide what to change.

OBJECTIVE

Run `ss -tulpn` and flag the unexpected service listening on 0.0.0.0, then trace its PID.

STEPS IN THE SIMULATOR

- 1 **Do:** Type `sudo ss -tulpn`

Simulator shows:

```
tcp LISTEN 0.0.0.0:22 sshd pid=811
tcp LISTEN 0.0.0.0:443 nginx pid=902
tcp LISTEN 0.0.0.0:3306 mysqld pid=1450
tcp LISTEN 127.0.0.1:631 cupsd pid=700
```

- 2 **Do:** Flag 0.0.0.0:3306

Simulator shows:

Marked unexpected: database exposed on all interfaces.

- 3 **Do:** Type `ps -fp 1450`

Simulator shows:

```
mysql 1450 1 0 Mar01 ? mysqld
```

- 4 **Do:** Type `ls -l /proc/1450/exe`

Simulator shows:

```
/proc/1450/exe -> /usr/sbin/mysqld
```

- 5 **Do:** Set bind-address = 127.0.0.1 and restart

Simulator shows:

Config saved; service restarted

- 6 **Do:** Type `sudo ss -tulpn | grep 3306`

Simulator shows:

```
tcp LISTEN 127.0.0.1:3306 mysqld pid=1451
```

ANSWER

mysqld (PID 1450) listened on 0.0.0.0:3306, exposing the database. Binding it to 127.0.0.1 limits access to the host. The CUPS listener on 127.0.0.1:631 is local-only and not an exposure.

Hint shown in the app: Look at the Local Address column for 0.0.0.0.

Linux IV: Pipes, Processes and Network: 10 questions, 80% to pass

1. Which grep option prints lines that do NOT match?
 - A) -i
 - B) -v
 - C) -r
 - D) -n
2. What does >> do?
 - A) Overwrites a file
 - B) Appends to a file
 - C) Pipes output
 - D) Discards errors
3. Why use sort before uniq -c?
 - A) uniq only merges adjacent duplicates
 - B) To save disk
 - C) uniq needs root
 - D) To color output
4. What does 2> redirect?
 - A) Standard output
 - B) Standard error
 - C) Standard input
 - D) Environment
5. Which signal is the default for kill?
 - A) SIGKILL
 - B) SIGTERM
 - C) SIGSTOP
 - D) SIGUSR1
6. Which gives a live, continuously updating process view?
 - A) ps aux
 - B) top
 - C) ls /proc
 - D) kill
7. What does apt update do?
 - A) Upgrades all software
 - B) Refreshes the package index
 - C) Removes old kernels
 - D) Reboots
8. Why does timely patching reduce risk?
 - A) It closes known vulnerabilities
 - B) It deletes logs
 - C) It disables firewalls
 - D) It adds users
9. In ss output, 0.0.0.0:3306 means:
 - A) MySQL reachable on all interfaces
 - B) Local only
 - C) Closed port
 - D) IPv6 only
10. Which command shows the default route?
 - A) ip route
 - B) ip a
 - C) ping
 - D) ss -t

Answers: 1-B 2-B 3-A 4-B 5-B 6-B 7-B 8-A 9-A 10-A

1. -v inverts.
2. It appends.
3. Duplicates must be adjacent.
4. Stream 2 is stderr.
5. Default is 15 (SIGTERM).
6. top refreshes.
7. It fetches package lists only.
8. Fixes remove known exploitable flaws.
9. 0.0.0.0 is every interface.
10. ip route lists the routing table.

Windows Fundamentals

Beginner | 6 lessons | 4 simulated labs | checkpoint | about 4h

Most of the world's workstations and many servers run Windows, so every defender must be at home in it. This module covers the file system and NTFS permissions, local users, groups and UAC, the registry and services, the inspection tools in Task Manager, cmd and PowerShell, an introduction to Active Directory and Kerberos, and the built-in protections of Defender and the Windows firewall. You will also learn the places attackers hide persistence.

BY THE END YOU CAN

- + Navigate the Windows file system, registry and services
- + Use Task Manager, cmd and PowerShell for basic inspection
- + Explain UAC, Active Directory basics, Defender and the firewall

IN THIS MODULE

Lesson 1 Windows File System and NTFS

Lesson 2 Users, Groups and UAC

Lesson 3 The Registry and Services

Lesson 4 Task Manager, cmd and PowerShell

Lesson 5 Active Directory Introduction

Lesson 6 Windows Defender and Firewall

Lab 1 Find the Autorun Entry (Config editor)

Lab 2 PowerShell Inspection (Terminal)

Lab 3 Firewall Rule Fix (Config editor)

Lab 4 Active Directory Tree (Diagram builder)

Checkpoint 10 questions, 80% to pass

Windows File System and NTFS

Knowing where Windows keeps programs, user data and system files tells you what is normal and what is out of place.

Key folders

Windows is usually installed on the C: drive. C:\Windows holds the operating system, and C:\Windows\System32 contains core executables and libraries such as cmd.exe and the DLLs that most programs load. C:\Users contains one profile folder per user, with Desktop, Documents, Downloads and the hidden AppData. C:\Program Files holds 64-bit applications and C:\Program Files (x86) holds 32-bit ones. C:\ProgramData holds shared application data. Paths use backslashes and are not case sensitive. Programs in System32 or Program Files are writable only by administrators, whereas user-writable places such as Downloads and AppData\Local\Temp are where unwanted programs often run from.

NTFS and permissions

NTFS is the standard Windows file system. It supports journaling, large volumes, compression, encryption (EFS) and fine-grained access control lists (ACLs). Each file or folder has a discretionary ACL made of access control entries (ACEs) that give a user or group rights such as Read, Write, Modify or Full control. Permissions are inherited from parent folders unless inheritance is broken. Explicit Deny entries override Allow. When a user is in several groups, rights add up, and the effective result is the combination. View and change them with the Security tab, or icacls from the command line.

Share versus NTFS permissions

When a folder is shared over the network, two layers apply: share permissions and NTFS permissions. The most restrictive combined result wins. A common practice is to set the share to broad access for authenticated users and manage real control with NTFS ACLs, so there is one place to audit. Administrative shares such as C\$ and ADMIN\$ exist by default and are used by management tools, but also by attackers moving laterally with stolen credentials. Auditing who has access to sensitive folders, and removing Everyone and overly broad groups, is a core hardening task.

Alternate data streams

NTFS files can have extra named streams besides the main content, written file.txt:streamname. The feature was created for compatibility and metadata, such as the Zone.Identifier stream that records a file came from the internet and drives the Mark of the Web warning. But a stream is not shown by a normal dir listing and does not change the visible file size, so attackers have hidden data or scripts in them. dir /r lists streams, and PowerShell Get-Item -Stream * does too. Copying to FAT file systems or some archives drops streams, which is also a reason they matter in forensics.

COMMAND OUTPUT

```
C:\> dir /r C:\Users\alice\Downloads
12/01/2024 09:10 1,024 report.txt
                24 report.txt:hidden.txt:$DATA
C:\> icacls C:\Data\Finance
C:\Data\Finance BUILTIN\Administrators:(OI)(CI)(F)
                CORP\Finance:(OI)(CI)(M)
                Everyone:(OI)(CI)(R)
```

WORKED EXAMPLE

An auditor runs icacls on a fictional finance share and sees Everyone with read access inherited from its parent. That means every user on the network can read payroll files. She removes the Everyone entry, grants read to the Finance group, and checks that inheritance flows correctly. She also runs dir /r on the folder to confirm no alternate streams hide additional content next to the files.

KEY TERMS

NTFS: The default Windows file system with ACLs, journaling and encryption support.

ACL: A list of entries that grant or deny rights to users and groups on an object.

System32: Folder holding core Windows executables and libraries.

Alternate data stream: A named hidden data stream attached to an NTFS file.

icacls: Command line tool to view and edit NTFS ACLs.

REMEMBER

- Know the key folders; user-writable paths are suspicious for executables.
- NTFS ACLs, inheritance and Deny rules control access.
- Alternate data streams can hide content from normal listings.

QUICK CHECK

1. Where do 64-bit apps normally install?
 - A) C:\Windows
 - B) C:\Program Files
 - C) C:\Users
 - D) C:\Temp
2. Which wins when permissions conflict?
 - A) Allow
 - B) Deny
 - C) Whichever is newer
 - D) Inherited
3. Which command lists alternate data streams?
 - A) dir /r
 - B) dir /a
 - C) ls -a
 - D) type
4. When share and NTFS permissions both apply, the result is:
 - A) The most permissive
 - B) The most restrictive
 - C) The share only
 - D) The NTFS only

Answers: 1-B 2-B 3-A 4-B

1. 64-bit applications go in Program Files.
2. Explicit Deny overrides Allow.
3. dir /r displays streams.
4. The tighter combination is effective.

Users, Groups and UAC

Privilege on Windows is built from accounts, groups and tokens, and UAC is a key barrier against silent elevation.

Local users and groups

A Windows computer has local accounts managed with `lusrmgr.msc`, `net user` or PowerShell. Built-in accounts include Administrator (often disabled), Guest and system accounts such as SYSTEM, which is more powerful than any administrator on the local machine. Groups bundle rights: Administrators have full control, Users have normal rights and Remote Desktop Users may log on over RDP. View membership with `net localgroup administrators`. Best practice is to use a standard account for daily work, keep admin rights separate, and give each person a named account to preserve accountability.

SIDs

Windows tracks identities by Security Identifier, not by name. A SID looks like S-1-5-21-1111111111-222222222-333333333-1001: the prefix identifies the authority, the long numeric part identifies the computer or domain, and the final relative ID (RID) identifies the account. Well-known RIDs include 500 for the built-in Administrator and 501 for Guest. Renaming an account does not change its SID. ACLs store SIDs, which is why a deleted and recreated account with the same name does not inherit old rights. `whoami /user` shows your SID and `whoami /groups` shows group SIDs.

User Account Control

Even if you are an administrator, UAC runs your programs with a filtered standard token until you approve elevation. When something needs admin rights, Windows shows a prompt on the secure desktop, which other programs cannot draw over or click. An administrator sees a consent prompt, whereas a standard user must enter admin credentials. This stops malware that you run by accident from silently changing the system. The prompt shows the publisher; unsigned programs show as unknown. Settings range from always notify to never notify, and lowering the slider weakens an important defense.

Security habits

Treat each UAC prompt as a question: did I just start something that should need admin rights? If a prompt appears unexpectedly, choose No. Do not stay logged in as an administrator, and limit who is in the Administrators group. Disable or rename unneeded built-in accounts, use strong unique passwords and enable Windows Hello or MFA where possible. Audit logon events in the Security log, such as 4624 for successful logon, 4625 for failed logon and 4672 for special privileges assigned. Local administrator password reuse across machines is a common lateral movement path, mitigated by tools like LAPS.

COMMAND OUTPUT

```
C:\> whoami /user
User Name SID
=====
corp\alice S-1-5-21-1111111111-222222222-333333333-1105
C:\> net localgroup administrators
Administrator
alice
svc-backup
```

WORKED EXAMPLE

On a fictional laptop an analyst runs `net localgroup administrators` and sees three members, including `svc-backup`, a service account nobody remembers creating. She checks its creation time, its last logon, and the Security log for event 4720 (account created). Meanwhile she reminds the user to stay logged in as a standard account, so that a UAC prompt, not silent execution, stands between a malicious download and system changes.

KEY TERMS

SID: A unique identifier for a user, group or computer account.

RID: The final part of a SID; 500 is the built-in Administrator.

UAC: Feature that runs admin users with a filtered token until elevation is approved.

SYSTEM: The most privileged local account, used by the operating system.

Event 4625: Security log event for a failed logon.

REMEMBER

- Accounts, groups and SIDs determine access.
- UAC filters admin tokens and prompts on a secure desktop.
- Keep admin membership small and audit logon events.

QUICK CHECK

1. What does UAC do?

- A) Encrypts disks
- B) Requires approval before elevated actions
- C) Blocks the internet
- D) Updates Windows

2. RID 500 identifies:

- A) Guest
- B) The built-in Administrator
- C) SYSTEM
- D) Any new user

3. Which event ID records a failed logon?

- A) 4624
- B) 4625
- C) 4672
- D) 4720

4. Why use a standard account daily?

- A) Better graphics
- B) Limits damage from malware or mistakes
- C) Faster boot
- D) Required by NTFS

Answers: 1-B 2-B 3-B 4-B

1. UAC prompts for elevation consent.
2. 500 is the built-in Administrator.
3. 4625 is failed logon.
4. Less privilege means less impact.

The Registry and Services

The registry holds configuration and autostart locations, and services run programs without a logged-in user, so both are prime places for persistence.

Registry structure

The registry is a hierarchical database of settings. It is organized into hives: HKEY_LOCAL_MACHINE (HKLM) holds machine-wide settings, HKEY_CURRENT_USER (HKCU) holds settings for the logged-in user, HKEY_CLASSES_ROOT holds file associations, HKEY_USERS holds all loaded profiles and HKEY_CURRENT_CONFIG holds the current hardware profile. Inside are keys, similar to folders, and values with names, types such as REG_SZ and REG_DWORD, and data. Browse with regedit or query with reg query. Changing the registry can break the system, so export a key before editing.

Run keys and persistence

Persistence means a program survives a reboot or logoff. The best-known registry locations are the Run and RunOnce keys: HKLM\Software\Microsoft\Windows\CurrentVersion\Run for all users and HKCU\Software\Microsoft\Windows\CurrentVersion\Run for one user. Every value there is a command launched at logon. The HKCU key needs no admin rights, so it is a common choice for malware. Other persistence places include the Startup folder, scheduled tasks, services, WMI subscriptions and Winlogon keys. Reviewing these locations, for instance with the Sysinternals Autoruns tool, is a standard threat-hunting step.

Services

A service is a background program managed by the Service Control Manager. It can start at boot without any user logged in and often runs as SYSTEM, LocalService or NetworkService. Open services.msc to view the name, status, startup type (Automatic, Manual, Disabled) and the account used. From the command line use sc query and sc qc name. Service definitions are stored under HKLM\SYSTEM\CurrentControlSet\Services. Because services run with high privilege, attackers install new services or modify existing ones, so a service with an unexpected name or executable path in a user folder should be investigated.

What to check

Compare against a known-good baseline. Look for Run values pointing to Temp, AppData or Downloads, long random names, commands that launch PowerShell with encoded parameters, and unsigned binaries. For services, check the binary path for unquoted paths with spaces, services running from user-writable folders, and non-Microsoft services you cannot explain. Disable first and investigate, rather than deleting evidence. Record the key's last write time, available in regedit export or tools, because it helps date the change. Protect the registry with least privilege and monitor key changes with Sysmon event ID 13 or auditing.

COMMAND OUTPUT

```
C:\> reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
    OneDrive REG_SZ "C:\Program Files\Microsoft OneDrive\OneDrive.exe" /background
    Updater REG_SZ C:\Users\alice\AppData\Local\Temp\svchost32.exe
C:\> sc query type= service state= all | findstr /i "SERVICE_NAME"
```

WORKED EXAMPLE

Looking at a fictional laptop, the analyst queries the HKCU Run key and finds a value named Updater launching svchost32.exe from the user's Temp folder. The genuine svchost.exe lives in System32 and has no such name. The mismatch in name and location marks it as suspicious persistence. She exports the key for evidence, removes the value after isolating the machine, and checks Temp for the file and its hash.

KEY TERMS

Hive: A top-level section of the registry such as HKLM or HKCU.

Run key: Registry key whose values start programs at logon.

Persistence: A technique that keeps malicious code running after reboot or logoff.

Service: A background program managed by the Service Control Manager.

Autoruns: Sysinternals tool that lists autostart locations.

REMEMBER

- The registry has hives; HKLM is machine-wide, HKCU per user.
- Run keys, services, tasks and Startup are persistence spots.
- Compare to a baseline and check paths and signatures.

QUICK CHECK

1. Which key runs programs for the current user at logon?

- A) HKCU\...\Run
- B) HKLM\SAM
- C) HKCR
- D) HKU\Default only

2. Which tool opens the Services console?

- A) services.msc
- B) regedit
- C) msinfo
- D) gpedit

3. Why is HKCU Run attractive to malware?

- A) No admin rights required
- B) It is encrypted
- C) It is invisible
- D) It is faster

4. Suspicious Run value?

- A) OneDrive in Program Files
- B) svchost32.exe in AppData\Temp
- C) Defender in ProgramData
- D) Edge updater signed by Microsoft

Answers: 1-A 2-A 3-A 4-B

1. HKCU Run applies to the current user.
2. services.msc is the console.
3. Users can write their own HKCU keys.
4. Wrong name, wrong location.

Task Manager, cmd and PowerShell

Command-line and built-in tools let you inspect a Windows host quickly and remotely, and they leave much of the same evidence attackers fear.

Task Manager

Task Manager opens with Ctrl+Shift+Esc. The Processes tab shows applications and background processes with CPU, memory, disk and network use. The Details tab shows PIDs and the user for each process, and the Services tab links services to PIDs. Right-click a process and choose Open file location to see where the executable lives. Startup apps shows what launches at logon. Resource Monitor and Process Explorer from Sysinternals add more detail, such as parent-child relationships and signature verification. Unexpected high CPU or a process with a system-like name in a strange folder deserves a closer look.

cmd essentials

The classic command prompt remains useful. tasklist lists processes, and taskkill /PID 1234 /F ends one. ipconfig /all shows addresses, DNS servers and MAC addresses, and ipconfig /flushdns clears the DNS cache. netstat -ano lists connections and listening ports with owning PIDs, which you then match to tasklist. ping and tracert test reachability, nslookup tests DNS, and systeminfo shows the OS version and installed patches. dir, type and findstr resemble ls, cat and grep. Run as administrator when a command needs elevation, as the prompt title will say so.

PowerShell basics

PowerShell is an object-based shell. Commands, called cmdlets, follow Verb-Noun naming. Get-Process lists processes, Get-Service lists services, Get-LocalUser lists local accounts, Get-NetTCPConnection shows connections and Get-ChildItem lists files. Output is objects, so you can filter with Where-Object and choose columns with Select-Object. For example, Get-Service | Where-Object Status -eq 'Stopped' lists stopped services. Get-Help cmdlet explains usage, and Get-Command finds cmdlets. Pipes pass objects, not text, so no parsing of columns is needed.

Defensive use and logging

Admins and defenders use PowerShell for inventory, hunting and remote management, and attackers abuse it for the same reason, because it is built in. Execution policy is a safety setting that is not a security boundary. Enable PowerShell script block logging (event 4104) and transcription so that commands, even obfuscated ones, are recorded, and consider Constrained Language Mode and application control for sensitive systems. A useful habit is to inspect before acting: Get-Process, Get-NetTCPConnection and Get-ScheduledTask give a quick picture of a host, and Export-Csv saves the results for the case file.

POWERSHELL OUTPUT

```
PS C:\> Get-Service | Where-Object Status -eq 'Stopped' | Select-Object -First 2
Status Name DisplayName
Stopped Fax Fax
Stopped WSearch Windows Search
PS C:\> Get-LocalUser
Name Enabled
Administrator False
alice True
support2 True
```

WORKED EXAMPLE

On a fictional workstation, the analyst runs Get-LocalUser and sees an enabled account called support2 that is not in the asset records. She checks Get-LocalGroupMember Administrators and confirms that it is an administrator, then reviews logon events. In the same session, netstat -ano shows an outbound connection to 198.51.100.77 owned by PID 3320, which tasklist maps to a process in AppData. Both findings go into the case notes.

KEY TERMS

cmdlet: A PowerShell command named in Verb-Noun form, such as Get-Process.

netstat -ano: Lists connections and listening ports with process IDs.

tasklist: cmd tool that lists running processes.

Pipeline objects: PowerShell passes structured objects between cmdlets rather than text.

Script block logging: PowerShell logging (event 4104) that records executed code.

REMEMBER

- Task Manager, tasklist and Get-Process show running programs.
- netstat -ano plus tasklist maps connections to processes.
- PowerShell outputs objects; enable its logging.

QUICK CHECK

1. Which cmdlet lists services?

- A) Get-Process
- B) Get-Service
- C) Get-LocalUser
- D) Get-Item

2. Which command shows connections with PIDs?

- A) ipconfig /all
- B) netstat -ano
- C) ping
- D) dir

3. What does Where-Object do?

- A) Filters objects
- B) Starts a service
- C) Deletes files
- D) Prints help

4. Which log event records PowerShell script blocks?

- A) 4104
- B) 4625
- C) 1102
- D) 7045

Answers: 1-B 2-B 3-A 4-A

1. Get-Service lists services.
2. netstat -ano includes PIDs.
3. It filters pipeline objects.
4. 4104 is script block logging.

Active Directory Introduction

Most organizations manage Windows computers and users through Active Directory, which makes it central to both administration and attacks.

Domains and controllers

Active Directory (AD) is Microsoft's directory service. A domain is a boundary for administration, such as corp.example.com, and its identities live in a database held by domain controllers (DCs), which replicate with each other. Computers join the domain so users sign in with one account anywhere. Multiple domains can form a tree and a forest, the outer security boundary. The directory is queried with LDAP on port 389 (636 for LDAPS), and DNS is essential, since clients locate DCs through DNS records. Because a DC holds every credential, it is the highest-value server on the network.

OUs, users and groups

Objects such as users, computers and groups are organized in Organizational Units (OUs), which work like folders and are the units to which you delegate administration and attach policy. Security groups give permissions, and the standard guidance is AGDLP or a similar scheme: place users in groups, then grant rights to groups rather than individuals. Privileged groups such as Domain Admins and Enterprise Admins must be tiny and monitored. Tools include Active Directory Users and Computers (dsa.msc) and PowerShell Get-ADUser from the RSAT module.

Group Policy

Group Policy Objects (GPOs) are collections of settings applied to users or computers in the OUs, sites or domain they link to. They configure password rules, firewall settings, software restrictions, drive mappings and security options, and are refreshed at intervals and at logon. gpupdate /force reloads them and gpresult /r shows which are applied. Policies process in order local, site, domain, OU, with later ones overriding earlier ones. Because a single GPO can reach thousands of machines, who may edit them must be strictly controlled.

Kerberos at a high level

Kerberos is the default domain authentication protocol, on port 88. When you log on, you prove your identity to the Key Distribution Center on the DC and receive a Ticket Granting Ticket (TGT). When you want to use a file server, you present the TGT and receive a service ticket for that service, which you present to the server. Passwords are not sent across the network after the initial step, and tickets expire, typically after about ten hours. Clock skew over five minutes breaks it, so time sync matters. Weaknesses such as weak service account passwords are why defenders monitor ticket requests.

COMMAND OUTPUT

```
C:\> gpresult /r
COMPUTER SETTINGS
    Applied Group Policy Objects
        Default Domain Policy
        Workstation Hardening
PS C:\> Get-ADUser -Identity jdoe -Properties MemberOf | Select-Object Name,MemberOf
```

WORKED EXAMPLE

A fictional company wants all laptops in the Sales OU to enforce a 14-character minimum password and lock after five minutes of inactivity. The administrator creates a GPO named Sales Security Baseline, links it to the Sales OU, not the whole domain, and runs gpupdate /force on a test laptop. gpresult /r confirms it applies. Linking at the OU level kept the change from affecting servers and other departments.

KEY TERMS

Domain controller: A server that holds the AD database and authenticates users.

OU: Organizational Unit, a container for delegating control and linking policy.

GPO: Group Policy Object, a bundle of settings applied to users or computers.

TGT: Kerberos Ticket Granting Ticket, issued after authentication.

LDAP: The protocol used to query and modify directory data.

REMEMBER

- A domain is run by domain controllers holding the directory.
- OUs organize objects; GPOs apply settings to them.
- Kerberos uses tickets; protect DCs and privileged groups.

QUICK CHECK

1. What is a domain controller?
 - A) A router
 - B) A server holding AD and authenticating users
 - C) A firewall
 - D) A printer
2. Where are GPOs linked to take effect?
 - A) To OUs, sites or the domain
 - B) Only to files
 - C) Only to firewalls
 - D) To DNS records
3. Kerberos default port?
 - A) 88
 - B) 443
 - C) 3389
 - D) 25
4. What does the TGT allow?
 - A) Requesting service tickets
 - B) Editing GPOs
 - C) Joining the forest
 - D) Disabling DNS

Answers: 1-B 2-A 3-A 4-A

1. DCs host the directory.
2. GPOs link to containers.
3. Kerberos uses 88.
4. A TGT is used to obtain service tickets.

Windows Defender and Firewall

Built-in antimalware and a host firewall are the first protective layers on every Windows machine, and configuring them correctly blocks a lot of attacks.

Microsoft Defender Antivirus

Microsoft Defender Antivirus is built into Windows. Real-time protection scans files as they are opened, downloaded or run. Cloud-delivered protection checks unknown files against Microsoft's service, and tamper protection blocks changes to Defender settings by malware or scripts. Run scans from the Windows Security app, or from PowerShell with `Start-MpScan -ScanType QuickScan`, and check state with `Get-MpComputerStatus`. Signatures update automatically through Windows Update. A full scan reads every file and takes longer than a quick scan. Turning protection off to run a tool is a habit to avoid, because malware may use that window.

Detections and exclusions

When Defender detects a threat it quarantines it, and the history lists the name, path and action. Treat each detection as an incident indicator: ask how the file arrived and whether other machines have it. Exclusions tell Defender to skip a path, file type or process, and are sometimes needed for compatibility but are a security trade-off, because attackers drop payloads into excluded folders. Review them regularly with `Get-MpPreference`. Enterprise tools, such as Defender for Endpoint, add behavior monitoring and central reporting. Antivirus alone is not enough, which is why layers such as patching, firewalling and least privilege matter.

Windows Defender Firewall

The firewall is a stateful host firewall that filters inbound and outbound traffic. By default it blocks unsolicited inbound connections and allows outbound traffic. Rules specify direction, action (allow or block), protocol, local and remote ports, program and remote addresses. Examples: an inbound allow for TCP 3389 enables Remote Desktop, and an inbound block for TCP 445 stops file sharing. Manage with `wf.msc`, Windows Security, or PowerShell commands such as `Get-NetFirewallRule` and `New-NetFirewallRule`. Because it is stateful, replies to allowed outbound connections are accepted automatically.

Profiles and rule design

Three profiles apply depending on the network: Domain when connected to the corporate domain, Private for trusted home or work networks and Public for untrusted networks like cafe Wi-Fi, which should be the most restrictive. Rules can be scoped to a profile. For services you must expose, restrict them: allow RDP only from a management subnet such as 192.0.2.0/24 rather than from any address, and prefer a VPN or gateway. Disable or delete broad rules like allow all inbound, and remove rules for software that is uninstalled. Test changes and monitor the firewall log, which records dropped connections when logging is enabled.

POWERSHELL OUTPUT

```
PS C:\> Get-MpComputerStatus | Select-Object RealTimeProtectionEnabled,AntivirusSignatureLastUpdated
RealTimeProtectionEnabled AntivirusSignatureLastUpdated
-----
True 3/3/2025 8:01:12 AM
PS C:\> New-NetFirewallRule -DisplayName 'RDP from mgmt' -Direction Inbound -Protocol TCP -LocalPort 3389
-RemoteAddress 192.0.2.0/24 -Action Allow
```

WORKED EXAMPLE

A server must allow Remote Desktop for administrators on 192.0.2.0/24 only. Currently an old rule allows TCP 3389 from any address, exposing it to the internet. The administrator creates a new inbound allow rule restricted to 192.0.2.0/24, disables the old broad rule, and confirms from an outside test address that port 3389 no longer answers. The service stays reachable for its intended users while the exposure disappears.

KEY TERMS

Real-time protection: Defender feature that scans files and processes as they are accessed.

Tamper protection: Setting that prevents unauthorized changes to Defender configuration.

Stateful firewall: A firewall that tracks connections and automatically allows replies.

Firewall profile: Domain, Private or Public set of rules chosen by network type.

Exclusion: A path or process that Defender is told not to scan.

REMEMBER

- Keep real-time protection and tamper protection on and review exclusions.
- The firewall blocks inbound by default; scope allow rules tightly.
- Use profiles; Public should be the strictest.

QUICK CHECK

1. Which firewall profile is for untrusted networks?
 - A) Domain
 - B) Private
 - C) Public
 - D) Admin
2. What does tamper protection prevent?
 - A) Unauthorized changes to Defender settings
 - B) Windows updates
 - C) Logins
 - D) Pings
3. Best way to expose RDP?
 - A) Allow from any address
 - B) Allow only from a management subnet or VPN
 - C) Disable the firewall
 - D) Use port 80
4. Default firewall behavior for unsolicited inbound traffic?
 - A) Allow
 - B) Block
 - C) Log only
 - D) Redirect

Answers: 1-C 2-A 3-B 4-B

1. Public is the most restrictive.
2. It blocks disabling Defender.
3. Scope the rule to trusted sources.
4. Inbound is blocked by default.

Find the Autorun Entry

SCENARIO

A fictional laptop WS-014 shows odd behavior at logon. You are given a registry browser with the Run keys. Review each value, judge by name, path and signature, and flag the entry that launches an unknown executable.

OBJECTIVE

Browse registry Run keys and flag the entry that launches an unknown executable at login.

STEPS IN THE SIMULATOR

- 1 **Do:** Open HKLM\Software\Microsoft\Windows\CurrentVersion\Run

Simulator shows:

SecurityHealth REG_EXPAND_SZ %windir%\system32\SecurityHealthSystray.exe
RealtekAudio REG_SZ C:\Program Files\Realtek\Audio\HDA\RtKNGUI64.exe

- 2 **Do:** Mark both as expected

Simulator shows:

Both paths are in protected folders and signed.

- 3 **Do:** Open HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Simulator shows:

OneDrive REG_SZ "C:\Program Files\Microsoft OneDrive\OneDrive.exe" /background
Updater REG_SZ C:\Users\alice\AppData\Local\Temp\svchost32.exe

- 4 **Do:** Inspect the Updater value

Simulator shows:

Signature: not signed. File created 1 day ago.

- 5 **Do:** Flag Updater

Simulator shows:

Flag recorded: suspicious persistence.

- 6 **Do:** Choose the response: disable value and isolate

Simulator shows:

Checker: correct, evidence exported first.

ANSWER

The entry Updater in HKCU Run launches C:\Users\alice\AppData\Local\Temp\svchost32.exe, an unsigned program in a user-writable folder with a name imitating svchost.exe. It is likely malware persistence and should be exported as evidence, then disabled.

Hint shown in the app: Check path and signature, not just the name.

PowerShell Inspection

SCENARIO

On a fictional workstation WS-021 you have a PowerShell prompt. Your task is a quick inventory: running processes, services that are stopped and local accounts, looking for anything that does not belong.

OBJECTIVE

Run Get-Process, Get-Service and Get-LocalUser to list stopped services and unexpected local accounts.

STEPS IN THE SIMULATOR

- 1 **Do:** Type Get-Process | Sort-Object CPU -Descending | Select-Object -First 3

Simulator shows:

```
Name Id CPU
chrome 4120 311.2
svchost 1012 80.1
Explorer 5088 55.0
```

- 2 **Do:** Type Get-Service | Where-Object Status -eq 'Stopped' | Select-Object -First 3

Simulator shows:

```
Stopped Fax
Stopped WSearch
Stopped RemoteRegistry
```

- 3 **Do:** Type Get-LocalUser

Simulator shows:

```
Name Enabled
Administrator False
Guest False
alice True
support2 True
```

- 4 **Do:** Type Get-LocalGroupMember Administrators

Simulator shows:

```
WS-021\Administrator
WS-021\alice
WS-021\support2
```

- 5 **Do:** Type Get-LocalUser support2 | Select-Object Name,LastLogon,PasswordLastSet

Simulator shows:

```
support2 3/2/2025 11:48 PM 3/2/2025 11:40 PM
```

ANSWER

The account support2 is an unexpected local administrator, enabled and created hours ago. Stopped services such as Fax and RemoteRegistry are normal. support2 should be investigated, disabled and removed from Administrators.

Hint shown in the app: Compare the account list to what your asset records say.

Firewall Rule Fix

SCENARIO

A fictional server SRV-APP has a firewall with a broad rule that allows inbound RDP (TCP 3389) from any address. Administrators work only from 192.0.2.0/24. Fix it by adding a scoped rule and disabling the risky one.

OBJECTIVE

Create an inbound rule that allows only RDP from one subnet and disable a risky open rule.

STEPS IN THE SIMULATOR

1 Do: Open Inbound Rules

Simulator shows:

Remote Desktop - Any (TCP 3389) Allow, Remote Address: Any, Enabled

2 Do: Select the risky rule and choose Disable

Simulator shows:

Rule disabled.

3 Do: Create new rule: Port, TCP, 3389

Simulator shows:

Wizard step: action.

4 Do: Action: Allow the connection

Simulator shows:

Wizard step: profile.

5 Do: Scope Remote IP: 192.0.2.0/24

Simulator shows:

Scope saved.

6 Do: Profiles: Domain only; Name: RDP from mgmt

Simulator shows:

Rule created and enabled.

7 Do: Run external test from 203.0.113.50

Simulator shows:

Connection to 3389 timed out (blocked).

ANSWER

A new inbound TCP 3389 allow rule scoped to remote address 192.0.2.0/24 exists, and the any-address RDP rule is disabled. External addresses can no longer connect, while the management subnet can.

Hint shown in the app: Scope the remote address, do not just open the port.

Active Directory Tree

SCENARIO

Build the logical structure for a fictional domain corp.example.com with a Sales OU and an IT OU. Create users and a group, then link a password-and-lockout GPO to the Sales OU only.

OBJECTIVE

Build a domain with OUs, users and groups and attach a group policy to the correct OU.

STEPS IN THE SIMULATOR

- 1 Do:** Add domain node corp.example.com
Simulator shows:
Domain placed.
- 2 Do:** Add OU Sales and OU IT under the domain
Simulator shows:
Two OUs created.
- 3 Do:** Add users jdoe, asmith to Sales; tadmin to IT
Simulator shows:
Users placed in OUs.
- 4 Do:** Add group SalesStaff in Sales; add jdoe and asmith
Simulator shows:
Group membership recorded.
- 5 Do:** Create GPO 'Sales Security Baseline'
Simulator shows:
GPO created, not linked.
- 6 Do:** Drag the GPO onto the Sales OU
Simulator shows:
Linked: applies to jdoe, asmith and Sales computers.
- 7 Do:** Run check
Simulator shows:
Pass: IT OU is unaffected; GPO linked at OU level.

ANSWER

The GPO Sales Security Baseline is linked to the Sales OU, which contains jdoe, asmith and the SalesStaff group. IT users are unaffected because the policy is linked at the OU level, not the domain.

Hint shown in the app: Link policy at the narrowest container that needs it.

Windows Fundamentals: 10 questions, 80% to pass

1. Where do 64-bit applications install by default?
 - A) C:\Windows
 - B) C:\Program Files
 - C) C:\Users
 - D) C:\Temp
2. What does UAC do when an admin starts an elevated task?
 - A) Prompts for consent
 - B) Disables antivirus
 - C) Deletes files
 - D) Reboots
3. Which registry path runs a program at each logon for one user?
 - A) HKCU\Software\Microsoft\Windows\CurrentVersion\Run
 - B) HKLM\SAM
 - C) HKCR\txt
 - D) HKLM\HARDWARE
4. What does `dir /r` reveal?
 - A) Alternate data streams
 - B) Hidden services
 - C) Registry keys
 - D) Open ports
5. Which cmdlet lists local accounts?
 - A) Get-LocalUser
 - B) Get-Process
 - C) Get-Service
 - D) Get-Item
6. Which tool shows connections with PIDs?
 - A) netstat -ano
 - B) ipconfig
 - C) dir
 - D) type
7. An OU is used to:
 - A) Encrypt files
 - B) Organize objects and delegate or apply policy
 - C) Route packets
 - D) Store passwords
8. Kerberos gives you a TGT after:
 - A) You authenticate to the KDC
 - B) You reboot
 - C) You open a file
 - D) You change DNS
9. Which Windows firewall profile should be strictest?
 - A) Public
 - B) Domain
 - C) Private
 - D) None
10. Safest way to expose RDP?
 - A) Any address
 - B) Restrict to a management subnet or VPN
 - C) Disable Defender
 - D) Use guest account

Answers: 1-B 2-A 3-A 4-A 5-A 6-A 7-B 8-A 9-A 10-B

1. Program Files holds 64-bit apps.
2. It asks for approval.
3. HKCU Run is per user.
4. /r lists NTFS streams.
5. Get-LocalUser.
6. netstat -ano includes PIDs.
7. OUs group objects.
8. The TGT is issued after login.
9. Public networks are untrusted.
10. Limit the source addresses.

Scripting Basics

Beginner | 6 lessons | 4 simulated labs | checkpoint | about 4h

Scripting turns repetitive security chores into repeatable tools. This module introduces Bash and Python side by side: variables and data types, conditions, loops, functions and arguments, reading files, and regular expressions. You finish by building a small log analyzer that counts failed logins and writes a report. These skills let you sweep a range of addresses, extract indicators from logs and automate checks, and they are the base for nearly every later security task.

BY THE END YOU CAN

- + Write Bash and Python scripts using variables, conditions and loops
- + Read files and extract data with regex
- + Automate small repetitive tasks with functions

IN THIS MODULE

Lesson 1 Variables and Data Types

Lesson 2 Conditions and Comparisons

Lesson 3 Loops

Lesson 4 Functions and Arguments

Lesson 5 Reading Files and Regex

Lesson 6 Automating a Small Task

Lab 1 Fix the Script (Code editor)

Lab 2 Ping Sweep Loop (Code editor)

Lab 3 Regex IP Extractor (Code editor)

Lab 4 Log Summary Tool (Code editor)

Checkpoint 10 questions, 80% to pass

Variables and Data Types

Variables hold the data every script works on, and understanding types prevents the most common beginner bugs.

Variables in Bash

In Bash a variable is set with name=value and no spaces around the equals sign: host=192.0.2.10. You read it with a dollar sign, as in echo \$host, and quoting it as "\$host" is the safe habit, because unquoted values containing spaces or wildcards get split and expanded. Bash treats nearly everything as a string, and arithmetic uses \$(()), for example count=\$((count + 1)). Command output is captured with \$(), as in today=\$(date +%F). Variables are local to the shell unless exported with export. By convention, uppercase names are used for environment variables such as PATH and HOME.

Variables in Python

Python variables are created by assignment and have a type determined by the value: host = '192.0.2.10', port = 22, ratio = 0.75, active = True. Spaces around = are fine. Python is dynamically typed but strongly typed, so '22' + 1 raises an error rather than guessing. The main built-in types are str, int, float, bool, list, tuple, dict and set. Use type(x) to inspect a value and int('22') or str(22) to convert. Names are case sensitive, should be lowercase with underscores, and cannot start with a digit. Comments begin with #, just as in Bash.

Strings, integers, lists and dictionaries

Strings hold text and can be sliced: host[0:3] gives the first three characters, and f-strings such as f'{host}:{port}' build text. Integers support arithmetic and comparison. A list is an ordered, changeable collection: ports = [22, 80, 443], with ports[0] being 22 and ports.append(8080) adding an item. A dictionary maps keys to values: user = {'name': 'alice', 'uid': 1001}, read with user['name']. Counting events per IP is a natural dictionary use. Bash has arrays, ports=(22 80 443) with \${ports[0]}, and associative arrays declared with declare -A, though Python is usually easier for structured data.

Good habits and security notes

Pick descriptive names such as failed_count rather than x. Never hard-code passwords or API keys in scripts; read them from environment variables or a protected file, because scripts get shared and committed to repositories. Treat values from outside the script, like arguments and file contents, as untrusted data: in Bash, an unquoted variable inside a command can lead to unexpected word splitting or command injection. Add set -u to Bash scripts to fail on unset variables. Test small pieces in an interactive session, python3 or bash, before assembling them into a program.

CODE

```
# Bash
host=192.0.2.10
ports=(22 80 443)
echo "Scanning $host on ${#ports[@]} ports"
# Python
host = '192.0.2.10'
ports = [22, 80, 443]
attempts = {'203.0.113.9': 212}
print(f'Scanning {host} on {len(ports)} ports')
```

WORKED EXAMPLE

A learner wants to track failed logins per address. In Python she creates an empty dictionary, attempts = {}, then for each address she writes attempts[ip] = attempts.get(ip, 0) + 1. After processing, attempts holds {'203.0.113.9': 212, '198.51.100.7': 14}. A list of ports and a string for the host cover the rest. Choosing a dictionary meant each address was a key and each count a value, avoiding two parallel lists.

KEY TERMS

Variable: A named storage location for a value.

String: A sequence of text characters.

List: An ordered, changeable collection of items.

Dictionary: A collection of key and value pairs.

Command substitution: \$() syntax that captures a command's output in Bash.

REMEMBER

- Bash uses name=value with no spaces; quote expansions.
- Python types include str, int, list and dict.
- Keep secrets out of scripts and treat inputs as untrusted.

QUICK CHECK

1. Which Bash assignment is correct?

- A) host = 1
- B) host=1
- C) \$host=1
- D) set host 1

2. Which Python type maps keys to values?

- A) list
- B) dict
- C) tuple
- D) str

3. How do you read a Bash variable?

- A) host
- B) \$host
- C) @host
- D) %host%

4. Why avoid hard-coded passwords in scripts?

- A) Python forbids them
- B) Scripts are shared and committed
- C) They slow the script
- D) They change types

Answers: 1-B 2-B 3-B 4-B

1. No spaces around = in Bash.
2. dict stores key and value pairs.
3. A dollar sign expands it.
4. Secrets in code leak easily.

Conditions and Comparisons

Branching lets a script react to what it finds, such as a host that is up or a file that is missing.

if in Python

In Python a condition is an `if`, optional `elif` and `else`, each ending in a colon, with the body indented. Comparison operators are `==`, `!=`, `<`, `>`, `<=`, `>=`, and the boolean operators are `and`, `or`, `not`. For example: `if failed > 100: print('block') elif failed > 10: print('watch') else: print('ok')`. Indentation, usually four spaces, defines blocks, so a wrong indent is a syntax error. A common mistake is using `=` (assignment) where `==` (comparison) is needed. Membership tests with `in` are handy: `if '22' in ports`. Empty strings, zero and empty lists count as false.

if in Bash

Bash uses `if`, `then`, `elif`, `else` and `fi`. The test goes in `[ ]` or the safer `[[ ]]`: `if [[ $count -gt 100 ]]; then echo block; fi`. Numeric comparisons use `-eq`, `-ne`, `-gt`, `-ge`, `-lt` and `-le`, while strings use `==` and `!=`. File tests include `-f` for a regular file, `-d` for a directory and `-r` for readable. Spaces inside the brackets are required. Using string operators on numbers or numeric operators on strings gives wrong results or errors, a very frequent beginner trap. Quote variables inside tests so an empty value does not break the syntax.

Exit codes

Every Unix command ends with an exit status: 0 means success and any other number means failure. The variable `$?` holds the status of the last command. You can branch on a command directly: `if grep -q 'Failed password' auth.log; then echo found; fi`, where `-q` suppresses output and only the status matters. Your own scripts should end with exit 0 on success and exit 1 or another code on failure so that other tools, such as cron or CI systems, know what happened. In Python, `sys.exit(1)` does the same and subprocess results expose `returncode`.

Combining and testing logic

Combine tests with `&&` and `||` in Bash or `and` and `or` in Python, and group with parentheses to control order. Build tests for the edge cases: zero events, one event, a missing file and unexpected input. For example, check that the file exists before reading it, and check that an argument was supplied before using it. Off-by-one errors with greater-than versus greater-or-equal are common, and so is comparing numbers stored as text. Print values while debugging, then remove the extra output. Simple, explicit conditions are easier to review and safer than clever one-liners.

CODE

```
# Python
if failed > 100:
    print('block')
elif failed > 10:
    print('watch')
else:
    print('ok')
# Bash
if [[ $failed -gt 100 ]]; then echo block; fi
if grep -q 'Failed' auth.log; then echo found; fi
```

WORKED EXAMPLE

A script must decide what to do with an address. It counts 212 failures for 203.0.113.9. With the thresholds above, 212 is greater than 100, so it prints block. For 14 failures, the first test fails and the `elif` for greater than 10 succeeds, printing watch. A learner who wrote `failed = 100` in the condition would overwrite the count and cause a syntax error, a reminder that `==` compares and `=` assigns.

KEY TERMS

Condition: An expression that is true or false and controls the flow.

Exit status: A number returned by a command; 0 means success.

elif: Python keyword for an additional condition after `if`.

-gt: Bash numeric comparison for greater than.

Indentation: Leading whitespace that defines blocks in Python.

REMEMBER

- Python uses if/elif/else with colons and indentation; Bash uses then and fi.
- Bash has separate numeric and string operators.
- Exit status 0 means success and drives branching.

QUICK CHECK

1. Which Bash operator tests numeric greater than?

- A) -gt
- B) >>
- C) gt
- D) =>

2. What exit status means success?

- A) 0
- B) 1
- C) -1
- D) 255

3. Which Python line is a valid comparison?

- A) if x = 5:
- B) if x == 5:
- C) if x := 5
- D) if (x) = 5

4. What does grep -q do in an if test?

- A) Prints more
- B) Suppresses output and returns status
- C) Counts lines
- D) Quits the shell

Answers: 1-A 2-A 3-B 4-B

- 1. -gt is numeric greater than.
- 2. Zero is success.
- 3. == compares.
- 4. -q is quiet and sets exit code.

Loops

Loops apply the same action to many items, such as every host in a range or every line of a log.

for loops in Python

A for loop in Python walks over any sequence: for port in [22, 80, 443]: print(port). The range function generates numbers: range(1, 6) gives 1 to 5, since the end is excluded, and range(5) starts at 0. You can iterate over dictionary items with for ip, count in attempts.items(). Use enumerate when you also need the position, and break to leave early or continue to skip to the next item. The loop body is indented. Looping over a collection is cleaner and safer than managing a counter yourself.

for loops in Bash

Bash for loops look like for host in web01 db01 app01; do ping -c 1 \$host; done. Brace expansion generates ranges, such as for i in {1..10}, and the C-style form for ((i=1; i<=10; i++)) also works. A ping sweep of a fictional network is for i in {1..254}; do ping -c 1 -W 1 192.0.2.\$i > /dev/null && echo 192.0.2.\$i is up; done. The && means the echo runs only if ping succeeded, and the output redirect hides the noise. Only scan networks you own or are authorized to test, since sweeping other people's networks may be illegal.

while loops

A while loop repeats as long as a condition is true. In Python: while attempts < 3: ... attempts += 1. In Bash: while read -r line; do echo "\$line"; done < auth.log, which reads a file one line at a time through input redirection. The -r option stops backslashes being interpreted. A loop must change something so its condition eventually becomes false, or it runs forever, which can be intended for a monitor but must then include a sleep to avoid burning CPU. Stop a runaway script with Ctrl+C.

Looping over file lines

Reading a log line by line is the core of many tools. In Python, with open('auth.log') as f: for line in f: ... opens the file safely, closes it automatically and does not load the entire file into memory. Strip the trailing newline with line.strip(). In Bash, use while read -r line, and avoid for line in \$(cat file), which splits on every space. Inside the loop test each line, extract fields and update counters. Keep the loop small and push complex work into functions, which also makes the script easier to test on a few sample lines first.

CODE

```
# Bash ping sweep (fictional lab range)
for i in {1..5}; do
    ping -c 1 -W 1 192.0.2.$i > /dev/null && echo "192.0.2.$i up"
done
# Python
with open('auth.log') as f:
    for line in f:
        if 'Failed' in line:
            print(line.strip())
```

WORKED EXAMPLE

An analyst wants a list of the live addresses in the lab range 192.0.2.1 to 192.0.2.20. She uses a for loop over {1..20}, pings each with one packet and a one-second timeout, and prints only those that answer. The output lists 192.0.2.1, .5 and .12. The same logic in Python would use range(1, 21) and the subprocess module. The loop replaced twenty manual commands with one.

KEY TERMS

for loop: Repeats a block for each item in a sequence.

while loop: Repeats a block while a condition remains true.

range(): Python function that generates a sequence of integers.

break: Statement that exits a loop early.

Brace range: Bash {1..10} expansion producing a number sequence.

REMEMBER

- for iterates over items; while repeats on a condition.
- Read files line by line rather than loading everything.
- Only sweep networks you are authorized to test.

QUICK CHECK

1. What does `range(1, 4)` produce in Python?
 - A) 1,2,3,4
 - B) 1,2,3
 - C) 0,1,2,3
 - D) 2,3,4
2. Which Bash loop reads a file line by line safely?
 - A) `for l in $(cat f)`
 - B) `while read -r line; do ... done < f`
 - C) `until f`
 - D) `loop f`
3. What does `&&` do after ping?
 - A) Runs the next command only if ping succeeds
 - B) Runs in the background
 - C) Pipes output
 - D) Repeats ping
4. Why use `with open()` in Python?
 - A) It auto-closes the file
 - B) It makes it faster
 - C) It encrypts
 - D) It sorts lines

Answers: 1-B 2-B 3-A 4-A

1. The end value is excluded.
2. `while read` handles whole lines.
3. `&&` chains on success.
4. The context manager closes the file.

Functions and Arguments

Functions let you name and reuse logic, and arguments make a script flexible instead of hard-coded.

Defining Python functions

A Python function is defined with `def`, a name, parameters in parentheses, a colon and an indented body: `def is_private(ip): ... return True`. `return` sends a value back to the caller; without it the function returns `None`. Call it by name with arguments, such as `is_private('192.0.2.5')`. Parameters can have defaults, `def top(items, n=5)`, and may be passed by keyword. Variables created inside a function are local to it. Small functions that do one thing are easier to test, reuse and understand, and a docstring in triple quotes documents what they do.

Functions in Bash

Bash defines a function as `name() { commands; }`. Inside it the arguments are `$1`, `$2` and so on, with `$#` giving the count and `"$@"` giving all of them. A function's return value is a numeric exit status, 0 to 255, set with `return`; to pass back text, print it with `echo` and capture it using `$( )`. Variables are global by default, so declare them local to avoid surprises. Example: `check_host() { ping -c 1 -W 1 "$1" > /dev/null; }` lets you write `if check_host 192.0.2.7; then ... fi`.

Script arguments

Scripts receive command-line arguments just like functions. In Bash the script name is `$0`, the first argument is `$1`, and `$#` is the count, so run it as `./scan.sh 192.0.2.10` and check `[[ $# -lt 1 ]]` to print a usage message and exit 1. In Python, import `sys` and read `sys.argv`, a list where `sys.argv[0]` is the script name and `sys.argv[1]` is the first argument; the `argparse` module adds flags, help text and type checks. Always validate arguments: check the count, check that a file exists, and never pass raw input into a command line without quoting.

Structuring a script

A tidy script has a short header comment, constants at the top, functions in the middle and a main section at the bottom. In Python, wrap the entry point in `if __name__ == '__main__': main()` so the file can also be imported and tested. Return values and print only at the edges, so the logic stays reusable. Handle errors clearly, with messages to standard error and nonzero exit codes. Avoid copy-pasting blocks of code; if you repeat something twice, make it a function. These habits let a security team trust and maintain tools written months earlier.

CODE

```
# Bash
check_host() { ping -c 1 -W 1 "$1" > /dev/null; }
[[ $# -lt 1 ]] && { echo "usage: $0 host"; exit 1; }
check_host "$1" && echo "$1 is up"
# Python
import sys
def top(counts, n=5):
    return sorted(counts.items(), key=lambda kv: kv[1], reverse=True)[:n]
print(top({'a': 3, 'b': 9}, 1))
```

WORKED EXAMPLE

A teammate keeps editing the address in a script. The learner changes it to read the address from the command line: Bash uses `$1`, and Python uses `sys.argv[1]`. She adds a check that prints usage and exits with code 1 when no argument is supplied. She then wraps the ping command in a function `check_host` so the same code is used for a single host and for a loop. The tool now works for any address and fails clearly when misused.

KEY TERMS

Function: A named, reusable block of code.

Parameter: A variable in a function definition that receives an argument.

\$1: The first positional argument in Bash.

sys.argv: Python list holding command-line arguments.

return: Statement that ends a function and gives back a value.

REMEMBER

- Functions name reusable logic; Python uses def and return.
- Bash functions use \$1, \$2 and exit status.
- Validate arguments and print usage on bad input.

QUICK CHECK

1. Which Bash variable holds the first argument?
 - A) \$0
 - B) \$1
 - C) \$#
 - D) \$@
2. What is sys.argv[0] in Python?
 - A) First argument
 - B) The script name
 - C) Number of arguments
 - D) The interpreter
3. What does \$# contain in Bash?
 - A) Process ID
 - B) Argument count
 - C) Exit status
 - D) Script name
4. What does a Python function return if there is no return statement?
 - A) 0
 - B) None
 - C) False
 - D) An error

Answers: 1-B 2-B 3-B 4-B

1. \$1 is the first argument.
2. Index 0 is the script name.
3. \$# is the number of arguments.
4. It returns None.

Reading Files and Regex

Security data is mostly text in logs, and regular expressions let you pull out the IPs, emails and timestamps that matter.

Reading files

In Python, open a file with `with open(path) as f:` and iterate for `line in f:`. Use `f.read()` for the whole content, `f.readlines()` for a list, and the mode `'w'` or `'a'` to write or append. Catch `FileNotFoundError` and `PermissionError` to fail gracefully. In Bash, redirect with `<` or use `while read -r`. Specify `encoding='utf-8'` and `errors='replace'` for logs that may contain odd bytes. Logs can be very large, so process them line by line. Check that the file path is what you expect, since a script reading a file chosen by a user should not follow input into sensitive locations.

Regular expression building blocks

A regular expression, or regex, describes a text pattern. Literal characters match themselves. A dot matches any character, `\d` matches a digit, `\w` a word character and `\s` whitespace. Quantifiers say how many: `*` zero or more, `+` one or more, `?` optional, `{3}` exactly three and `{1,3}` one to three. Brackets define sets, like `[A-Za-z]`, and parentheses capture groups. The anchors `^` and `$` match start and end of a line, and the pipe means or. A backslash makes a special character literal, so a literal dot is `\.` in the pattern. Use raw strings in Python, `r'\d+'`, to avoid doubled backslashes.

Matching IPs, emails and timestamps

A practical IPv4 pattern is `r'\b(?:\d{1,3}\.){3}\d{1,3}\b'`. It finds the shape of an address but also accepts `999.1.1.1`, so validate ranges afterwards with the `ipaddress` module. A simple email pattern is `r'[\w.-]+@[[\w.-]+\.[\w.-]+'`. A syslog timestamp such as `Mar 3 09:20:37` matches `r'[A-Z][a-z]{2}\s+\d{1,2} \d{2}:\d{2}:\d{2}'`. In Python use `re.search` for the first match, `re.findall` for all matches and `re.compile` to reuse a pattern. In Bash, `grep -oE` prints only the matching part.

Pitfalls and testing

Regex is powerful but easy to get wrong. Greedy quantifiers match as much as possible, so `.*` can swallow too much; use `.*?` for the shortest match. Test patterns on sample lines, including ones that should not match, with a tool or a small script. Avoid patterns with nested quantifiers on untrusted input, which can cause catastrophic backtracking and a denial of service. Do not try to parse complex formats like HTML with regex, and prefer a JSON or CSV library when data is structured. Regex finds candidates; verify them. Remember that extracted indicators, such as IPs, may be spoofed or shared hosts.

CODE

```
import re
ip_re = re.compile(r'\b(?:\d{1,3}\.){3}\d{1,3}\b')
line = 'Mar 3 09:20:37 web01 sshd[2210]: Failed password for root from 203.0.113.9 port 51520'
print(ip_re.findall(line))
# ['203.0.113.9']
# Bash
grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' auth.log | sort -u
```

WORKED EXAMPLE

A learner has a sample log with lines from `203.0.113.9`, `198.51.100.7` and `203.0.113.9` again. She compiles the IPv4 pattern, runs `findall` on each line and adds each result to a set, which stores only unique values. The set ends with two addresses. She then tests the pattern against a line containing the text version `1.2.3`, to make sure the pattern does not wrongly report a three-part number as an address.

KEY TERMS

Regex: A pattern language for matching text.

Quantifier: A symbol like `*`, `+` or `{n}` that says how many times to match.

Capture group: Part of a pattern in parentheses whose match is extracted.

Raw string: Python `r"` string where backslashes are literal.

Greedy match: A match that takes as much text as possible.

REMEMBER

- Read files line by line using `with open` or `while read`.

- Regex uses classes, quantifiers, anchors and groups.
- Test patterns, validate results and avoid greedy surprises.

QUICK CHECK

1. What does `\d` match?
 - A) A digit
 - B) A dot
 - C) Any letter
 - D) Whitespace
2. What is the safest way to write a literal dot in regex?
 - A) `.`
 - B) `\.`
 - C) `*`
 - D) `?`
3. Which function returns all matches in Python's `re`?
 - A) `re.search`
 - B) `re.findall`
 - C) `re.sub`
 - D) `re.split`
4. Why validate matches from an IPv4 regex?
 - A) Pattern also matches invalid numbers like 999.1.1.1
 - B) It is too slow
 - C) Regex cannot match digits
 - D) IPs are secret

Answers: 1-A 2-B 3-B 4-A

1. `\d` is a digit.
2. Escape the dot.
3. `findall` returns a list of matches.
4. The pattern checks shape, not range.

Automating a Small Task

Putting loops, conditions, functions and regex together shows how scripting saves hours and produces consistent reports.

Plan before coding

Start by stating the task in one sentence: read auth.log, count failed SSH logins per source address, print the top five and save a report. Then list the inputs (a file path), the processing (find failed lines, extract addresses, count) and outputs (a text report and an exit code). Sketch the steps as functions: parse_line, count_failures, write_report. Try each step by hand on a few sample lines before scripting them. Planning avoids the common problem of a long script that no one, including its author, can debug.

Parsing and counting

The core is a loop that reads lines, filters, extracts and counts. In Python: for line in f: if 'Failed password' in line: m = ip_re.search(line); if m: counts[m.group()] = counts.get(m.group(), 0) + 1. Counter from the collections module simplifies this: counts = Counter(); counts.update([ip]). Sorting by count uses counts.most_common(5). The Bash equivalent is the pipeline `grep 'Failed password' auth.log | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort | uniq -c | sort -nr | head -n 5`. Both give the same result; choose Python when you need more logic.

Writing the report

A good report states what was analyzed and when, and presents the results clearly. Write it with `open('report.txt', 'w')`, including a header with the file name and date, the total number of failed attempts and one line per top address. Using the `datetime` module adds a timestamp. Use append mode only if you intend to keep history. Choose a sensible output location and permissions, because a report with attacker data and usernames is sensitive; set the file mode to 640 or 600. Add a nonzero exit code if a threshold is exceeded so that a scheduler can alert someone.

Running and maintaining it

Test with a small sample file whose answer you know, an empty file and a missing file. Add error handling for each. Then schedule it with cron, for example `0 6 * * * /usr/local/bin/failed_report.sh`, writing output to a log. Document usage in a comment header and keep the script in version control. Review it as you would any code, since automation that runs with high privileges is itself a target. When requirements grow, such as new log formats, extend the parsing function rather than rewriting. Always keep the original logs untouched; the script should only read them.

CODE

```
from collections import Counter
import re
ip_re = re.compile(r'\b(?:\d{1,3}\.){3}\d{1,3}\b')
counts = Counter()
with open('auth.log') as f:
    for line in f:
        if 'Failed password' in line:
            m = ip_re.search(line)
            if m: counts[m.group()] += 1
for ip, n in counts.most_common(5):
    print(f'{n:5d} {ip}')
```

WORKED EXAMPLE

A small company wants a daily summary of SSH failures. The learner writes the Counter-based script above, tests it on a ten-line sample where she knows the answer, then adds a header with the date and a total. The script writes report.txt with mode 640 and exits with code 2 if any address exceeds 100 attempts. Cron runs it at 06:00, and an alert system watches for exit code 2. Her manual twenty-minute task now takes seconds.

KEY TERMS

Automation: Using scripts to perform repetitive tasks without manual steps.

Counter: A Python collections class that counts items.

most_common: Counter method that returns items ordered by count.

cron: Linux scheduler that runs commands at set times.

Sample data: Small known input used to test a script.

REMEMBER

- Plan the inputs, steps and outputs, then script each step.
- Combine loop, regex and a counter to summarize a log.
- Test with samples, protect the report and schedule with cron.

QUICK CHECK

1. Which Python class counts occurrences easily?
 - A) Counter
 - B) List
 - C) Tuple
 - D) Pipe
2. Which scheduler runs scripts at set times on Linux?
 - A) cron
 - B) grep
 - C) sudo
 - D) ping
3. Why test with a sample file whose answer you know?
 - A) To verify correctness
 - B) To speed it up
 - C) To encrypt data
 - D) To avoid functions
4. Should the script modify the original log?
 - A) Yes, to tidy it
 - B) No, only read it
 - C) Only on Fridays
 - D) Always delete it

Answers: 1-A 2-A 3-A 4-B

1. collections.Counter counts items.
2. cron schedules jobs.
3. A known answer proves the logic.
4. Preserve original evidence.

Fix the Script

SCENARIO

A fictional Python script is meant to flag any host with more than 10 failed logins, but it has a syntax error and a wrong comparison. The editor runs tests after each change.

OBJECTIVE

Repair a Python script with a syntax error and a wrong comparison until it passes the tests.

STEPS IN THE SIMULATOR

- 1 **Do:** Run the tests on the original code

Simulator shows:

SyntaxError: expected ':' at line 3 (if failed > 10)

- 2 **Do:** Add the missing colon after the if condition

Simulator shows:

Syntax OK. Test 2 failed: host with 10 failures was flagged.

- 3 **Do:** Read the condition: if failed >= 10

Simulator shows:

Spot the off-by-one: threshold is 'more than 10'.

- 4 **Do:** Change >= to >

Simulator shows:

Test 2 passes.

- 5 **Do:** Run all tests

Simulator shows:

3/3 tests passed. flag(12)=True, flag(10)=False, flag(0)=False.

ANSWER

The fixes were adding the missing colon after the if line and changing >= to > so that exactly 10 failures is not flagged. All three tests then pass.

Hint shown in the app: Read the error line number and check the comparison operator.

Ping Sweep Loop

SCENARIO

In a fictional lab you must finish a Bash loop that tests addresses 192.0.2.1 to 192.0.2.10 and prints only those that reply. The simulator provides canned ping replies for hosts .1, .4 and .7. Only test authorized lab ranges.

OBJECTIVE

Complete a Bash for loop that pings a host range and prints only the live addresses.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the starter: for i in ____; do ... done

Simulator shows:

Blank for the range.

- 2 **Do:** Fill in {1..10}

Simulator shows:

Loop header valid.

- 3 **Do:** Add ping -c 1 -W 1 192.0.2.\$i > /dev/null

Simulator shows:

Runs, but output prints nothing yet.

- 4 **Do:** Add && echo "192.0.2.\$i up"

Simulator shows:

Output: 192.0.2.1 up, 192.0.2.4 up, 192.0.2.7 up

- 5 **Do:** Run tests

Simulator shows:

Pass: only live hosts printed, no ping noise.

ANSWER

The completed loop is for i in {1..10}; do ping -c 1 -W 1 192.0.2.\$i > /dev/null && echo "192.0.2.\$i up"; done, which prints 192.0.2.1, .4 and .7 as live hosts.

Hint shown in the app: Use && so echo runs only if ping succeeds.

Regex IP Extractor

SCENARIO

A fictional log sample contains several lines with IPv4 addresses, some repeated. Write a regex to extract every address and count the unique ones. The sample includes the text version 1.2.3, which must not match.

OBJECTIVE

Write a regex that extracts every IPv4 address from a sample log and count unique matches.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the sample log

Simulator shows:

6 lines; addresses 203.0.113.9 (3 times), 198.51.100.7, 192.0.2.44; a line says 'version 1.2.3'

- 2 **Do:** Enter pattern `\d+\.\d+\.\d+\.\d+`

Simulator shows:

Matches 7 items but check the version string.

- 3 **Do:** Refine to `\b(?:\d{1,3}\.){3}\d{1,3}\b`

Simulator shows:

Matches 5 items; version 1.2.3 excluded.

- 4 **Do:** Use `findall` and put results in a set

Simulator shows:

Set size: 3

- 5 **Do:** Print `sorted(set)`

Simulator shows:

`['192.0.2.44', '198.51.100.7', '203.0.113.9']`

ANSWER

The pattern `\b(?:\d{1,3}\.){3}\d{1,3}\b` finds five matches in the sample and three unique addresses: 192.0.2.44, 198.51.100.7 and 203.0.113.9. The three-part version number is correctly ignored.

Hint shown in the app: Require exactly four numeric groups separated by dots.

Log Summary Tool

SCENARIO

A fictional auth log lists failed logins for usernames. Finish a Python function `top_failed(path, n)` that returns the `n` usernames with the most failures. A test file holds known data, and the editor runs unit checks.

OBJECTIVE

Finish a Python function that reads a log and prints the top five failed-login usernames.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the starter code

Simulator shows:

```
def top_failed(path, n=5): counts = Counter(); # TODO
```

- 2 **Do:** Add a loop: with `open(path)` as `f`: for `line` in `f`:

Simulator shows:

Syntax OK.

- 3 **Do:** Filter: if 'Failed password' in line:

Simulator shows:

Lines reduced to failures.

- 4 **Do:** Extract the user: `m = re.search(r'for (?:(invalid user )?(S+) from', line)`

Simulator shows:

Group 1 holds the username.

- 5 **Do:** Count and return: `counts[m.group(1)] += 1 ... return counts.most_common(n)`

Simulator shows:

Test output: `[('root', 40), ('admin', 22), ('test', 9), ('oracle', 5), ('guest', 3)]`

- 6 **Do:** Run tests

Simulator shows:

4/4 passed, including empty file.

ANSWER

The function reads each line, keeps lines with Failed password, extracts the username with a regex, counts with Counter and returns `most_common(n)`. On the test file the top names are root 40, admin 22, test 9, oracle 5 and guest 3.

Hint shown in the app: Handle both 'for root' and 'for invalid user admin' forms.

Scripting Basics: 10 questions, 80% to pass

1. Which Bash line sets a variable correctly?
 - A) name = alice
 - B) name=alice
 - C) \$name=alice
 - D) let name alice
2. Which Python structure maps IPs to counts?
 - A) list
 - B) dict
 - C) str
 - D) bool
3. What does exit status 0 mean?
 - A) Failure
 - B) Success
 - C) Warning
 - D) Timeout
4. Which is a Python comparison?
 - A) a = b
 - B) a == b
 - C) a := b only
 - D) a != b
5. What does range(5) give in Python?
 - A) 1 to 5
 - B) 0 to 4
 - C) 0 to 5
 - D) 5 only
6. Which Bash form reads a file line by line?
 - A) for l in \$(cat f)
 - B) while read -r line; do ...; done < f
 - C) cat f | for
 - D) loop f
7. In Bash, \$1 refers to:
 - A) The script name
 - B) The first argument
 - C) The exit code
 - D) The PID
8. What is sys.argv[1]?
 - A) Script name
 - B) First command-line argument
 - C) Python version
 - D) Exit code
9. Which regex matches one or more digits?
 - A) \d+
 - B) \w?
 - C) \s*
 - D) [a-z]
10. Why validate IPs after a regex match?
 - A) Regex only checks shape, not numeric range
 - B) Regex cannot match digits
 - C) IPs are secret
 - D) Validation is faster

Answers: 1-B 2-B 3-B 4-B 5-B 6-B 7-B 8-B 9-A 10-A

1. No spaces around = in Bash.
2. A dict maps keys to values.
3. Zero is success.
4. == compares values.
5. It starts at 0 and excludes 5.
6. while read handles lines intact.
7. \$1 is argument one.
8. Index 1 is first argument.
9. \d+ is one or more digits.
10. 999.1.1.1 matches the shape.

Security Foundations

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

This module gives you the core vocabulary and mental models of security. You will define the CIA triad and its extensions, separate threats, vulnerabilities and risk and score them with likelihood times impact, and map attack surfaces and threat actors. You will then see how layered controls, least privilege and separation of duties limit damage, and how policies, standards, NIST CSF and ISO 27001 turn practice into a managed program. Every later technical module reuses these ideas.

BY THE END YOU CAN

- + Apply the CIA triad to classify security failures
- + Rank risks using threat, vulnerability and impact
- + Choose layered controls and write basic policy statements

IN THIS MODULE

Lesson 1 The CIA Triad and Beyond

Lesson 2 Threats, Vulnerabilities and Risk

Lesson 3 Attack Surface and Threat Actors

Lesson 4 Defense in Depth and Least Privilege

Lesson 5 Security Policies and Frameworks

Lab 1 Classify the Incident (Scenario)

Lab 2 Build a Risk Register (Scenario)

Lab 3 Layer the Defenses (Diagram builder)

Lab 4 Shrink the Attack Surface (Config editor)

Checkpoint 10 questions, 80% to pass

The CIA Triad and Beyond

The CIA triad gives you a shared vocabulary for naming exactly what went wrong in any security incident, which is the first step toward fixing it.

Confidentiality

Confidentiality means information is disclosed only to people and systems authorized to see it. Controls that protect it include encryption, access control lists, authentication and data classification labels such as public, internal and restricted. A confidentiality failure is any unauthorized disclosure: a misconfigured storage bucket that exposes customer files, a laptop stolen with an unencrypted disk, or an employee emailing a payroll sheet to the wrong person. Notice that no data has to be changed or destroyed for confidentiality to fail. The data can be perfectly intact and still be in the wrong hands, which is why breach notification laws focus on disclosure.

Integrity and availability

Integrity means data and systems are accurate, complete and changed only by authorized parties in authorized ways. Typical controls are file hashes, digital signatures, input validation, version control and change management. An integrity failure is a silent or unauthorized modification: an attacker altering a price in a database, malware patching a program, or a bank balance edited by an insider. Availability means systems and data are usable when needed. Redundancy, backups, capacity planning and DDoS protection support it. A ransomware lockout, a power failure, a deleted database and a flooded web server are all availability failures, whether the cause is hostile or accidental.

Authenticity and non-repudiation

Two further properties are often added to the triad. Authenticity means you can verify that a user, message or system is genuinely what it claims to be, using passwords, certificates, multi-factor authentication or message signatures.

Non-repudiation means a party cannot credibly deny an action they performed, such as sending a message or approving a payment. It relies on digital signatures and tamper-resistant audit logs that tie each action to a unique identity. Shared accounts destroy non-repudiation because several people can claim that someone else used the login. Accountability is the related idea that every action must be traceable to an individual.

Classifying a failure in practice

To classify an incident, ask three questions in order. Was information seen by someone who should not see it (confidentiality)? Was it altered without permission (integrity)? Could authorized users no longer reach it (availability)? Many incidents break more than one goal. Double extortion ransomware first copies data, breaking confidentiality, then encrypts it, breaking availability. The goals can also conflict: strict access controls improve confidentiality but may slow legitimate work, and heavy replication improves availability but creates more copies to protect. Security design is therefore a balance based on what the business values most, such as integrity for financial ledgers and availability for hospital systems.

TRIAD AND EXTENSIONS

```
C Confidentiality -> encryption, access control
I Integrity -> hashes, signatures, change control
A Availability -> backups, redundancy, DDoS defense
+ Authenticity -> MFA, certificates
+ Non-repudiation -> signatures, audit logs
```

WORKED EXAMPLE

At the fictional company Example Retail, an analyst finds three events in one week. First, a public web folder at `www.example.com/backup` lists customer order files. Second, a database record shows a product priced at 0.01 instead of 49.99, edited outside the change window. Third, the checkout site is unreachable for two hours during a traffic flood. The three events map cleanly to the triad: the exposed folder is a confidentiality failure, the altered price is an integrity failure, and the outage is an availability failure. Each needs a different fix.

KEY TERMS

Confidentiality: Only authorized parties can view information.

Integrity: Information is accurate and altered only by authorized parties.

Availability: Systems and data are usable when needed.

Authenticity: Assurance that an identity or message is genuine.

Non-repudiation: A party cannot credibly deny having performed an action.

REMEMBER

- The triad names three goals: confidentiality, integrity, availability.
- Authenticity and non-repudiation extend it to identity and accountability.
- One incident can break several goals, and controls often trade off against each other.

QUICK CHECK

1. A stolen laptop with an unencrypted disk is primarily which failure?
 - A) Integrity
 - B) Availability
 - C) Confidentiality
 - D) Non-repudiation
2. An attacker silently changes a price in a database. Which goal failed?
 - A) Confidentiality
 - B) Integrity
 - C) Availability
 - D) Authenticity of the server
3. Why do shared administrator accounts weaken non-repudiation?
 - A) They are always encrypted
 - B) Actions cannot be tied to one individual
 - C) They are too slow
 - D) They require MFA
4. Double extortion ransomware breaks which goals?
 - A) Only availability
 - B) Only integrity
 - C) Confidentiality and availability
 - D) Authenticity only

Answers: 1-C 2-B 3-B 4-C

1. Data may be read by an unauthorized person, which is disclosure.
2. Unauthorized modification is an integrity failure.
3. With a shared login, several people could have performed the action.
4. Data is copied (confidentiality) and then encrypted (availability).

Threats, Vulnerabilities and Risk

Risk language lets you rank problems objectively so limited time and money go to the issues that matter most.

Four terms, kept separate

A vulnerability is a weakness, such as unpatched software, a default password or a missing check in code. A threat is anything that could exploit a weakness and cause harm: a criminal group, a careless employee, a flood. An exploit is the specific technique or tool that takes advantage of a vulnerability. Risk is the potential for loss when a threat meets a vulnerability and affects an asset. A vulnerability with no threat able to reach it is low risk, and a strong threat facing no vulnerability is also low risk. Confusing these terms leads to bad decisions, such as spending heavily on a scary-sounding threat while ignoring a trivial and exposed weakness.

Likelihood times impact

The classic quantitative-style shortcut is $\text{risk} = \text{likelihood} \times \text{impact}$. Teams commonly score each factor from 1 to 5, giving a result from 1 to 25. A 4 for likelihood and a 5 for impact gives 20, which is critical. Likelihood depends on threat capability, exposure and how easy the vulnerability is to exploit. Impact depends on asset value and the effect on confidentiality, integrity, availability, money, law and reputation. Scores are estimates, so use consistent definitions: for example, impact 5 might mean more than 1 million in loss or a regulatory penalty. Consistency matters more than precision.

Quantitative measures

Quantitative analysis uses money. Single loss expectancy (SLE) is asset value times exposure factor, the fraction of the asset lost in one event. Annualized rate of occurrence (ARO) is how many times per year the event is expected. Annualized loss expectancy (ALE) equals SLE times ARO. If a server worth 100,000 would lose 50 percent of its value in an incident (SLE 50,000) and such incidents are expected once every five years (ARO 0.2), the ALE is 10,000. A control costing 30,000 per year to prevent a 10,000 per year loss is hard to justify on cost alone.

Treating risk

Once risk is scored, an organization chooses one of four responses. Mitigate by adding controls that reduce likelihood or impact, such as patching. Transfer by shifting financial impact, usually through cyber insurance or outsourcing. Avoid by ceasing the risky activity, for example not storing card numbers at all. Accept by knowingly tolerating the risk, which should be documented and signed by someone with authority. What remains after controls is residual risk. Findings are recorded in a risk register with an owner, a score, a treatment and a review date so that nothing is forgotten.

RISK REGISTER ROW AND ALE

Finding : OpenSSH 7.x exposed on 192.0.2.10
Likelihood 4 x Impact 4 = Risk 16 (High)
Treatment: Mitigate - upgrade, restrict source IPs

$\text{SLE} = 100000 \times 0.5 = 50000$
 $\text{ARO} = 0.2$
 $\text{ALE} = 50000 \times 0.2 = 10000$

WORKED EXAMPLE

A small firm finds two issues on the same day. A public file server at 192.0.2.20 runs unpatched software with a known remote flaw and holds client contracts. A forgotten test printer has a default password but sits on an isolated segment with no sensitive data. The file server scores likelihood 4 and impact 5, risk 20. The printer scores likelihood 2 and impact 1, risk 2. Both are real vulnerabilities, but the register shows that the file server must be fixed this week while the printer can wait for routine maintenance.

KEY TERMS

Vulnerability: A weakness that could be exploited.

Threat: A potential cause of harm, such as an actor or event.

Exploit: A method or tool that takes advantage of a vulnerability.

Risk: Likelihood of loss multiplied by its impact.

Residual risk: Risk remaining after controls are applied.

REMEMBER

- Threat, vulnerability, exploit and risk are four different things.
- Risk = likelihood x impact; ALE = SLE x ARO.
- Treat risk by mitigating, transferring, avoiding or accepting, and track it in a register.

QUICK CHECK

1. Which best describes a vulnerability?
 - A) A hacker group
 - B) A weakness that could be exploited
 - C) A stolen password
 - D) A security policy
2. Likelihood 3 and impact 5 give what score?
 - A) 8
 - B) 15
 - C) 35
 - D) 2
3. An ALE is calculated as which product?
 - A) SLE x ARO
 - B) Asset value x ARO only
 - C) Likelihood + impact
 - D) SLE / ARO
4. Buying cyber insurance is an example of which risk response?
 - A) Avoid
 - B) Accept
 - C) Transfer
 - D) Eliminate

Answers: 1-B 2-B 3-A 4-C

1. A vulnerability is a weakness; the group is a threat.
2. Risk = $3 \times 5 = 15$.
3. Annualized loss expectancy equals single loss expectancy times annual rate of occurrence.
4. Insurance shifts financial impact to a third party.

Attack Surface and Threat Actors

Knowing what is exposed and who is likely to attack it tells you where to focus defenses.

What the attack surface is

The attack surface is the sum of all points where an unauthorized party could try to enter, extract data or cause harm. It includes network surface, such as open ports and internet-facing services; software surface, such as web forms, APIs and installed packages; human surface, such as staff who can be phished; and physical surface, such as USB ports and unlocked server rooms. Cloud adoption, remote work and third-party integrations widen it. Shadow IT, systems deployed without security knowing, is a common blind spot. You cannot defend what you do not know exists, so asset inventory and regular scanning are the starting point.

Reducing the surface

Reduction follows a simple order: remove, restrict, then protect. Remove services you do not need, such as an old FTP daemon listening on port 21 or a Telnet service on port 23. Restrict what remains by limiting access to required source addresses with firewall rules and network segmentation. Protect what must stay exposed with patching, strong authentication and monitoring. Disabling unused accounts, closing unneeded ports, removing sample applications and uninstalling unused software all shrink exposure. Every feature is potential attack surface, which is why secure defaults and minimal installations are recommended by hardening guides such as the CIS Benchmarks.

Threat actors and motives

Threat actors differ in skill, resources and motive. Script kiddies run publicly available tools with little understanding, usually for notoriety. Hacktivists attack to make a political or social point, often through defacement or DDoS. Cybercriminals seek profit through ransomware, fraud and data resale. Insiders, malicious or careless, already hold legitimate access. Competitors may steal trade secrets. Nation-state groups, often called advanced persistent threats (APTs), are well funded, patient and focused on espionage or disruption. Knowing the likely actor shapes expectations: a small shop faces mostly opportunistic criminals, while a defense contractor must also plan for APTs.

Vectors and modeling

An attack vector is the route used to reach a target, such as phishing email, an exposed remote desktop service on port 3389, a vulnerable web application, a malicious USB drive or a compromised supplier. Pairing actors with vectors produces a threat model, a structured answer to who might attack, how, and what they want. Simple models list assets, entry points and plausible actors, then rank them. Frameworks such as STRIDE help brainstorm threats per component. Models should be revisited whenever the system changes, for example when a new public API or vendor connection is added.

QUICK EXPOSURE REVIEW

```
Host 192.0.2.30 open ports:
22/tcp ssh (needed, restrict to admin IPs)
23/tcp telnet (remove)
80/tcp http (redirect to 443)
443/tcp https (needed)
3389/tcp rdp (remove from internet)
```

WORKED EXAMPLE

Example Corp scans its public range and finds that a web server at 198.51.100.15 exposes ports 22, 23, 80, 443 and 3389. Telnet and RDP are not needed by outsiders. The team removes Telnet, moves RDP behind a VPN, limits SSH to the office address, and keeps only ports 80 and 443 public. The attack surface shrinks from five exposed services to two, and the remaining two are covered by patching and monitoring. The likely actors are opportunistic criminals running automated scans, so the highest-value change was removing easy targets.

KEY TERMS

Attack surface: All the points where an attacker could attempt entry or harm.

Attack vector: The path or method used to reach a target.

APT: Advanced persistent threat; a skilled, well-resourced, long-running attacker.

Insider threat: Risk from someone with legitimate access.

Shadow IT: Systems used without the knowledge of the security team.

REMEMBER

- The attack surface covers network, software, human and physical entry points.
- Remove, restrict, then protect to shrink it.
- Actors differ in motive and skill; match defenses to the likely actor.

QUICK CHECK

1. Which step should come first when reducing attack surface?
 - A) Add monitoring
 - B) Remove unneeded services
 - C) Buy insurance
 - D) Write a slogan
2. Which actor is typically well funded, patient and focused on espionage?
 - A) Script kiddie
 - B) Nation-state APT
 - C) Hacktivist
 - D) Careless user
3. An exposed RDP service is an example of what?
 - A) A network attack surface item
 - B) A policy
 - C) A hash
 - D) A residual risk
4. What does a threat model primarily answer?
 - A) Who might attack, how, and what they want
 - B) Which firewall is cheapest
 - C) How to write code
 - D) How to compress logs

Answers: 1-B 2-B 3-A 4-A

1. Removing what you do not need eliminates exposure entirely.
2. APTs are state-linked groups with resources and long timelines.
3. It is an internet-reachable service an attacker can probe.
4. It pairs assets, entry points and plausible actors.

Defense in Depth and Least Privilege

No single control is perfect, so layered controls and tight permissions limit the damage when one layer fails.

Layers of defense

Defense in depth places several independent controls between an attacker and an asset so that one failure does not mean compromise. Common layers are physical (locks, badges), network (firewalls, segmentation, IDS), host (patching, endpoint detection and response, disk encryption), application (input validation, secure coding), data (encryption, backups) and people (training, clear reporting). The layers should be diverse, since two controls with the same weakness provide little extra protection. Think of an email attack: a mail filter, a user who hesitates, endpoint protection that blocks the payload and MFA that stops stolen credentials each give a separate chance to stop it.

Preventive, detective, corrective

Controls are also grouped by function. Preventive controls stop an event: firewalls, MFA, patches, access control. Detective controls reveal an event: log monitoring, intrusion detection, file integrity checks, CCTV. Corrective controls restore normal state after an event: backups, incident response, patch rollouts. Related categories include deterrent controls such as warning banners, compensating controls that substitute when the ideal control is impractical, and directive controls such as policy. A balanced design uses all of them. Heavy prevention with no detection leaves you blind, and detection with no corrective capability leaves you watching the damage.

Least privilege

The principle of least privilege says every user, process and system should have only the access needed for its job and only for as long as needed. A help-desk worker may reset passwords but should not read finance files. A web server process should run as an unprivileged account, not root or Administrator. Practices include role-based access control, just-in-time elevation, separate admin and everyday accounts, and periodic access reviews to remove privilege creep, where permissions pile up as people change roles. If an account with minimal rights is compromised, the attacker inherits very little.

Separation of duties and related ideas

Separation of duties splits a sensitive task among several people so that no single person can complete fraud or a major mistake alone. One employee requests a payment and another approves it, or one developer writes code and another reviews it before release. Job rotation and mandatory vacation expose hidden misconduct. Need to know limits information to those whose duties require it, even when their clearance would permit more. Fail-safe defaults mean access is denied unless explicitly granted. Together with defense in depth, these principles make a system resilient to both attackers and honest errors.

LAYERS ON ONE ATTACK PATH

```
Phish email -> [mail filter] -> [user report] -> [EDR on host]
-> [MFA on login] -> [segmentation] -> [logging/SIEM]
-> [offline backup] (corrective)
```

WORKED EXAMPLE

At Example Corp a user clicks a phishing link and enters a password on a fake page. The mail filter missed it, but MFA blocks the attacker's login because the second factor is missing. Had the login worked, the account was a standard user with no admin rights, so the attacker could not install tools. Log monitoring would have flagged the unusual sign-in location, and offline backups protected against any later ransomware. Five independent layers, none perfect alone, turned a likely breach into a reported event.

KEY TERMS

Defense in depth: Using multiple independent layers of controls.

Least privilege: Granting only the minimum access needed, for the minimum time.

Separation of duties: Splitting sensitive tasks among several people.

Preventive control: A control that stops an event from occurring.

Privilege creep: Accumulation of unneeded permissions over time.

REMEMBER

- Layer preventive, detective and corrective controls.
- Least privilege limits what a compromised account can do.
- Separation of duties prevents any one person completing fraud alone.

QUICK CHECK

1. Which is a detective control?
 - A) Firewall rule
 - B) Log monitoring alert
 - C) Backup restore
 - D) Warning banner
2. A user keeps old permissions after changing departments. This is called?
 - A) Separation of duties
 - B) Privilege creep
 - C) Defense in depth
 - D) Fail-safe default
3. Requiring one person to request and another to approve a payment applies?
 - A) Separation of duties
 - B) Non-repudiation
 - C) Hashing
 - D) Transfer of risk
4. Why should layers be diverse?
 - A) To look complex
 - B) So one shared weakness does not defeat all layers
 - C) To reduce logging
 - D) To avoid patching

Answers: 1-B 2-B 3-A 4-B

1. Monitoring reveals events after or as they occur.
2. Permissions accumulate beyond current need.
3. No single person can complete the sensitive task alone.
4. Independent controls fail in different ways.

Security Policies and Frameworks

Policies and frameworks turn good intentions into repeatable, auditable practice that an entire organization can follow.

The policy hierarchy

Organizations document security in a hierarchy. A policy is a high-level, mandatory statement of management intent, for example 'All laptops must be encrypted.' A standard states specific measurable requirements that support the policy, such as 'Use AES-256 full-disk encryption.' A procedure gives step-by-step instructions for performing a task, and a guideline offers recommended but optional advice. Policies change rarely, while procedures change often as tools change. A good policy statement is clear, enforceable and assigned to an owner, and it says who must comply, what is required, and what happens on violation.

Common policies

Typical policies include an acceptable use policy defining proper use of company systems, an access control policy, a password or authentication policy, a data classification and handling policy, a remote work policy, an incident response policy and a backup and retention policy. Policies must be approved by senior management, communicated to staff, acknowledged in writing and reviewed at least annually. A policy no one has read cannot be enforced. Exceptions should go through a documented approval so that deviations are visible. Policies also support legal defense, because they show the organization defined expected behavior before an incident.

NIST CSF and ISO 27001

Frameworks provide a tested structure so you do not invent your program from nothing. The NIST Cybersecurity Framework organizes work into functions: Identify, Protect, Detect, Respond and Recover, with version 2.0 adding Govern. It is flexible and widely used as a planning and communication tool. ISO/IEC 27001 specifies requirements for an information security management system (ISMS) and allows an organization to be independently certified. Its Annex A lists controls, selected through risk assessment and recorded in a Statement of Applicability. NIST SP 800-53 is a detailed control catalog, and the CIS Controls give a prioritized practical list.

Compliance and improvement

Compliance means meeting external requirements such as PCI DSS for card data, HIPAA for US health information, or GDPR for personal data of people in the EU. Being compliant is not the same as being secure: compliance sets a minimum baseline while security addresses real threats. Frameworks work as a cycle. Assess the current state, find gaps, set priorities, implement controls, measure and repeat, which ISO expresses as Plan, Do, Check, Act. Audits, internal or external, test whether documented controls actually operate. Evidence such as logs, tickets and screenshots proves that practice matches policy.

POLICY TO PROCEDURE CHAIN

```
POLICY : Laptops must be encrypted.  
STANDARD : Use AES-256 full-disk encryption.  
PROCEDURE : 1. Open disk settings 2. Enable encryption  
              3. Store recovery key in the vault  
GUIDELINE : Reboot weekly to apply patches.
```

WORKED EXAMPLE

Example Corp leadership signs a policy that all remote access must use multi-factor authentication. The security team writes a standard saying that MFA must use an authenticator app or hardware key, not SMS alone. The help desk then follows a procedure for enrolling new users. During an annual audit, the auditor samples ten accounts and checks the logs for MFA events as evidence. Because the chain from policy to evidence is documented, the company can show it meets its own rule and map the work to the Protect function of NIST CSF.

KEY TERMS

Policy: A high-level mandatory statement of management intent.

Standard: A specific measurable requirement supporting a policy.

Procedure: Step-by-step instructions to perform a task.

NIST CSF: Framework of Govern, Identify, Protect, Detect, Respond, Recover.

ISMS: Information security management system, the basis of ISO 27001.

REMEMBER

- Policy states intent, standards set measures, procedures give steps.
- NIST CSF and ISO 27001 give structure for a security program.
- Compliance is a baseline; audits check practice against documents.

QUICK CHECK

1. Which document gives step-by-step instructions?

- A) Policy
- B) Standard
- C) Procedure
- D) Framework

2. Which is an NIST CSF function?

- A) Detect
- B) Encrypt
- C) Sign
- D) Route

3. ISO 27001 certification applies to what?

- A) A single product
- B) An information security management system
- C) A password
- D) A firewall model

4. Why is compliance not the same as security?

- A) It is a minimum baseline, not a defense against every threat
- B) It is illegal
- C) It has no audits
- D) It only covers hardware

Answers: 1-C 2-A 3-B 4-A

1. Procedures describe exactly how to carry out a task.
2. Identify, Protect, Detect, Respond and Recover (plus Govern in 2.0).
3. Organizations are certified against ISMS requirements.
4. Meeting requirements does not guarantee protection from current threats.

Classify the Incident

SCENARIO

You are an analyst at the fictional Example Corp reviewing eight incident summaries from the past quarter. The ticket system asks you to tag each with the security goal that failed: confidentiality, integrity or availability. Some incidents involve more than one goal, so you must choose the primary failure and note the secondary one in the comment field. Work through each summary slowly, and ask what happened to the data or service before choosing.

OBJECTIVE

Label each of eight incidents with the primary CIA failure.

STEPS IN THE SIMULATOR

- 1 Do:** Open incident 1: a public storage bucket exposed customer files.

Simulator shows:

The tag menu shows C, I, A.

- 2 Do:** Tag incident 1 as confidentiality.

Simulator shows:

Correct. Unauthorized disclosure.

- 3 Do:** Open incident 2: a ransomware attack locked the file server.

Simulator shows:

Summary shown with a ransom note.

- 4 Do:** Tag incident 2 as availability, with confidentiality as secondary if data was copied.

Simulator shows:

Correct. Files unusable.

- 5 Do:** Tag incident 3, an employee changing invoice amounts, as integrity.

Simulator shows:

Correct. Unauthorized modification.

- 6 Do:** Tag incident 4, a DDoS against the web shop, as availability.

Simulator shows:

Correct. Service unreachable.

- 7 Do:** Tag incident 5, a lost unencrypted USB stick, as confidentiality, and incident 6, tampered log files, as integrity.

Simulator shows:

Both correct.

- 8 Do:** Tag incident 7, a power outage in the data center, as availability, and incident 8, a phishing credential theft, as confidentiality.

Simulator shows:

Score 8 of 8.

ANSWER

Confidentiality: exposed bucket, lost USB, phishing credential theft. Integrity: altered invoices, tampered logs. Availability: ransomware lockout, DDoS, power outage. Ransomware that also copies data adds a confidentiality failure as a secondary tag.

Hint shown in the app: Ask whether data was seen, changed or made unreachable.

Build a Risk Register

SCENARIO

Example Corp has five findings from a recent scan and review. The simulator provides a spreadsheet with columns for asset, likelihood, impact and score. You rate likelihood and impact from 1 to 5 using the definitions on the side panel, multiply them, and sort the register from highest to lowest risk. Finally you choose a treatment for the top item: mitigate, transfer, avoid or accept. Be consistent in how you score similar findings.

OBJECTIVE

Score five findings and sort them into a prioritized register.

STEPS IN THE SIMULATOR

- 1 Do:** Open finding 1: an unpatched file server at 192.0.2.20 holding contracts.
Simulator shows:
Definitions panel appears.

- 2 Do:** Score it likelihood 4, impact 5.
Simulator shows:
Risk shows 20.

- 3 Do:** Score finding 2, a default password on an isolated test printer, as 2 and 1.
Simulator shows:
Risk shows 2.

- 4 Do:** Score finding 3, no MFA on the VPN, as 4 and 4.
Simulator shows:
Risk shows 16.

- 5 Do:** Score finding 4, an expired internal wiki certificate, as 3 and 2, and finding 5, no offline backups, as 3 and 5.
Simulator shows:
Risks show 6 and 15.

- 6 Do:** Click Sort descending.
Simulator shows:
Order: 20, 16, 15, 6, 2.

- 7 Do:** Choose Mitigate for the top item and add an owner and date.
Simulator shows:
Register saved.

ANSWER

The sorted register is file server (20), VPN without MFA (16), no offline backups (15), expired certificate (6), test printer (2). The top item is mitigated by patching and restricting access. Scores are estimates, but consistent scale use keeps the order defensible.

Hint shown in the app: Multiply likelihood by impact, then sort high to low.

Layer the Defenses

SCENARIO

A diagram shows an attack path at Example Corp: a phishing email reaches a workstation, the attacker steals a password, logs in remotely, moves to a file server and encrypts data. You are given five control icons: firewall, EDR, MFA, backups and logging. Your task is to drag each icon to the point on the path where it helps most, and label it as preventive, detective or corrective. Aim for coverage of every stage.

OBJECTIVE

Place five controls on the attack path and label their types.

STEPS IN THE SIMULATOR

- 1 **Do:** Look at the attack path stages shown on the canvas.

Simulator shows:

Five stages are highlighted.

- 2 **Do:** Drag the firewall to the network boundary.

Simulator shows:

Label as preventive; it blocks unneeded inbound ports.

- 3 **Do:** Drag EDR to the workstation.

Simulator shows:

Label as preventive and detective.

- 4 **Do:** Drag MFA to the remote login step.

Simulator shows:

Label as preventive; stolen password alone fails.

- 5 **Do:** Drag logging to the file server and network segments.

Simulator shows:

Label as detective.

- 6 **Do:** Drag backups to the data store.

Simulator shows:

Label as corrective.

- 7 **Do:** Click Check coverage.

Simulator shows:

All stages covered, no single point of failure.

ANSWER

Firewall at the boundary, EDR on the workstation, MFA on the remote login, logging on servers and network, and offline backups on the data store. This gives preventive, detective and corrective coverage so one failing layer does not complete the attack.

Hint shown in the app: Match each control to the stage it can stop, notice or repair.

Shrink the Attack Surface

SCENARIO

You are hardening a fictional server at 203.0.113.10 for Example Corp. The config editor shows a services list, a firewall allow list and a user account section. The server only needs to host a website, but it also runs FTP, Telnet and a database listener open to the internet. A service account runs as root. Edit the configuration to remove or restrict everything that is not needed and run the validator.

OBJECTIVE

Disable unneeded services and ports and set a minimal-privilege account.

STEPS IN THE SIMULATOR

- 1 **Do:** Open services.conf and review the enabled list.

Simulator shows:

Services: ssh, ftp, telnet, http, https, mysql.

- 2 **Do:** Set ftp and telnet to disabled.

Simulator shows:

Validator notes two services removed.

- 3 **Do:** Change the firewall rule for port 3306 from any to 10.0.0.0/24 or deny.

Simulator shows:

Database no longer internet exposed.

- 4 **Do:** Restrict ssh to the admin address 198.51.100.5.

Simulator shows:

Rule updated.

- 5 **Do:** Open accounts.conf and change webapp from root to a user named webapp with no shell.

Simulator shows:

Privilege level shows standard.

- 6 **Do:** Click Validate.

Simulator shows:

Open ports: 80, 443, and 22 from one address.

- 7 **Do:** Review the score.

Simulator shows:

Attack surface reduced from six services to three.

ANSWER

Disable FTP and Telnet, restrict MySQL to an internal range, limit SSH to one admin address, keep only 80 and 443 public, and run the web application under an unprivileged account with no shell. This applies remove, restrict, protect and least privilege together.

Hint shown in the app: Remove first, then restrict what must stay, then lower privilege.

Security Foundations: 10 questions, 80% to pass

1. A database record is silently changed by an attacker. Which goal failed?

- A) Confidentiality
- B) Integrity
- C) Availability
- D) Accountability only

2. Which pair is correctly matched?

- A) Threat: unpatched server
- B) Vulnerability: criminal group
- C) Exploit: technique that uses a weakness
- D) Risk: firewall

3. Likelihood 5 and impact 4 give a risk score of?

- A) 9
- B) 20
- C) 1
- D) 54

4. ALE equals?

- A) $SLE \times ARO$
- B) $SLE + ARO$
- C) Asset value / ARO
- D) Impact only

5. Which risk response is removing a risky activity entirely?

- A) Accept
- B) Transfer
- C) Avoid
- D) Mitigate

6. Removing an unused Telnet service is mainly which activity?

- A) Reducing attack surface
- B) Increasing availability
- C) Signing data
- D) Transferring risk

7. Backups are what type of control?

- A) Preventive only
- B) Corrective
- C) Deterrent
- D) Directive

8. Giving a web server process only the permissions it needs applies?

- A) Least privilege
- B) Separation of duties
- C) Non-repudiation
- D) Risk transfer

9. Which statement is a standard rather than a policy?

- A) Data must be protected
- B) Use TLS 1.2 or higher for all web traffic
- C) Security is everyone's job
- D) Be careful online

10. Which actor is best described as a nation-state APT?

- A) Bored teenager with a tool
- B) Well-funded long-term espionage group
- C) Disgruntled competitor only
- D) Automated scanner

Answers: 1-B 2-C 3-B 4-A 5-C 6-A 7-B 8-A 9-B 10-B

1. Unauthorized modification breaks integrity.
2. An exploit takes advantage of a vulnerability.
3. $5 \times 4 = 20$.
4. Annualized loss expectancy equals SLE times ARO.
5. Avoidance stops the activity that creates the risk.
6. Fewer exposed services means a smaller surface.
7. They restore normal state after an event.
8. Minimum rights limit damage from compromise.
9. Standards are specific and measurable.
10. APTs are resourced, patient and goal-driven.